

INFORMATIQUE • TLE TI

MODULE 1 : PROGRAMMATION

LEÇON N01

RAPPELS DE PROGRAMMATION
WEB : HTML ET JAVASCRIPT

DURÉE CONSEILLÉE : 6 H

DANS CETTE LEÇON

Cette leçon réactive les acquis de Première sur la structure d'une page HTML, les formulaires et les tableaux, puis introduit la sélection d'éléments du DOM et la gestion d'événements en JavaScript. Elle aboutit au contrôle de la saisie utilisateur côté client, compétence clé pour le développement web au programme de Terminale TI.

- À la fin de cette leçon, je sais décrire la structure minimale d'une page HTML5 (doctype, html, head, body).
- À la fin de cette leçon, je sais créer un formulaire HTML complet avec champs de saisie, listes déroulantes, zones de texte, étiquettes et boutons.
- À la fin de cette leçon, je sais construire un tableau HTML structuré avec table, tr, th et td.
- À la fin de cette leçon, je sais sélectionner un élément du DOM en JavaScript (getElementById, querySelector).
- À la fin de cette leçon, je sais lire et modifier le contenu ou la valeur d'un élément HTML depuis JavaScript.
- À la fin de cette leçon, je sais associer un gestionnaire d'événement à une action de l'utilisateur (clic, saisie, soumission).

TÉLÉCHARGE
ONBUCH

Gratuit • Google Play
Tuteur IA, annales,
concours, résultats



PASSE À ONBUCH+

Tous les cours signés, Tuteur IA
illimité, fiches PDF — onglet
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

COURS

complet, illustré, pas à
pas

MÉTHODES

et exercices résolus

EXERCICES

type examen + corrigés

FICHE BILAN

l'essentiel à retenir



Signé • OnBuch+

Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

SOMMAIRE

- | | |
|--|----------------------------------|
| 1. Structure d'une page HTML | 7. Activité d'intégration |
| 2. Les formulaires HTML | 8. Méthodes et exercices résolus |
| 3. Les tableaux HTML | 9. Exercices |
| 4. JavaScript et le DOM | 10. Corrigés des exercices |
| 5. Gestion des événements | 11. Fiche bilan |
| 6. Validation de la saisie côté client | |

OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais décrire la structure minimale d'une page HTML5 (doctype, html, head, body).
- À la fin de cette leçon, je sais créer un formulaire HTML complet avec champs de saisie, listes déroulantes, zones de texte, étiquettes et boutons.
- À la fin de cette leçon, je sais construire un tableau HTML structuré avec table, tr, th et td.
- À la fin de cette leçon, je sais sélectionner un élément du DOM en JavaScript (getElementById, querySelector).
- À la fin de cette leçon, je sais lire et modifier le contenu ou la valeur d'un élément HTML depuis JavaScript.
- À la fin de cette leçon, je sais associer un gestionnaire d'événement à une action de l'utilisateur (clic, saisie, soumission).
- À la fin de cette leçon, je sais valider un champ de formulaire côté client (champ vide, format numérique, longueur minimale) et afficher un message d'erreur.
- À la fin de cette leçon, je sais empêcher l'envoi d'un formulaire invalide avec preventDefault().

PRÉREQUIS

- Notions de base de HTML vues en Première (balises, attributs, imbrication)
- Notions élémentaires de JavaScript : variables, types, conditions if/else, fonctions
- Utilisation d'un éditeur de texte et d'un navigateur web

- Notion d'algorithme et de test (structures conditionnelles)

1 STRUCTURE D'UNE PAGE HTML

Situation d'accroche. Aminatou, élève de Terminale TI à Bafoussam, veut créer un site web pour la coopérative de maïs de son quartier : chaque membre doit saisir en ligne ses apports (nom, quantité en sacs, numéro de téléphone). Mais lors des premiers tests, tout se passe mal : certains saisissent des lettres à la place des quantités, d'autres oublient leur numéro, et le tableau récapitulatif s'affiche n'importe comment. Avant même de parler de contrôle de saisie, Aminatou réalise qu'elle doit d'abord **structurer correctement sa page**. C'est le point de départ de cette leçon : comment une page web est-elle construite, balise par balise, pour que le navigateur l'affiche fidèlement ?

Problématique. Quelles sont les règles de construction d'une page HTML valide, et comment organiser son contenu pour qu'il soit correctement interprété par un navigateur ?

1.1 HTML : un langage de balisage, pas de programmation

Avant d'écrire la moindre ligne, il faut comprendre la nature de HTML. Quand Aminatou tape du texte dans un traitement de texte, elle met en forme visuellement. En HTML, elle ne dessine rien : elle décrit le contenu en l'entourant de **balises**, et c'est le navigateur (Chrome, Firefox, Safari...) qui interprète cette description pour produire l'affichage.

DÉFINITION HTML

HTML (HyperText Markup Language, langage de balisage hypertexte) est un **langage de balisage** qui sert à décrire la structure et le contenu d'une page web à l'aide de balises. Ce n'est pas un langage de programmation : il ne contient ni variables, ni conditions, ni boucles, et n'exécute aucun calcul. La version actuelle est HTML5.

ATTENTION HTML n'est pas un langage de programmation

Au Baccalauréat, une question classique demande de distinguer HTML d'un langage comme JavaScript ou PHP. Retiens : HTML décrit (statique, déclaratif), JavaScript agit (dynamique, algorithmique). Dire « j'ai programmé en HTML » est un abus de langage : on écrit ou on code une page HTML.

1.2 Balises, attributs et éléments

Une page HTML est un texte dans lequel on insère des marqueurs. Trois notions de vocabulaire sont indispensables et souvent confondues.

DÉFINITION Balise, attribut, élément

- Une **balise** est un marqueur écrit entre chevrons : `<p>`. La plupart des balises vont par paire : une balise ouvrante `<p>` et une balise fermante `</p>` (avec une barre oblique). Certaines balises sont dites orphelines (auto-fermantes) : `<br>`, `<img ...>`, `<input ...>`, `<meta ...>`.
- Un **attribut** est une information complémentaire placée dans la balise ouvrante, sous la forme `nom="valeur"`. Exemples : `id`, `class`, `name`, `src`.
- Un **élément** HTML est l'ensemble complet : balise ouvrante + contenu + balise fermante. Exemple : `<p>Bonjour</p>` est un élément de type paragraphe.

EXEMPLE Anatomie d'un élément HTML

```
<p class="intro" id="parag1">Bienvenue a la cooperative !</p>
```

Ici : `<p ...>` est la balise ouvrante, `class="intro"` et `id="parag1"` sont deux attributs, « Bienvenue à la coopérative ! » est le contenu, `</p>` est la balise fermante. Le tout forme un élément `p`.

Trois attributs reviennent constamment, surtout dès qu'on manipulera JavaScript :

Attribut	Rôle	Particularité
<code>id</code>	Identifiant de l'élément	Unique dans toute la page; sert à cibler l'élément en JavaScript (<code>getElementById</code>)
<code>class</code>	Étiquette de catégorie	Peut être partagée par plusieurs éléments; sert surtout au style CSS
<code>name</code>	Nom d'un champ de formulaire	C'est sous ce nom que la donnée saisie est envoyée au serveur

ATTENTION Pièges sur les attributs

- Deux éléments ne doivent jamais porter le même `id` dans une page : JavaScript ne saurait plus lequel sélectionner.
- Les valeurs d'attributs s'écrivent entre guillemets : `id="nom"` et non `id=nom`.
- Les noms de balises et d'attributs s'écrivent en minuscules par convention en HTML5.

1.3 Le squelette minimal d'une page HTML5

Toute page HTML5 valide repose sur la même armature. Le navigateur s'attend à trouver des

zones bien précises : une zone d'en-tête (invisible, destinée aux machines) et un corps (visible, destiné aux visiteurs).

MÉTHODE Écrire le squelette minimal d'une page HTML5 valide

1. Commencer par la déclaration `<!DOCTYPE html>` sur la toute première ligne : elle annonce au navigateur que le document est en HTML5.
2. Ouvrir l'élément racine `<html lang="fr">` : tout le reste de la page vit à l'intérieur.
3. Créer l'en-tête `<head>` et y placer au minimum `<meta charset="UTF-8">` (encodage des caractères, indispensable pour les accents français) et `<title>` (titre affiché dans l'onglet du navigateur).
4. Créer le corps `<body>` : c'est là que va tout le contenu visible (titres, textes, images, formulaires, tableaux).
5. Fermer les balises dans l'ordre inverse de leur ouverture : `</body>` puis `</html>`.
6. Enregistrer le fichier avec l'extension `.html` (par exemple `index.html`) et l'ouvrir dans un navigateur pour vérifier.

EXEMPLE Squelette minimal commenté

```
<!DOCTYPE html>    <!-- 1. Declaration : document HTML5 -->
<html lang="fr">    <!-- 2. Element racine, langue francaise -->
<head>             <!-- 3. En-tete : invisible pour le visiteur -->
  <meta charset="UTF-8"> <!-- Encodage : accents corrects -->
  <title>Cooperative de Bafoussam</title> <!-- Titre de l'onglet -->
</head>
<body>             <!-- 4. Corps : tout le contenu visible -->
  <h1>Bienvenue a la cooperative</h1>
  <p>Premier paragraphe de la page.</p>
</body>            <!-- 5. Fermeture dans l'ordre inverse -->
</html>
```

Le `<head>` peut contenir bien plus que le titre et l'encodage. C'est notamment là qu'on relie la page à ses ressources externes :

- une feuille de style CSS : `<link rel="stylesheet" href="style.css">`;
- un script JavaScript : `<script src="controle.js"></script>` (souvent placé en fin de `<body>` pour que la page soit chargée avant l'exécution du script);
- des méta-informations pour le référencement ou l'affichage mobile : `<meta name="viewport" content="width=device-width, initial-scale=1.0">`.

ATTENTION Oublier `<meta charset="UTF-8">`

Sans cette balise, les caractères accentués (é, è, à) et les symboles (FCFA) peuvent s'afficher sous forme de hiéroglyphes du type « CoopÃ©rative ». C'est l'erreur d'affichage la plus fréquente chez les débutants : elle se corrige en une seule ligne, placée en premier dans le `<head>`.

1.4 Les balises de structuration du contenu

À l'intérieur du `<body>`, on organise le contenu avec des balises de structure. Chacune répond à un besoin précis.

Balise	Rôle
<code><h1></code> à <code><h6></code>	Titres, du plus important (<code>h1</code> , un seul par page en principe) au moins important (<code>h6</code>)
<code><p></code>	Paragraphe de texte
<code><div></code>	Conteneur générique en bloc : regroupe plusieurs éléments pour les structurer ou les styliser ensemble
<code></code>	Conteneur générique en ligne : cible un morceau de texte à l'intérieur d'une phrase sans créer de saut de ligne
<code>
</code>	Saut de ligne (balise orpheline)
<code><hr></code>	Ligne horizontale de séparation (balise orpheline)

La distinction **bloc** / **en ligne** est fondamentale : un élément de type bloc (`h1`, `p`, `div`) occupe toute la largeur disponible et provoque un passage à la ligne ; un élément en ligne (`span`, `b`, `i`) s'insère dans le flux du texte.

EXEMPLE Bloc contre en ligne

```
<div class="membre">
  <h2>Apport de la semaine</h2>
  <p>Membre : <span class="nom">Aminatou</span> –
    <span class="qte">12 sacs</span></p>
</div>
```

Le `div` encadre tout le bloc ; les deux `span` ciblent des mots précis à l'intérieur du paragraphe, sans casser la phrase. JavaScript pourra plus tard modifier uniquement la quantité grâce à ce découpage fin.

1.5 Imbrication des balises : la règle d'or

Les éléments HTML s'emboîtent comme des poupées russes : un élément ouvert à l'intérieur d'un autre doit être **fermé avant** celui-ci. On dit que les balises doivent être correctement **imbriquées**.

PROPRIÉTÉ Règle d'imbrication

Si une balise *B* est ouverte à l'intérieur d'une balise *A*, alors *B* doit être fermée avant la fermeture de *A*. Autrement dit, la dernière balise ouverte est la première fermée (structure « dernier arrivé, premier sorti », comme une pile).

ATTENTION L'erreur classique : croisement de balises

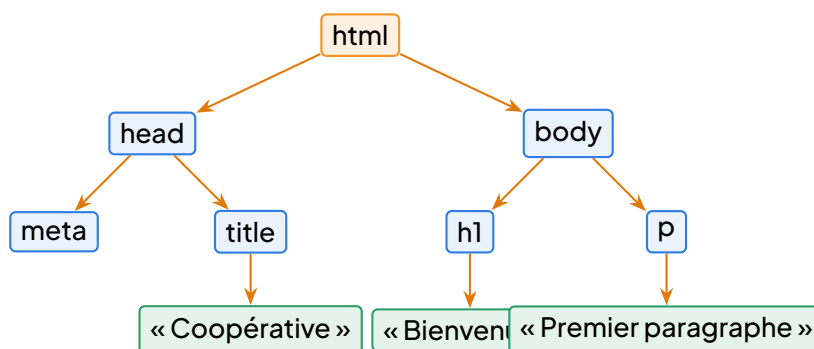
```
<!-- FAUX : les balises se croisent -->
<b><i>Texte important</b></i>

<!-- CORRECT : i est ouvert en dernier, donc ferme en premier -->
<b><i>Texte important</i></b>
```

Le navigateur tente parfois de « réparer » seul le code fautif, mais le résultat devient imprévisible : mise en forme cassée, JavaScript qui ne trouve plus les éléments, tableau déformé — exactement le problème du tableau récapitulatif d'Aminatou. Indente ton code (décalage d'espaces à chaque niveau) : c'est le meilleur moyen de repérer visuellement une balise non fermée.

1.6 L'arbre du DOM : la page vue comme une hiérarchie

Quand le navigateur lit une page HTML, il la transforme en mémoire en une structure arborescente appelée **DOM** (Document Object Model). Chaque élément devient un **nœud** de l'arbre : l'élément `html` est la racine, `head` et `body` sont ses enfants, et ainsi de suite. Cette vision en arbre est essentielle : c'est elle qui permettra à JavaScript (section 4) de parcourir et modifier la page.



Arbre du DOM de la page minimale : chaque élément est un nœud ; les feuilles vertes sont les nœuds texte.

À RETENIR Vision en arbre

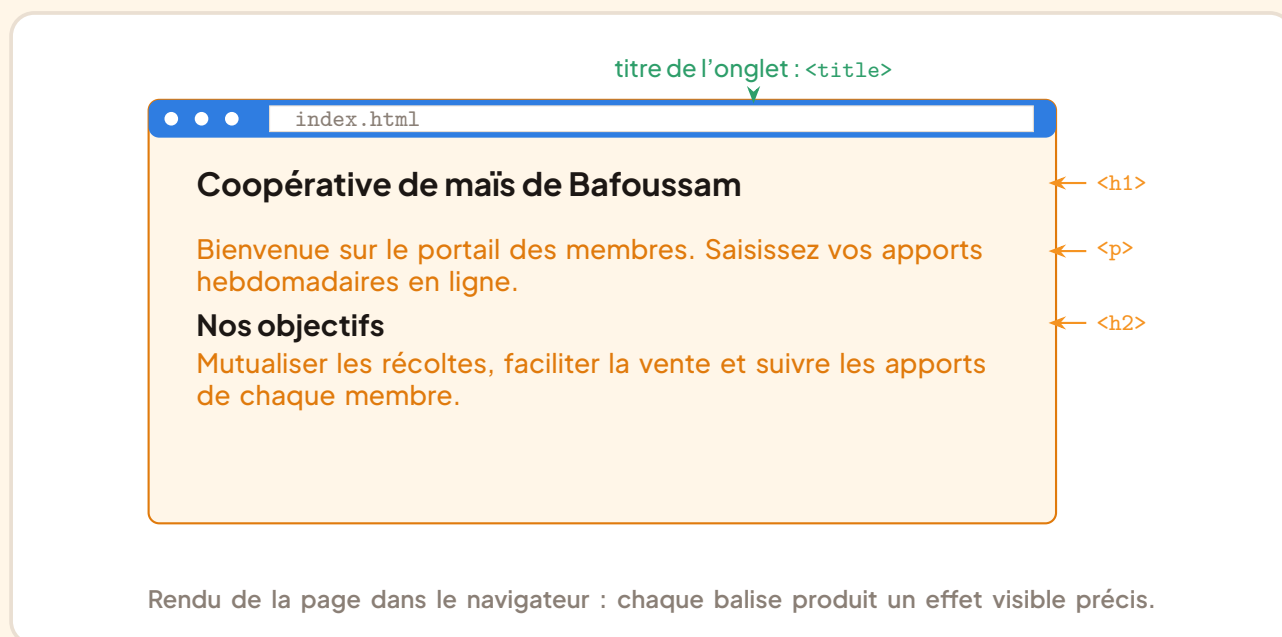
Une page HTML = un arbre. La racine est `html` ; ses deux enfants sont `head` (métadonnées) et `body` (contenu visible). Deux éléments qui partagent le même parent sont des frères ; un élément contenu dans un autre en est l'enfant. Ce vocabulaire sera utilisé tel quel par JavaScript.

1.7 Exemple complet : la page de la coopérative de Bafoussam

Mettons tout en œuvre. Voici la première version de la page d'Aminatou, avec le rendu que le navigateur en produit.

EXEMPLE Page « Coopérative de Bafoussam » — code complet

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Coopérative de mais de Bafoussam</title>
</head>
<body>
  <h1>Coopérative de mais de Bafoussam</h1>
  <p>Bienvenue sur le portail des membres.
    Saisissez vos apports hebdomadaires en ligne.</p>
  <h2>Nos objectifs</h2>
  <p>Mutualiser les récoltes, faciliter la vente
    et suivre les apports de chaque membre.</p>
</body>
</html>
```



1.8 Complément au programme : les balises sémantiques HTML5

LESAVAIS-TU ? Complément utile pour le Bac

HTML5 a introduit des balises dites sémantiques qui remplacent avantageusement les `<div>` génériques en donnant du sens aux zones de la page : `<header>` (en-tête de page), `<nav>` (menu de navigation), `<section>` (partie thématique), `<article>` (contenu autonome), `<footer>` (pied de page). À contenu identique, une page utilisant ces balises est mieux comprise par les moteurs de recherche et les lecteurs d'écran pour malvoyants. Ce point n'est pas exigé au programme de Terminale TI, mais sa mention dans une copie ou un projet est valorisée.

LESAVAIS-TU ? Le premier site web de l'histoire

Le tout premier site web a été mis en ligne en 1991 par Tim Berners-Lee, chercheur au CERN (Genève), inventeur du Web, du HTML et du protocole HTTP. Sa page expliquait... ce qu'était le World Wide Web. Ce site historique a été restauré et est toujours accessible en ligne à l'adresse `info.cern.ch` : il ne contient que du texte et des liens, sans aucune image — la preuve qu'une structure HTML bien faite suffit à transmettre de l'information, trente ans plus tard.

1.9 Exercice résolu et entraînement

EXERCICE RÉSOLU 1 Corriger une page mal formée

Énoncé. Voici le premier essai de page écrit par un membre de la coopérative. Il contient quatre erreurs. Identifie-les et réécris le code corrigé.

```
<html>
<head>
  <title>Suivi des apports
</head>
<body>
  <h1>Apports de la semaine</h2>
  <p><b>Membre : Fotso</p></b>
</body>
\ tcblower
\ textbf{Solution détaillée.} Analysons ligne par ligne :
\ begin{enumerate}
  \ item Il manque la déclaration \texttt{<!DOCTYPE html>} en première ligne : sans elle, le navigateur peut
    basculer en mode de compatibilité et afficher la page de façon incohérente.
  \ item La balise \texttt{<title>} n'est pas fermée : il manque \texttt{</title>} après « Suivi des apports ».
    Le navigateur considère alors que tout le reste fait partie du titre.
  \ item Incohérence de titres : \texttt{<h1>} est fermé par \texttt{</h2>}. Une balise fermante doit
    correspondre exactement à sa balise ouvrante.
  \ item Croisement de balises : \texttt{<p><b>...</p></b>} viole la règle d'imbrication ; \texttt{<b>},
    ouvert en dernier, doit être fermé en premier : \texttt{<p><b>...</b></p>}.
\ end{enumerate}
Code corrigé :
\ begin{lstlisting}[language=HTML]
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Suivi des apports</title>
</head>
<body>
  <h1>Apports de la semaine</h1>
  <p><b>Membre : Fotso</b></p>
</body>
</html>
```

On a aussi ajouté `<meta charset="UTF-8">` et `lang="fr"` par bonne pratique.

EXERCICE 1 À toi de jouer

Écris le code HTML complet d'une page `ecole.html` pour le lycée de ta localité, contenant : la déclaration HTML5, l'en-tête avec encodage UTF-8 et un titre d'onglet « Lycée de ... », un titre principal `h1`, un sous-titre `h2` « Présentation », et deux paragraphes. Dans le second paragraphe, mets le nom du proviseur en valeur avec un `span` portant l'identifiant `proviseur`. Dessine ensuite l'arbre du DOM correspondant.

CORRIGÉ Exercice 1

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Lycee de Bafoussam</title>
</head>
<body>
  <h1>Lycee de Bafoussam</h1>
  <h2>Presentation</h2>
  <p>Notre lycee accueille plus de 3000 eleves,
    de la sixieme a la terminale.</p>
  <p>Le proviseur est <span id="proviseur">M. Kamdem</span>.
    Il dirige l'etablissement depuis 2021.</p>
</body>
</html>
```

Arbre du DOM : racine `html` ; enfants `head` (avec `meta` et `title`) et `body` (avec `h1`, `h2`, et deux `p` ; le second `p` possède un enfant `span` d'attribut `id="proviseur"`). Vérifier que chaque balise ouverte est bien fermée dans l'ordre inverse.

À RETENIR L'essentiel de la section

- HTML est un langage de balisage qui décrit la structure d'une page ; ce n'est pas un langage de programmation.
- Toute page HTML5 valide commence par `<!DOCTYPE html>` et s'organise en `<html>` → `<head>` (métadonnées : `charset`, `title`, liens CSS/JS) + `<body>` (contenu visible).
- Un élément = balise ouvrante + contenu + balise fermante ; les attributs (`id` unique, `class` partagée, `name` pour les formulaires) précisent l'élément.
- Les balises s'imbriquent comme une pile : dernière ouverte, première fermée. Jamais de croisement.
- Le navigateur représente la page en mémoire sous forme d'arbre DOM, base de toute manipulation JavaScript.

Maintenant que la page d'Aminatou est solidement structurée, l'étape suivante consiste à y construire le **formulaire de saisie des apports** : c'est l'objet de la section suivante.

2 LES FORMULAIRES HTML

Dans la section précédente, nous avons vu comment structurer le squelette d'une page web avec les balises sémantiques (`<header>`, `<main>`, `<section>`, etc.). Une page statique ne suffit pas pour une application moderne : il faut collecter de l'information auprès du visiteur. Que ce

soit pour s'inscrire à la coopérative agricole de sa zone, commander du crédit MTN Mobile Money, ou déposer une réclamation sur le portail `impots.cm`, l'interface de dialogue est le **formulaire HTML**. C'est le point d'entrée de toute donnée utilisateur vers le serveur.

2.1 Le rôle du formulaire et la balise `<form>`

DÉFINITION Formulaire HTML

Un formulaire HTML est une zone délimitée par la balise `<form>` qui regroupe des contrôles (champs de saisie, listes, boutons) permettant à l'utilisateur de saisir des données. À la soumission, ces données sont envoyées au serveur sous forme de paires **nom=valeur** pour être traitées (enregistrement en base, envoi d'email, calcul, etc.).

La balise `<form>` est un conteneur de bloc. Elle possède deux attributs fondamentaux qui définissent où et comment les données voyagent :

PROPRIÉTÉ Attributs essentiels de `<form>`

- **action** : **URL** du script côté serveur qui traitera les données (ex. `action="traitement_inscription.php"`). Si absent, le formulaire se soumet sur la page courante.
- **method** : **Protocole HTTP** d'envoi. Deux valeurs possibles :
 - **GET (défaut)** : les données sont ajoutées à l'URL (`?nom=Dupond&age=20`). Visible dans la barre d'adresse, non sécurisé pour les mots de passe, limité en longueur (≈2048 caractères). Adapté aux recherches, filtres.
 - **POST** : les données sont envoyées dans le corps de la requête HTTP. Invisible dans l'URL, pas de limite pratique de taille, obligatoire pour toute donnée sensible (mot de passe, paiement, modification de base de données).
- **enctype** (optionnel) : Type d'encodage des données. Par défaut `application/x-www-form-urlencoded`. Indispensable : `enctype="multipart/form-data"` si le formulaire contient un `<input type="file">` (upload de fichier).

ATTENTION Choix de la méthode : piège classique

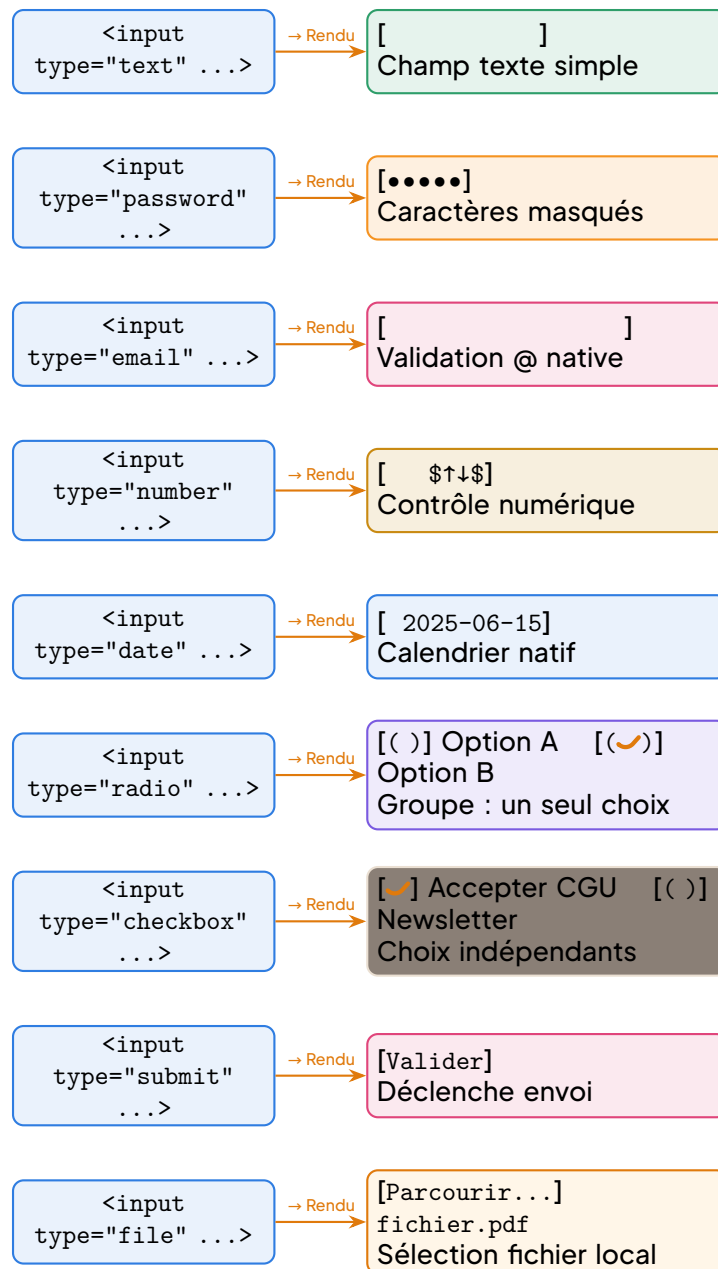
Ne jamais utiliser `method="GET"` pour un formulaire de connexion, de paiement (Orange Money, Mobile Money) ou d'inscription. Les identifiants apparaîtraient dans l'historique du navigateur, les logs du proxy de l'établissement (lycée, cybercafé) et pourraient être volés par un simple shoulder surfing. Par défaut, pensez POST.

2.2 Les contrôles de saisie : la balise `<input>` et ses types

La balise `<input>` (balise auto-fermante, void element) est le couteau suisse des formulaires. Son comportement change radicalement selon son attribut `type`.

PROPRIÉTÉ Types courants de `<input>`

Type	Rôle / Rendu visuel	Cas d'usage typique (contexte camerounais)
text	Champ texte simple, une ligne.	Nom complet, matricule élève, référence paiement.
password	Champ masqué (points/astérisques).	Mot de passe, code PIN Mobile Money.
email	Champ texte + validation native format @.	Adresse email pour notification résultats BAC.
number	Champ numérique avec flèches up/down.	Quantité (sacs de maïs), montant (FCFA), âge.
date	Sélecteur de date calendrier (navigateur moderne).	Date de naissance, date de dépôt dossier.
radio	Bouton circulaire, choix unique dans un groupe (même <code>name</code>).	Sexe (M/F), Filière (S/STI2D/STMG), Type abonnement.
checkbox	Case carrée, choix multiple indépendant (ou booléen unique).	CGU acceptées, options multiples (sports, clubs).
submit	Bouton déclenchant l'envoi (<code>action/method</code>).	Valider inscription, Payer, Envoyer.
file	Sélecteur de fichier (nécessite <code>enctype="multipart/form-data"</code>).	Photo profil, pièce justificative, devoir numérisé.



Correspondance type d'`<input>` ↔ Rendu visuel et comportement dans le navigateur

Au-delà du type, quatre attributs sont **indispensables** sur presque chaque contrôle :

PROPRIÉTÉ Attributs universels des contrôles

- **name** : **Nom de la variable** envoyée au serveur (`$_POST['nom_champ']` en PHP). Obligatoire pour que la valeur soit transmise. Plusieurs contrôles (radio) partagent le même name.

- **id** : **Identifiant unique** dans la page (DOM). Utilisé par `<label for="...">`, par JavaScript (`document.getElementById`), et par les ancres CSS (`#id`). Doit être unique dans le document.
- **value** : Valeur par défaut (pré-remplie) ou valeur envoyée si l'utilisateur ne modifie pas le champ (crucial pour `radio/checkbox`).
- **placeholder** : Texte d'aide disparu à la saisie (ex. "Ex: 6XX XX XX XX"). N'est pas une valeur par défaut et n'est pas envoyé au serveur.
- **required** : Validation native HTML5 : empêche la soumission si vide (ou invalide pour `email/number`). Message d'erreur géré par le navigateur (localisé).

ATTENTION Confusion `name` vs `id` : Erreur n°1 au BAC

- `name` = Côté serveur (PHP, Python, Node.js). Clé du tableau `$_POST / $_GET`. Peut être dupliqué (groupe `radio`).
- `id` = Côté client (HTML `label`, CSS `#id`, JS `getElementById`). Doit être unique dans toute la page.
- Règle d'or : Souvent on met `id="champ"` ET `name="champ"` (même valeur), mais leurs rôles sont disjoints. Oublier `name` → donnée perdue côté serveur. Oublier `id` → label inopérant, JS incapable de cibler le champ.

2.3 Listes déroulantes et zones de texte multiligne

Pour les choix parmi une liste fermée longue (régions du Cameroun, filières lycée) ou les textes longs (message, rédaction), `<input>` ne suffit pas.

PROPRIÉTÉ Liste déroulante : `<select>` et `<option>`

```
<label for="region">Région d'origine :</label>
<select name="region" id="region" required>
  <option value="">-- Choisir une région --</option> <!-- Placeholder visuel -->
  <option value="centre">Centre (Yaoundé)</option>
  <option value="littoral" selected>Littoral (Douala)</option> <!-- Présélectionné -->
  <option value="ouest">Ouest (Bafoussam)</option>
  <option value="nord">Nord (Garoua)</option>
  <!-- ... autres régions ... -->
</select>
```

- `<select>` porte `name` et `id`.
- `<option>` porte `value` (ce qui est envoyé). Le texte entre balises est l'affichage.

- Attribut `selected` sur une `<option>` = choix par défaut.
- Attribut `multiple` sur `<select>` = choix multiples (Ctrl+clic), rendu liste box.

PROPRIÉTÉ Zone multiligne : `<textarea>`

```
<label for="message">Motivation / Remarques :</label>
<textarea name="message" id="message" rows="5" cols="40"
  placeholder="Décrivez votre projet en quelques lignes..."></textarea>
```

- Pas d'attribut `value` : le contenu par défaut se place entre les balises d'ouverture/fermeture.
- `rows` (hauteur en lignes) et `cols` (largeur en caractères) : dimensions initiales (CSS `width/height` préférable pour le design final).
- L'utilisateur peut taper des retours à la ligne (conservés à l'envoi).

2.4 Accessibilité et structuration : `<label>`, `<fieldset>`, `<legend>`

Un formulaire professionnel (et conforme aux standards W3C/Accessibilité RGAA) doit associer explicitement chaque champ à son libellé.

DÉFINITION Association `<label>` / Contrôle

L'attribut `for` du `<label>` doit exactement égaliser l'attribut `id` du contrôle associé.

```
<!-- CORRECT : Clic sur "Téléphone" focus le champ input -->
<label for="tel">Téléphone (format 6XX XX XX XX) :</label>
<input type="tel" name="telephone" id="tel" placeholder="6XX XX XX XX" required>

<!-- INCORRECT : Pas de lien, clic sur label ne fait rien, lecteur d'écran muet -->
<label>Téléphone :</label> <input type="tel" name="telephone">
```

Avantages : Accessibilité (lecteurs d'écran), UX (clic sur le label = focus sur le champ, crucial sur mobile), CSS (ciblage `label[for="tel"]`).

PROPRIÉTÉ Regroupement sémantique : `<fieldset>` et `<legend>`

Utilisé pour grouper des contrôles logiquement liés (ex. un groupe de `radio`, une section "Adresse", "Coordonnées bancaires"). Le navigateur dessine souvent une bordure ; le `<legend>` en est le titre.

```

<fieldset>
  <legend>Coordonnées Mobile Money</legend>

  <label for="opérateur">Opérateur :</label>
  <select name="opérateur" id="opérateur">
    <option value="mtn">MTN Mobile Money</option>
    <option value="orange">Orange Money</option>
  </select>

  <label for="num_momo">Numéro :</label>
  <input type="tel" name="num_momo" id="num_momo" pattern="[62]\d{8}" placeholder="6XX XX XX
  XX" required>
</fieldset>

```

2.5 Les boutons d'action

Trois types de boutons existent via `<button>` (préférable à `<input type="...">` car permet du contenu HTML riche : icône + texte).

PROPRIÉTÉ **Attribut type de** `<button>`

- `type="submit"` (défaut) : Envoie le formulaire vers `action` avec `method`. Un seul par formulaire en général.
- `type="reset"` : Remet tous les champs à leur valeur initiale (vide ou `value` par défaut). Dangereux : l'utilisateur peut cliquer par erreur et tout perdre. À utiliser avec parcimonie, jamais à côté de "Valider" sans confirmation.
- `type="button"` : Aucun comportement natif. Point d'ancrage pur pour JavaScript (ex. "Calculer total", "Vérifier dispo", "Ouvrir modal").

EXEMPLE **Boutons dans un formulaire de paiement**

```

<fieldset>
  <legend>Paiement frais d'inscription -- 15 000 FCFA</legend>
  <!-- ... champs sélection opérateur, numéro ... -->

  <div class="actions-form"> <!-- Conteneur pour flexbox/grid CSS -->
    <button type="button" id="btn-verifier" class="btn-secondaire">
      Vérifier le solde
    </button>
    <button type="reset" class="btn-annuler">
      Annuler / Réinitialiser
    </button>
    <button type="submit" name="action" value="payer" class="btn-principal">
      Confirmer le paiement

```

```
        </button>
    </div>
</fieldset>
```

2.6 Méthode : Construire un formulaire HTML complet et robuste

MÉTHODE Créer un formulaire HTML complet – Checklist en 7 étapes

1. **Balise conteneur** : `<form action="url_traitement" method="POST" novalidate>` (`novalidate` désactive la validation native si on veut uniquement du JS custom, sinon on la garde).
2. **Structure par blocs** : Un `<div class="champ">` (ou `<p>`) par contrôle pour le placement CSS (Flexbox/Grid).
3. **Label + Contrôle** : Pour chaque saisie : `<label for="id">Libellé </label>` suivi du `<input>/<select>/<textarea>` avec `id="id"` ET `name="nom_serveur"`.
4. **Types adaptés** : `email, tel, number, date, file` selon la donnée attendue (clavier mobile adapté).
5. **Contraintes natives** : `required, min/max/step (number), pattern (regex), minlength/maxlength`.
6. **Groupes logiques** : `<fieldset><legend>Titre section</legend>...` pour radio, adresses, paiement.
7. **Boutons clairs** : `<button type="submit">Valider</button>` (principal), éventuellement `type="button"` pour actions JS, éviter `reset`.

2.7 Exemple résolu : Formulaire de saisie d'un apport en sacs de maïs

EXERCICE RÉSOLU 2 Saisie d'un apport à la coopérative agricole – 5 contrôles variés

Énoncé : Réaliser le code HTML d'un formulaire permettant à un paysan de déclarer son apport de maïs à la coopérative. Champs requis :

1. Nom du paysan (texte).
2. Numéro de téléphone (format camerounais 9 chiffres commençant par 6 ou 2).
3. Quantité en sacs (entier positif, max 1000).
4. Filière de culture (liste : Maïs blanc, Maïs jaune, Maïs pop-corn).

5. Observations (texte libre multiligne).

Le formulaire envoie les données en POST vers `traitement_apport.php`.

Solution détaillée :

```
<form action="traitement_apport.php" method="POST" class="formulaire-apport">

  <!-- 1. Nom : texte simple, requis -->
  <div class="champ">
    <label for="nom_paysan">Nom complet du paysan :</label>
    <input type="text" name="nom_paysan" id="nom_paysan"
      placeholder="Ex : Mbarga Jean"
      required
      maxlength="100">
  </div>

  <!-- 2. Téléphone : type tel + pattern regex Cameroun (6 ou 2 suivi de 8 chiffres) -->
  <div class="champ">
    <label for="telephone">Téléphone (WhatsApp) :</label>
    <input type="tel" name="telephone" id="telephone"
      placeholder="6XX XX XX XX ou 2XX XX XX XX"
      pattern="[62]\d{8}"
      required
      title="Format : 9 chiffres commençant par 6 ou 2 (ex : 677123456)">
    <!-- title = tooltip navigateur si pattern échoue -->
  </div>

  <!-- 3. Quantité : number avec bornes -->
  <div class="champ">
    <label for="quantite">Quantité (sacs de 100kg) :</label>
    <input type="number" name="quantite" id="quantite"
      min="1" max="1000" step="1"
      value="1"
      required>
  </div>

  <!-- 4. Filière : select avec option par défaut vide -->
  <div class="champ">
    <label for="filiere">Filière de culture :</label>
    <select name="filiere" id="filiere" required>
      <option value="">-- Sélectionner la variété --</option>
      <option value="blanc">Maïs blanc (alimentation humaine)</option>
      <option value="jaune" selected>Maïs jaune (alimentation animale)</option>
      <option value="popcorn">Maïs pop-corn (transformation)</option>
    </select>
  </div>

  <!-- 5. Observations : textarea -->
  <div class="champ">
    <label for="observations">Observations (état du grain, humidité, etc.) :</label>
    <textarea name="observations" id="observations"
      rows="4" cols="50"
      placeholder="Ex : Maïs bien séché, taux humidité 13%, trié manuellement."></textarea>
    <!-- Pas de required : champ optionnel -->
  </div>

  <!-- Boutons -->
```

```
<div class="actions">
  <button type="submit" name="action" value="enregistrer">Enregistrer l'apport</button>
  <button type="button" onclick="this.form.reset()">Effacer le formulaire</button>
  <!-- type="button" + JS reset() évite le piège du type="reset" accidentel -->
</div>

</form>
```

Analyse des 5 contrôles :

1. `text + maxlength` : Sécurité longueur côté client.
2. `tel + pattern="[62]8"` : Validation format Cameroun native (regex). Le `title` aide l'utilisateur.
3. `number + min/max/step` : Contrôle intervalle natif (navigateur empêche saisie hors bornes).
4. `select + required + option value=""` : Force le choix actif (l'option vide n'est pas une valeur valide).
5. `textarea sans required` : Champ optionnel.

LESAVAIS-TU ? L'attribut `required` : nécessaire mais insuffisant

L'attribut `required` (et `type="email"`, `pattern`, `min/max`) active la validation native du navigateur (HTML5 Constraint Validation API). C'est une première ligne de défense gratuite, sans une ligne de JavaScript. Mais elle est insuffisante en production car :

- Elle est côté client uniquement : un utilisateur malveillant (ou un bot) peut désactiver le JS, modifier le HTML via les outils de développement (Inspecter l'élément → supprimer `required`), ou envoyer une requête POST brute avec `curl/Postman` en contournant totalement le navigateur.
- Les messages d'erreur sont imposés par le navigateur (difficiles à styliser, parfois confus, langue selon config OS).
- Elle ne gère pas les règles métier complexes (ex. "Numéro déjà existant en base", "Montant supérieur au solde Mobile Money", "Date de fin antérieure à date de début").

D'où la nécessité impérieuse de la validation côté serveur (PHP) et l'intérêt de la validation côté client JavaScript personnalisée (prochaines sections) pour l'UX.

Formulaire d'inscription – `<form action="inscription.php" method="POST">`

Nom complet

```
<label for="nom">
<input type="text" id="nom" name="nom"
required>
```

← `name="nom" → $_POST['nom']`
`id="nom" → lié au label et au JS`

Téléphone

```
<label for="tel">
<input type="tel" id="tel" name="tel"
pattern="[62]\d{8}" required>
```

← `pattern` : expression régulière du numéro camerounais
validation native HTML5

Quantité (sacs)

```
<label for="qte">
<input type="number" id="qte" name="qte"
min="1" max="1000">
```

← `min / max` : bornes numériques

Filière (dans un `<fieldset>`)

```
<select id="fil" name="filiere" required>
<option value="">-- Choisir --</option>
<option value="mais">Mais</option> ...
</select>
```

← `<fieldset>` : groupe de champs liés
l'option `value=""` sert de texte d'invite

Message

```
<label for="msg">
<textarea id="msg" name="message" rows="3">
</textarea>
```

← contenu entre les balises
pas d'attribut `value`

```
<button type="submit">Valider</button>
<button type="button" onclick="verif()">
Vérifier</button>
```

← `submit` : envoi au serveur
`button` : déclenche seulement le JS

Schéma annoté d'un formulaire d'inscription coopérative : structure, attributs clés et flux de données

EXPÉRIENCE

TP Guidé : Créer et tester un formulaire d'adhésion club informatique

Objectif : Mettre en pratique la structure complète d'un formulaire HTML5 valide et accessible. Outils : VS Code (ou Notepad++), Navigateur (Chrome/Firefox) + Outils développeur (F12 onglet Network/Console).

Étapes :

1. Créer `adhesion.html` avec le squelette HTML5 (`<!DOCTYPE html>`, `<html lang="fr">`, `<head>` `charset UTF-8`, `viewport`, titre).
2. Dans `<main>`, écrire un `<form action="traitement.php" method="POST" novalidate>` (on met `novalidate` pour tester notre JS plus tard, mais on garde les attributs HTML5).
3. Ajouter 4 blocs `<div class="form-group">` contenant chacun :
 - `<label for="...">` explicite.
 - Contrôle adapté :
 - (a) Prénom/Nom (`type="text", required, minlength="2"`).
 - (b) Email (`type="email", required`).

- (c) Niveau d'études (<select> avec options : 3ème, 2nde, 1ère, Terminale, Supérieur; required).
- (d) Motivation (<textarea rows="4">, optionnel).
4. Grouper les deux premiers champs dans un <fieldset><legend>Identité</legend>...</fieldset>.
 5. Ajouter les boutons : <button type="submit">S'inscrire</button> et <button type="button" id="btn-reset">Réinitialiser</button>.
 6. Ouvrir dans le navigateur. Tester :
 - Soumettre vide → Bulles natives HTML5 (requis, email invalide).
 - Remplir correctement → Vérifier dans l'onglet Network (F12) que la requête est POST, voir les Form Data (nom=..., email=..., niveau=..., motivation=...).
 - Cliquer "Réinitialiser" → Ne fait rien (pas de JS attaché). On le fera en Section 5.

Observations attendues : L'onglet Network montre Content-Type: application/x-www-form-urlencoded. Les données name=valeur sont encodées (espaces → + ou %20). Le champ motivation n'apparaît pas s'il est vide (comportement standard). Interprétation : Le formulaire HTML seul assure la collecte et l'envoi structuré. La validation native aide l'utilisateur honnête. La sécurité et la logique métier viendront côté serveur (PHP) et côté client avancé (JS).

3 LES TABLEAUX HTML

Après avoir vu comment concevoir des formulaires pour collecter les données auprès de l'utilisateur, il est tout naturel d'aborder la manière de les présenter de façon structurée. Dans une application de gestion de notes, un tableau de bord administratif ou un site de e-commerce camerounais comme Jumia.cm ou Cdiscount.cm, les données s'affichent presque toujours sous forme de lignes et de colonnes. Le tableau HTML est l'outil standard pour cette restitution tabulaire.

3.1 Rôle et structure de base

DÉFINITION Tableau HTML, ligne, cellule, cellule d'en-tête

Un tableau HTML est un conteneur sémantique (<table>) destiné à organiser des données tabulaires — c'est-à-dire des informations qui ont du sens lorsqu'elles sont croisées selon deux dimensions (lignes et colonnes). Il ne doit jamais être utilisé pour la mise en page visuelle (positionner un logo à gauche, un menu à droite), rôle dévolu au CSS (Flexbox, Grid).

Une ligne (<tr> pour Table Row) est une division horizontale du tableau. Une cellule de

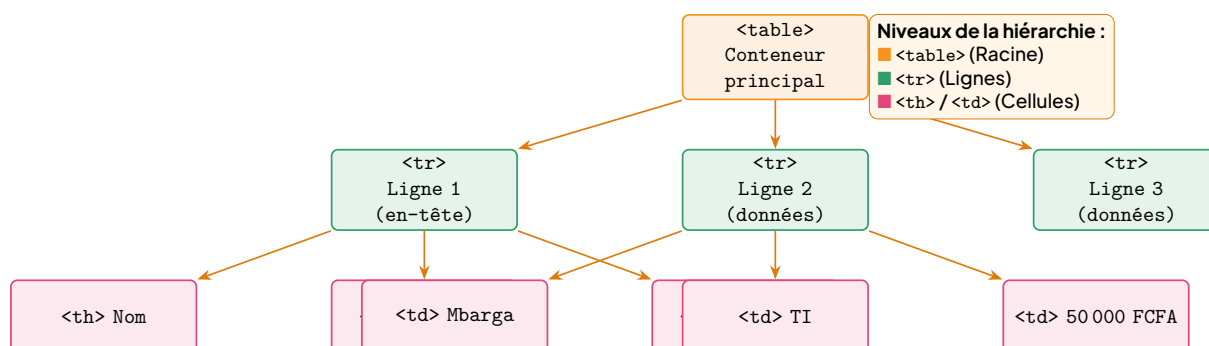
données (<td> pour Table Data) contient une valeur brute. Une cellule d'en-tête (<th> pour Table Header) étiquette une colonne ou une ligne ; elle est rendue en gras et centrée par défaut par les navigateurs, et elle porte une portée sémantique (`scope="col"`, `scope="row"`, `scope="colgroup"`, `scope="rowgroup"`) cruciale pour l'accessibilité (lecteurs d'écran).

PROPRIÉTÉ Squelette minimal d'un tableau

La structure arborescente stricte est la suivante :

- <table> : racine du tableau.
- <tr> : enfants directs de <table> (ou de <thead>, <tbody>, <tfoot>).
- <th> ou <td> : enfants directs de <tr>.

L'attribut historique `border="1"` sur <table> force un trait de séparation visuel entre les cellules. Rappel important : la présentation (bordures, espacements, couleurs, largeur) relève exclusivement du CSS. L'attribut `border` est obsolète en HTML5 mais toléré pour un débogage rapide.



Hiérarchie DOM d'un tableau HTML 3 lignes x 3 colonnes : table → tr → th/td

3.2 Structuration sémantique : <thead>, <tbody>, <tfoot> et <caption>

Pour les tableaux de grande taille (plus de quelques dizaines de lignes), le navigateur peut dupliquer l'en-tête à l'impression sur chaque page, ou permettre le défilement du corps tout en gardant l'en-tête fixe (via CSS `position: sticky`). Cela impose de regrouper les lignes par rôle sémantique.

DÉFINITION Groupes de lignes et légende

- `<thead>` (Table Head) : regroupe une ou plusieurs lignes d'en-tête (`<tr>` contenant des `<th>`). Doit apparaître avant `<tbody>` et `<tfoot>` dans le code source (mais s'affiche en haut visuellement).
- `<tbody>` (Table Body) : regroupe le corps principal des données (`<tr>` contenant des `<td>`). Il peut y en avoir plusieurs pour séparer des sections logiques (ex : par classe dans un bulletin).
- `<tfoot>` (Table Foot) : regroupe les lignes de synthèse (totaux, moyennes, notes de bas de tableau). En HTML5, il se place après `<tbody>` (ordre logique de lecture) ; il s'affiche toujours en bas visuellement.
- `<caption>` : unique, premier enfant de `<table>`. Donne un titre accessible au tableau (lu par les technologies d'assistance). Ne pas confondre avec un `<h2>` placé avant le tableau.

MÉTHODE Construire un tableau HTML structuré et accessible

Suis cette démarche pas à pas :

1. Ouvre `<table>` et ajoute immédiatement `<caption>` avec un titre clair (ex : « Liste des élèves de Terminale TI — Session 2025 »).
2. Crée `<thead>` : une `<tr>` contenant autant de `<th scope="col">` que de colonnes. Le `scope="col"` lie explicitement l'en-tête à sa colonne.
3. Crée `<tbody>` : pour chaque ligne de données, une `<tr>` contenant des `<td>`. Si la première cellule de la ligne est un identifiant (nom, matricule), utilise `<th scope="row">` pour cette cellule.
4. (Optionnel) Crée `<tfoot>` : une `<tr>` avec des `<td>` (ou `<th scope="row">` pour l'étiquette « Total ») pour les calculs de synthèse.
5. Ferme `</table>`.
6. Style au CSS : `border-collapse: collapse;` sur `table`, `padding` et `border` sur `th`, `td`, `background-color` sur `thead th`.

EXEMPLE Tableau complet avec `thead`, `tbody`, `tfoot` et `caption`

Voici le code source d'un bulletin simplifié pour un élève. Remarque l'ordre `thead` → `tbody` → `tfoot` (ordre logique HTML5, recommandé).

```
<table>
  <caption>Bulletin de notes — Mbarga Jean, Terminale TI</caption>

  <thead>
```

```
<tr>
  <th scope="col">Matière</th>
  <th scope="col">Coef.</th>
  <th scope="col">Devoir</th>
  <th scope="col">Examen</th>
  <th scope="col">Moyenne</th>
</tr>
</thead>

<tbody>
  <tr>
    <th scope="row">Mathématiques</th>
    <td>4</td>
    <td>15</td>
    <td>14</td>
    <td>14,5</td>
  </tr>
  <tr>
    <th scope="row">Physique–Chimie</th>
    <td>3</td>
    <td>12</td>
    <td>13</td>
    <td>12,5</td>
  </tr>
  <tr>
    <th scope="row">Informatique</th>
    <td>5</td>
    <td>16</td>
    <td>15</td>
    <td>15,5</td>
  </tr>
  <tr>
    <th scope="row">Anglais</th>
    <td>2</td>
    <td>14</td>
    <td>13</td>
    <td>13,5</td>
  </tr>
</tbody>

<tfoot>
  <tr>
    <th scope="row">Moyenne générale</th>
    <td colspan="2"></td>
    <td><strong>14,2</strong></td>
    <td><strong>13,8</strong></td>
  </tr>
</tfoot>
</table>
```

Rendu visuel (avec CSS basique) : un tableau lisible, en-tête figé en haut, total en bas, légende annoncée par les lecteurs d'écran.

3.3 Fusion de cellules : colspan et rowspan

Parfois, une information s'étend sur plusieurs colonnes (un titre de section au-dessus de plu-

sieurs matières) ou plusieurs lignes (un nom d'enseignant responsable de plusieurs modules). HTML prévoit deux attributs sur `<th>` et `<td>` :

PROPRIÉTÉ Attributs de fusion

- `colspan="n"` : la cellule occupe `n` colonnes vers la droite. La ligne courante doit contenir moins de balises `<td>/<th>` physiques pour compenser.
- `rowspan="n"` : la cellule occupe `n` lignes vers le bas. Les lignes suivantes doivent omettre la cellule correspondante dans leur code.

La valeur par défaut est 1. `n` doit être un entier strictement positif.

ATTENTION Erreur classique : nombre de cellules incohérent

C'est la première cause de tableaux « cassés » (colonnes décalées, cellules manquantes).

- Si tu mets `colspan="2"` sur une cellule, cette ligne perd une balise de cellule par rapport aux autres lignes.
- Si tu mets `rowspan="2"` sur une cellule de la ligne 1, la ligne 2 ne doit pas écrire la balise correspondante.
- Astuce de pro : dessine le tableau sur papier, compte les cases réelles par ligne, puis écris le HTML. Vérifie avec l'inspecteur du navigateur (onglet `Elements`) : le navigateur corrige parfois silencieusement, masquant l'erreur.

Enseignant responsable		Matière
Mathématiques	Algèbre	15
	Analyse	14
		13

Légende des fusions :

- `colspan="2"` : fusion horizontale (orange)
- `rowspan="2"` : fusion verticale (vert)
- Cellules normales (bleu)

Représentation visuelle d'un tableau 4×3 avec `colspan` (ligne 0) et `rowspan` (colonne 0, lignes 1–2). Les cellules fusionnées sont colorées.

EXEMPLE Code du tableau fusionné ci-dessus

```

<table>
  <caption>Notes de Mathématiques et Physique</caption>
  <thead>
    <tr>
      <!-- colspan=2 : cette cellule mange les 2 premières colonnes logiques -->
      <th colspan="2" scope="colgroup">Enseignant : M. Fouda</th>
      <th scope="col">Matière</th>
    </tr>
    <tr>
      <th scope="col">Module</th>
      <th scope="col">Chapitre</th>
      <th scope="col">Note</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <!-- rowspan=2 : cette cellule descend sur la ligne suivante -->
      <th scope="rowgroup" rowspan="2">Mathématiques</th>
      <td>Algèbre</td>
      <td>15</td>
    </tr>
    <tr>
      <!-- Pas de <th> pour Maths ici ! Elle est déjà rendue par le <rowspan> du dessus -->
      <td>Analyse</td>
      <td>14</td>
    </tr>
    <tr>
      <th scope="row">Physique</th>
      <td>Mécanique</td>
      <td>13</td>
    </tr>
  </tbody>
</table>

```

3.4 Exemple chiffré : Récapitulatif des apports d'une coopérative

Contexte camerounais : une coopérative agricole du Littoral (ex : COOPECAM) centralise les apports de ses membres pour la campagne cacaoyère. Le trésorier veut un tableau HTML à intégrer dans l'intranet de la coopérative.

EXEMPLE Tableau des apports — 4 membres, 4 colonnes

Données brutes :

Nom	Filière	Sacs (50 kg)	Montant (FCFA)
Mbarga Paul	Cacao	12	1 800 000
Essomba Grace	Café	8	960 000
Fouda Jean	Cacao	15	2 250 000
Atangana Marie	Café-Cacao	10	1 350 000
TOTAL		45	6 360 000

Code HTML structuré :

```
<table>
  <caption>Apports campagne cacaoyère 2025 — Coopérative COOPECAM</caption>
  <thead>
    <tr>
      <th scope="col">Nom du membre</th>
      <th scope="col">Filière</th>
      <th scope="col">Sacs (50 kg)</th>
      <th scope="col">Montant (FCFA)</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Mbarga Paul</th>
      <td>Cacao</td>
      <td class="num">12</td>
      <td class="num">1 800 000</td>
    </tr>
    <tr>
      <th scope="row">Essomba Grace</th>
      <td>Café</td>
      <td class="num">8</td>
      <td class="num">960 000</td>
    </tr>
    <tr>
      <th scope="row">Fouda Jean</th>
      <td>Cacao</td>
      <td class="num">15</td>
      <td class="num">2 250 000</td>
    </tr>
    <tr>
      <th scope="row">Atangana Marie</th>
      <td>Café-Cacao</td>
      <td class="num">10</td>
      <td class="num">1 350 000</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <th scope="row" colspan="2">TOTAL GÉNÉRAL</th>
      <td class="num">45</td>
      <td class="num">6 360 000</td>
    </tr>
  </tfoot>
</table>
```

Points clés :

- `scope="row"` sur la première cellule de chaque `<tbody>` (nom du membre) et sur la cellule d'étiquette du `<tfoot>`.
- `colspan="2"` sur « TOTAL GÉNÉRAL » pour fusionner les deux premières colonnes de la ligne de pied.
- Alignement à droite des nombres via classe CSS `.num` (préférable au style inline).
- Séparateur d'espace insécable dans les montants pour la lisibilité (ou ` ` en HTML).

3.5 Bonnes pratiques, accessibilité et pièges à éviter

ATTENTION Tableaux pour mise en page : pratique interdite

Avant 2005 (ère IE6, Netscape), on imbriquait des `<table>` pour caler un logo à gauche, un menu à droite, le contenu au centre. C'est une très mauvaise pratique aujourd'hui :

- Accessibilité : les lecteurs d'écran annoncent « Tableau de 12 lignes, 4 colonnes » pour une simple page d'accueil, perdant l'utilisateur malvoyant.
- SEO : Google pénalise le contenu non sémantique.
- Maintenance : modifier une mise en page en tableaux imbriqués est un cauchemar (changer une largeur casse tout).
- Responsive : un tableau ne se « casse » pas naturellement sur mobile ; il faut `display: block` sur les lignes, ce qui détruit la sémantique.

Solution : Utilise `<header>`, `<nav>`, `<main>`, `<aside>`, `<footer>` + CSS Grid / Flexbox.

LESAVAIS-TU ? L'histoire des tableaux de mise en page

Dans les années 1990–2000, le Web n'avait pas de modèle de boîte flexible. Le standard HTML 3.2 (1997) n'offrait que les tableaux pour positionner des éléments côte à côte. Les sites du gouvernement camerounais de l'époque (premiers portails `*.gov.cm` vers 2002) étaient presque tous construits ainsi. L'arrivée de CSS2 (1998) puis de CSS Grid (2017, supporté partout depuis 2020) a rendu cette technique obsolète. Aujourd'hui, un développeur qui fait de la mise en page en `<table>` lors d'un entretien technique à ActivSpaces (Buea) ou Jongo Hub (Yaoundé) rate son test technique.

À RETENIR Check-list du tableau HTML parfait

- `<table>` unique, `<caption>` descriptive en premier enfant.
- Structure `<thead>` / `<tbody>` / `<tfoot>` respectée (ordre HTML5 : `thead`, `tbody`, `tfoot`).
- `<th scope="col">` pour les en-têtes de colonnes, `<th scope="row">` pour les en-têtes de lignes, `scope="colgroup"/"rowgroup"` pour les groupes.
- `colspan` / `rowspan` utilisés uniquement pour la fusion de données réelles (pas pour l'espace).
- Nombre de cellules effectives (après fusion) identique sur chaque `<tr>`.
- Zéro style inline (`style="..."`) en production : classes CSS (`.text-right`, `.total-row`).
- Testé au validateur W3C (validator.w3.org) et au lecteur d'écran (NVDA, VoiceOver).

EXERCICE RÉSOLU 3 Construction d'un tableau complexe avec fusions

Énoncé : Écris le code HTML complet (sans CSS) d'un tableau « Emploi du temps Terminale TI — Semestre 1 » respectant ces contraintes :

1. Légende : « Emploi du temps officiel — Terminale TI — Lycée Bilingue de Nkolbisson ».
2. En-tête : une ligne « JOUR / HEURE » fusionnée sur 2 colonnes, puis 5 colonnes « Lundi » à « Vendredi ».
3. Corps : 4 créneaux horaires (07h-09h, 09h-11h, 11h-13h, 13h-15h).
4. Le cours « Projet TI » occupe les deux derniers créneaux du Mercredi (`rowspan="2"` sur la cellule Mercredi de la 3ème ligne).
5. Pied de tableau : une ligne « Total heures : 30 h » fusionnée sur toutes les colonnes.

SOLUTION

Solution détaillée :

```
<table>
  <caption>Emploi du temps officiel — Terminale TI — Lycée Bilingue de Nkolbisson</caption>

  <thead>
    <!-- Ligne 1 : Titre fusionné sur 2 cols + 5 jours -->
    <tr>
      <th colspan="2" scope="colgroup">JOUR / HEURE</th>
      <th scope="col">Lundi</th>
      <th scope="col">Mardi</th>
      <th scope="col">Mercredi</th>
      <th scope="col">Jeudi</th>
      <th scope="col">Vendredi</th>
    </tr>
```

```

</thead>

<tbody>
  <!-- Ligne 1 : 07h-09h -->
  <tr>
    <th scope="row">07h-09h</th>
    <td>Mathématiques</td>
    <td>Physique</td>
    <td>Anglais</td>
    <td>Informatique</td>
    <td>Sport</td>
  </tr>
  <!-- Ligne 2 : 09h-11h -->
  <tr>
    <th scope="row">09h-11h</th>
    <td>Français</td>
    <td>Chimie</td>
    <td>Mathématiques</td>
    <td>Philosophie</td>
    <td>Informatique</td>
  </tr>
  <!-- Ligne 3 : 11h-13h (Début rowspan Mercredi) -->
  <tr>
    <th scope="row">11h-13h</th>
    <td>Histoire-Géo</td>
    <td>Informatique</td>
    <th scope="rowgroup" rowspan="2">Projet TI</th> <!-- Cellule fusionnée verticalement -->
    <td>Mathématiques</td>
    <td>Physique</td>
  </tr>
  <!-- Ligne 4 : 13h-15h (Pas de cellule Mercredi !) -->
  <tr>
    <th scope="row">13h-15h</th>
    <td>Informatique</td>
    <td>Sport</td>
    <!-- Pas de <td> pour Mercredi : absorbé par rowspan=2 du dessus -->
    <td>Anglais</td>
    <td>Chimie</td>
  </tr>
</tbody>

<tfoot>
  <tr>
    <td colspan="6">Total heures : 30 h</td>
  </tr>
</tfoot>
</table>

```

Explications :

- Ligne thead : 1 th (colspan="2") + 5 th = 6 colonnes logiques.
- Lignes tbody : chacune a 1 th scope="row" (heure) + 5 td = 6 cellules, sauf la 4ème ligne (13h-15h) qui n'a que 4 td car la 3ème colonne (Mercredi) est fournie par le rowspan="2" de la ligne précédente.
- tfoot : 1 td colspan="6" couvre toute la largeur.

- Note l'usage de `<th scope="row">` pour l'heure (première colonne) et pour « Projet TI » (en-tête de ligne implicite pour ce bloc fusionné avec `scope="rowgroup"`).

EXERCICE 2 Entraînement : Tableau de notation d'un hackathon

Réalise le code HTML (fichier `hackathon.html`) pour afficher les résultats du « Cameroon Digital Boost 2025 ». Colonnes : Rang, Équipe, Projet, Score technique (sur 50), Score pitch (sur 20), Total (sur 70). Données : 1. Team KmerAI — « Diagnostic palu par IA » — 48 — 18 — 66 2. Team MboaCode — « Marketplace agro » — 45 — 19 — 64 3. Team Silicon-Mountain — « FinTech inclusion » — 47 — 16 — 63 4. Team DoualaTech — « Logistique urbaine » — 44 — 17 — 61 Pied : « Moyenne des totaux : 63,5 » centrée sur toute la largeur. Contraintes : `<caption>`, `<thead>`, `<tbody>`, `<tfoot>`, `scope`, alignement nombres à droite (classe CSS `.num`), `colspan` au pied.

CORRIGÉ Exercice 1

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Résultats Hackathon Cameroon Digital Boost 2025</title>
  <style>
    table { border-collapse : collapse; width : 100%; font-family : system-ui, sans-serif; }
    th, td { border : 1px solid #333; padding : 0.5rem; }
    th { background : #1e3a5f; color : white; }
    .num { text-align : right; }
    caption { caption-side : top; font-weight : bold; margin-bottom : 0.5rem; font-size : 1.1rem; }
    tfoot td { background : #f0f4f8; font-weight : bold; text-align : center; }
  </style>
</head>
<body>
<table>
  <caption>Résultats — Cameroon Digital Boost 2025</caption>
  <thead>
    <tr>
      <th scope="col">Rang</th>
      <th scope="col">Équipe</th>
      <th scope="col">Projet</th>
      <th scope="col">Score technique (sur 50)</th>
      <th scope="col">Score pitch (sur 20)</th>
      <th scope="col">Total (sur 70)</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td class="num">1</td>
      <td>Team KmerAI</td>
```

```
<td>Diagnostic palu par IA</td>
<td class="num">48</td>
<td class="num">18</td>
<td class="num">66</td>
</tr>
<tr>
<td class="num">2</td>
<td>Team MboaCode</td>
<td>Marketplace agro</td>
<td class="num">45</td>
<td class="num">19</td>
<td class="num">64</td>
</tr>
<tr>
<td class="num">3</td>
<td>Team SiliconMountain</td>
<td>FinTech inclusion</td>
<td class="num">47</td>
<td class="num">16</td>
<td class="num">63</td>
</tr>
<tr>
<td class="num">4</td>
<td>Team DoualaTech</td>
<td>Logistique urbaine</td>
<td class="num">44</td>
<td class="num">17</td>
<td class="num">61</td>
</tr>
</tbody>
<tfoot>
<tr>
<td colspan="6">Moyenne des totaux : 63,5</td>
</tr>
</tfoot>
</table>
</body>
</html>
```

4

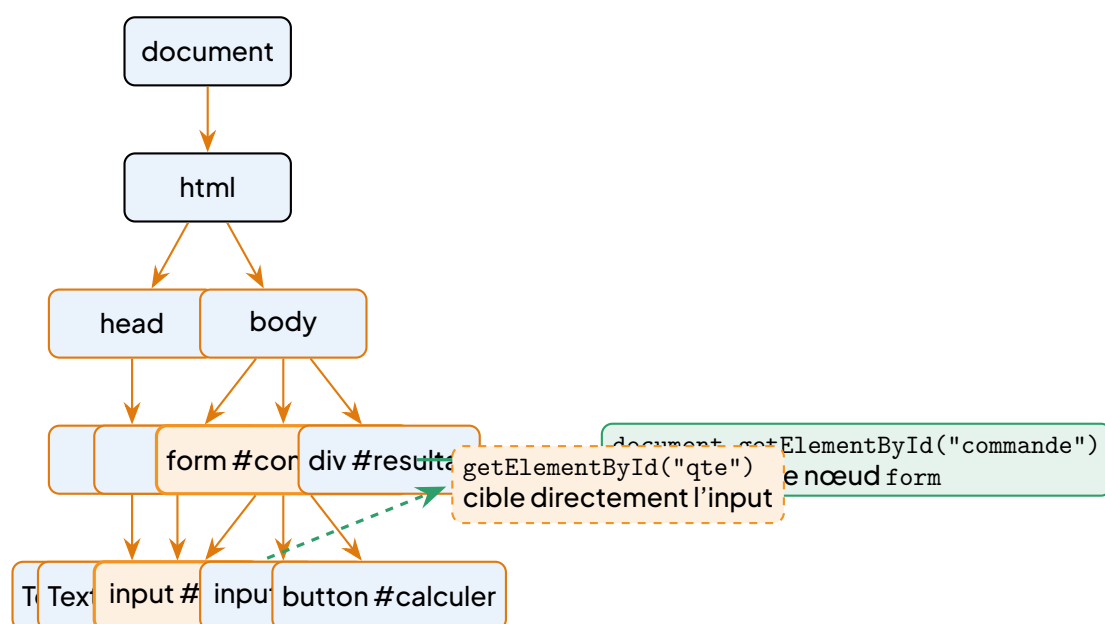
JAVASCRIPT ET LE DOM

Après avoir structuré nos données avec des tableaux HTML (section précédente), nous devons maintenant les rendre vivantes : réagir aux clics, vérifier une saisie, calculer un total, afficher un message d'erreur sans recharger la page. C'est le rôle de **JavaScript**, le langage de programmation natif du navigateur. Pour agir sur la page, JavaScript a besoin d'une représentation manipulable du document HTML : c'est le **DOM** (Document Object Model).

4.1 Le DOM : la page web comme arbre d'objets

DÉFINITION DOM et nœud

Le DOM (Document Object Model) est une interface de programmation (API) qui représente la page HTML sous la forme d'un arbre de nœuds (tree). Chaque balise HTML, chaque attribut, chaque portion de texte devient un nœud (Node) dans cet arbre. Le nœud racine est l'objet `document` (l'entrée unique pour accéder à la page). JavaScript parcourt et modifie cet arbre pour transformer l'affichage ou le comportement de la page sans recharger.

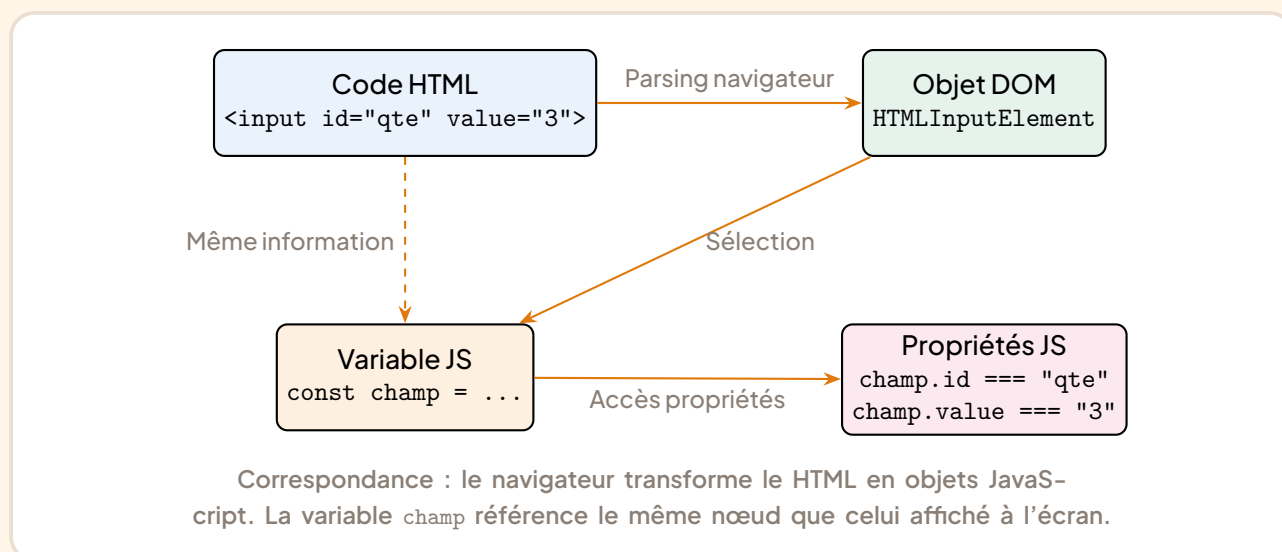


Arbre DOM simplifié : chaque balise devient un nœud. Les nœuds surlignés (orange) sont ciblés par `getElementById`. Les flèches vertes montrent la sélection programmatique.

PROPRIÉTÉ Correspondance HTML ↔ DOM

- Balise HTML → Element (nœud de type élément)
- Attribut (id, class, value...) → propriété de l'objet Element (`element.id`, `element.value`...)
- Texte entre balises → Text (nœud de type texte), accessible via `textContent`
- Commentaire → Comment

L'objet global `document` est la porte d'entrée : `document.getElementById(...)` cherche dans tout l'arbre.



4.2 Intégrer du JavaScript dans la page

Deux méthodes standards, la seconde étant **recommandée** pour la maintenabilité (séparation structure / comportement).

MÉTHODE Intégrer un script JavaScript

1. Balise `<script>` en fin de `<body>` (juste avant `</body>`) : le HTML est déjà parsé, le DOM est prêt.
2. Fichier externe `.js` (meilleure pratique) : `<script src="script.js" defer></script>` dans le `<head>`. L'attribut `defer` garantit l'exécution après la construction du DOM, dans l'ordre des scripts.

EXEMPLE Fichier externe `script.js` lié à `index.html`

```
<!-- index.html -->
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Commande - Exemple DOM</title>
  <script src="script.js" defer></script>
</head>
<body>
  <form id="commande">
    <label for="qte">Quantité :</label>
    <input type="number" id="qte" name="qte" min="1" value="1">
    <button type="button" id="calculer">Calculer le montant</button>
  </form>
  <div id="resultat"></div>
```

```
</body>
</html>
```

```
// script.js
// Tout le code ici s'exécute APRES la construction complète du DOM
console.log("DOM prêt, script chargé.");
// ... logique de sélection et d'événements (sections suivantes)
```

4.3 Sélectionner des éléments du DOM

PROPRIÉTÉ Méthodes de sélection principales

Méthode	Description	Retour
<code>document.getElementById("id")</code>	Sélectionne l'unique élément dont l'attribut <code>id</code> correspond.	Element ou null
<code>document.querySelector("selecteurCSS")</code>	Premier élément correspondant au sélecteur CSS (id, classe, attribut, combinateurs).	Element ou null
<code>document.querySelectorAll("selecteurCSS")</code>	Tous les éléments correspondants (liste statique <code>NodeList</code>).	<code>NodeList</code> (itérable)
<code>document.getElementsByTagName("tag")</code>	Tous les éléments de ce nom de balise (liste vivante <code>HTMLCollection</code>).	<code>HTMLCollection</code>

MÉTHODE Sélectionner un élément et lire sa valeur — 3 étapes

1. Identifier l'élément dans le HTML : ajouter un `id` unique (`<input id="qte">`).

2. Sélectionner le nœud DOM : `const champ = document.getElementById("qte");`
3. Lire la valeur : `const valeur = champ.value;` (renvoie toujours une chaîne de caractères).

ATTENTION Piège classique : `.value` renvoie une chaîne

`input.value` est toujours de type `string`, même pour `<input type="number">`.

```
const qte = document.getElementById("qte").value; // "3" (string)
console.log(typeof qte); // "string"
const total = qte * 25000; // JS convertit implicitement, MAIS...
const total2 = qte + 10; // "310" (concaténation !) -> BUG
```

Règle d'or : convertir explicitement avant tout calcul : `Number(qte)`, `parseInt(qte, 10)`, `parseFloat(qte)` ou `+qte` (unaire plus).

4.4 Manipuler le contenu et les styles

Une fois l'élément sélectionné (stocké dans une variable), on agit sur ses propriétés.

PROPRIÉTÉ Propriétés clés de manipulation

- `element.value` : lire/écrire la valeur d'un champ de formulaire (`input`, `select`, `textarea`).
- `element.textContent` : lire/écrire le texte brut (sécurisé, ignore le HTML). Préféré pour l'affichage utilisateur.
- `element.innerHTML` : lire/écrire le contenu HTML (interprète les balises). Attention : risque XSS si on y injecte des données non contrôlées.
- `element.style` : objet donnant accès aux propriétés CSS en camelCase (`style.backgroundColor`, `style.border`, `style.display...`).

EXEMPLE Mettre une bordure rouge en cas d'erreur + afficher un message

```
const champ = document.getElementById("qte");
const msg = document.getElementById("message-erreur");

function verifier() {
  const val = Number(champ.value);
  if (isNaN(val) || val < 1) {
    // Style inline via element.style
  }
}
```

```
    champ.style.border = "2px solid red";
    champ.style.backgroundColor = "#fff0f0";
    // Contenu texte sécurisé
    msg.textContent = "Veuillez saisir une quantité valide ≥ (1).";
    msg.style.color = "red";
    return false;
}
// Succès : reset styles
champ.style.border = "";
champ.style.backgroundColor = "";
msg.textContent = "";
return true;
}
```

4.5 Rappels JavaScript indispensables

Avant d'écrire la logique de validation, rappelons les bases du langage (ES6+).

PROPRIÉTÉ Variables, types et conversions

- `const` (réaffectation impossible, par défaut), `let` (réaffectable), `var` (obsolète, portée fonction).
- `typeof x` : "string", "number", "boolean", "object", "undefined", "function", "bigint", "symbol".
- Conversion en nombre :
 - `Number("3.14")` → 3,14 (strict, renvoie NaN si invalide)
 - `parseInt("3px", 10)` → 3 (parse jusqu'au premier caractère non numérique, toujours préciser la base 10)
 - `parseFloat("3.14")` → 3,14
 - `+"3.14"` (unaire plus) → 3,14 (court, idiomatique)
- `isNaN(x)` : vrai si `x` est NaN (Not-a-Number). Astuce : `Number.isNaN(x)` est plus fiable (ne convertit pas l'argument).

À RETENIR Check-list conversion saisie utilisateur

1. Récupérer `.value` (string).
2. Convertir : `const n = Number(valeur);` OU `const n = parseInt(valeur, 10);`.
3. Tester : `if (isNaN(n) || n < min || n > max) { ... erreur ... }.`

4. Utiliser `n` (number) pour les calculs.

4.6 La console du navigateur : votre meilleur ami pour déboguer

EXPÉRIENCE Découvrir la console développeur (F12)

- Objectif : Inspecter le DOM, tester des sélecteurs, tracer des valeurs, repérer les erreurs JS.
- Outils : Navigateur (Chrome, Firefox, Edge) — touche F12 ou clic droit Inspecter.
- Manipulation :
 1. Ouvrir la page `index.html` (exemple ci-dessus).
 2. Appuyer sur F12 → onglet Console.
 3. Taper : `document.getElementById("qte")` → Entrée. La console affiche l'élément ; cliquer dessus le surligne dans l'onglet Elements.
 4. Taper : `document.getElementById("qte").value` → Entrée. Affiche la valeur courante.
 5. Dans `script.js`, ajouter `console.log("Valeur lue :", champ.value)` ; puis recharger. Le message apparaît dans la console.
 6. Provoquer une erreur (ex. `champ.valu`) : la console affiche `Uncaught TypeError` en rouge avec le numéro de ligne.
- Observation : La console est un REPL (Read-Eval-Print Loop) live : on y teste n'importe quelle expression JS sur la page courante.
- Interprétation : `console.log()` pour tracer, `console.error()` pour les erreurs volontaires, `console.table()` pour les tableaux/objets. Indispensable avant de coder la validation finale.

4.7 Exemple complet : calcul du montant d'une commande (contexte camerounais)

EXERCICE RÉSOLU 4 Calcul montant sac de riz — 25 000 FCFA l'unité

Énoncé : Créer une page permettant à un commerçant du marché Mokolo de saisir la quantité de sacs de riz (50 kg) commandée par un client. Le prix unitaire est fixé à 25 000 FCFA. Au clic sur le bouton, afficher le montant total formaté (ex. « 75 000 FCFA ») ou un message d'erreur si la saisie est invalide.

SOLUTION

Solution détaillée :

1. HTML (index.html) — structure sémantique, id uniques, type="number" avec min.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Calcul commande riz – Marché Mokolo</title>
  <script src="script.js" defer></script>
  <style>
    body { font-family : system-ui, sans-serif; max-width : 480px; margin : 2rem auto; padding : 0 1rem; }
    .form-group { margin-bottom : 1rem; }
    label { display : block; margin-bottom : .3rem; font-weight : bold; }
    input[type="number"] { width : 100%; padding : .6rem; font-size : 1.1rem; box-sizing : border-box; }
    button { width : 100%; padding : .8rem; font-size : 1.1rem; background : #006400; color : white; border : none; border-radius : 4px; cursor : pointer; }
    button:hover { background : #004d00; }
    #resultat { margin-top : 1.5rem; padding : 1rem; border-radius : 4px; font-size : 1.2rem; font-weight : bold; text-align : center; display : none; }
    .success { background : #e8f5e9; color : #1b5e20; border : 1px solid #4caf50; }
    .error { background : #fdecea; color : #c62828; border : 1px solid #ef5350; }
    .input-error { border : 2px solid #ef5350 !important; background : #fdecea; }
  </style>
</head>
<body>
  <h1>Calcul montant commande</h1>
  <p>Prix unitaire : <strong>25 000 FCFA</strong> / sac de 50 kg</p>

  <form id="commande" novalidate>
    <div class="form-group">
      <label for="qte">Quantité (sacs) :</label>
      <input type="number" id="qte" name="qte" min="1" step="1" value="1" required autocomplete="off">
    </div>
    <button type="button" id="calculer">Calculer le montant</button>
  </form>

  <div id="resultat"></div>
</body>
</html>
```

2. JavaScript (script.js) — sélection, écouteur d'événement click, validation, conversion, affichage.

```
// script.js
// 1. Sélection des éléments (exécuté APRES construction DOM grâce à defer)
const champQte = document.getElementById("qte");
const btnCalculer = document.getElementById("calculer");
const zoneResultat = document.getElementById("resultat");

const PRIX_UNITAIRE = 25000; // FCFA – constante métier
```

```
// 2. Fonction de validation et calcul
function calculerMontant(){
  // a) Lecture brute (string)
  const valeurBrute = champQte.value.trim();

  // b) Conversion explicite en nombre
  const quantite = Number(valeurBrute);

  // c) Validation robuste
  if (valeurBrute === "" || isNaN(quantite) || quantite < 1 || !Number.isInteger(quantite)){
    afficherErreur("Veuillez saisir un nombre entier de sacs (minimum 1).");
    champQte.classList.add("input-error");
    champQte.focus(); // UX : replace le curseur
    return;
  }

  // d) Calcul métier
  const montant = quantite * PRIX_UNITAIRE;

  // e) Formatage affichage (séparateur d'espace pour les milliers)
  const montantFormate = montant.toLocaleString("fr-FR") + " FCFA";

  // f) Affichage succès
  afficherSucces("Montant total : ${montantFormate}");
  champQte.classList.remove("input-error");
}

// 3. Fonctions d'affichage (séparation responsabilités)
function afficherSucces(message){
  zoneResultat.textContent = message;
  zoneResultat.className = "success"; // reset + classe unique
  zoneResultat.style.display = "block";
}

function afficherErreur(message){
  zoneResultat.textContent = message;
  zoneResultat.className = "error";
  zoneResultat.style.display = "block";
}

// 4. Attachement de l'événement (section suivante : Gestion des événements)
btnCalculer.addEventListener("click", calculerMontant);

// 5. Bonus : validation à la saisie (retour visuel immédiat)
champQte.addEventListener("input", () => {
  if (champQte.classList.contains("input-error")){
    const val = Number(champQte.value);
    if (!isNaN(val) && val >= 1 && Number.isInteger(val)){
      champQte.classList.remove("input-error");
      zoneResultat.style.display = "none";
    }
  }
});

console.log("Application initialisée. Prête pour les commandes.");
```

Points clés illustrés :

- `defer` sur le script → DOM prêt, pas besoin de `DOMContentLoaded`.
- `Number()` + `isNaN()` + `Number.isInteger()` pour une validation stricte.
- `toLocaleString("fr-FR")` : formatage local (espace insécable comme séparateur de milliers), adapté au contexte camerounais.
- `classList` (`add/remove/contains`) » manipulation de `style` directement : plus maintenable, CSS gère l'apparence.
- `trim()` sur la valeur brute : évite les espaces invisibles.
- `novalidate` sur le `<form>` : on gère la validation nous-mêmes en JS (côté client), le navigateur ne bloque pas la soumission (le bouton est `type="button"`, pas `submit`).

LESAVAIS-TU ? JavaScript ≠ Java

JavaScript n'a aucun lien technique avec Java. En 1995, Brendan Eich (Netscape) crée Mocha → LiveScript en 10 jours. Netscape, pour surfer sur la hype Java (Sun Microsystems), renomme le langage JavaScript via un accord de licence marketing. La syntaxe s'inspire de C/Java (accolades, point-virgule), mais le modèle objet est à prototypes (pas de classes avant ES2015), le typage est dynamique et faible, et l'exécution se fait dans le navigateur (moteur V8, SpiderMonkey, JavaScriptCore), pas sur une JVM. Java sert au backend/entreprise ; JavaScript règne sur le frontend (et maintenant le backend via Node.js, Deno, Bun).

EXERCICE 3 Entraînement : sélection et conversion

Écrire un script qui :

1. Sélectionne l'input `<input id="age" type="number" min="0">`.
2. Lit sa valeur au clic sur `<button id="verifier">`.
3. Affiche dans `<div id="info">` : « Majeur » si âge ≥ 18, « Mineur » sinon, « Saisie invalide » si vide ou non numérique.

Indication : utilise `Number()`, `isNaN()`, `textContent`, `classList`.

CORRIGÉ Exercice : sélection et conversion

```
const inputAge = document.getElementById("age");
const btn = document.getElementById("verifier");
const info = document.getElementById("info");
```

```
btn.addEventListener("click", () => {
  const val = inputAge.value.trim();
  const age = Number(val);

  if (val === "" || isNaN(age) || age < 0) {
    info.textContent = "Saisie invalide";
    info.className = "error";
  } else if (age >= 18) {
    info.textContent = "Majeur";
    info.className = "success";
  } else {
    info.textContent = "Mineur";
    info.className = "success"; // ou "info" selon charte
  }
  info.style.display = "block";
});
```

4.8 Transition vers la gestion des événements

Nous savons maintenant **sélectionner** un élément, **lire** sa valeur, **convertir** le type, **modifier** son style ou son contenu. Il manque la **réactivité** : exécuter ce code au bon moment (clic, saisie, soumission, chargement). La section suivante — **Gestion des événements** — couvre `addEventListener`, l'objet `Event`, les événements `click`, `input`, `submit`, `change`, et la prévention du comportement par défaut (`preventDefault()`). C'est la clé pour transformer une page statique en application interactive.

5 GESTION DES ÉVÉNEMENTS

5.1 Introduction : de la structure statique à l'interaction dynamique

Dans la section précédente (JavaScript et le DOM), nous avons appris à **sélectionner** des éléments HTML (`document.getElementById`, `querySelector`) et à **modifier** leur contenu, leurs attributs ou leur style (`textContent`, `value`, `classList`). Nous savons désormais comment agir sur la page.

Il reste à répondre à la question fondamentale : **quand** agir ? Une page web ne doit pas s'exécuter uniquement au chargement ; elle doit réagir aux actions de l'utilisateur : un clic sur un bouton, une saisie dans un champ, l'envoi d'un formulaire, le passage de la souris, etc. C'est le rôle de la **programmation événementielle**.

LESAVAIS-TU ? Le cœur des interfaces modernes

La programmation événementielle (event-driven programming) n'est pas une particularité du web. C'est le paradigme central de toutes les interfaces graphiques modernes : appli-

cations mobiles (Android, iOS), logiciels de bureau (Qt, WPF, Electron), jeux vidéo (Unity, Unreal Engine), et bien sûr le navigateur web. Maîtriser les événements en JavaScript, c'est acquérir une compétence transférable à tout développement d'interface utilisateur.

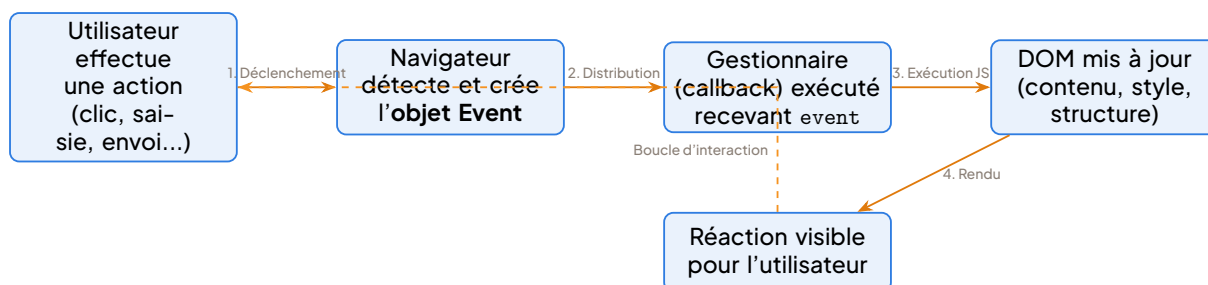
5.2 Notion d'événement et de gestionnaire

DÉFINITION Événement DOM

Un événement est un signal émis par le navigateur pour indiquer qu'une action spécifique s'est produite : interaction utilisateur (clic, frappe clavier, déplacement souris), cycle de vie de la page (chargement, déchargement), erreur réseau, fin d'une animation CSS, etc.

DÉFINITION Gestionnaire d'événement (Event Listener / Callback)

Un gestionnaire d'événement (ou écouteur, callback) est une fonction JavaScript qui sera exécutée automatiquement par le navigateur après la survenue de l'événement ciblé sur un élément donné.



Cycle de vie d'un événement : de l'action utilisateur à la mise à jour de l'interface.

5.3 Les événements essentiels pour les formulaires

Pour la famille de situations « Développement d'un site web ; contrôle des données utilisateur », cinq événements sont indispensables.

PROPRIÉTÉ Événements formulaires — Mémo

Nom	Déclencheur typique	Cible habituelle
click	Clic souris (bouton gauche) sur l'élément	<code><button></code> , <code><a></code> , <code><input type="button"></code>
input	Chaque modification de la valeur d'un champ (texte, checkbox, range, color)	<code><input></code> , <code><textarea></code> , <code><select></code>
change	Perte du focus après modification de la valeur (pour select, checkbox, radio, file)	<code><select></code> , <code><input type="checkbox/radio/file"></code>
submit	Tentative d'envoi du formulaire (bouton <code>type="submit"</code> ou touche Entrée)	<code><form></code>
keyup	Relâchement d'une touche clavier (utile pour filtres en temps réel)	<code><input></code> , <code><textarea></code> , <code>document</code>

ATTENTION input vs change : le piège classique

- `input` : se déclenche à chaque frappe (idéal pour aperçu instantané, compteur de caractères, validation continue).
- `change` : se déclenche une seule fois quand l'utilisateur quitte le champ (perte de focus) et que la valeur a changé (idéal pour validation finale, sauvegarde automatique).
- Pour un `<select>`, `change` est le standard ; `input` ne se déclenche pas sur certains navigateurs.

5.4 Trois façons d'associer un gestionnaire — Historique et bonne pratique

Il existe trois syntaxes pour lier un événement à une fonction. Seule la dernière est recommandée en développement professionnel.

Méthode	Syntaxe	Avantages		Verdict
1. Attribut HTML onclick="..."	<code><button onclick="calculer()"></code>	Simple, visible dans le HTML	Mélange HTML/JS (séparation des responsabilités violée), un seul ges- tionnaire possible, Un seul gestionnaire par événement (écrase le pré- cédent), pas de contrôle sur la phase de capture	À proscrire
2. Propriété DOM element.onclick	<code>btn.onclick = calculer;</code>	Séparation HTML/JS	Standard moderne, robuste, extensible	Dépassé (héritage DOM Level 0)
3. addEventListener() (Standard W3C)	<code>btn.addEventListener('click', calculer);</code>	Multiple gestionnaires sur le même événe- ment, contrôle capture/bubbling (useCapture), options once, passive, sup- pression possible (removeEventListener)		Recommandée exclusivement

Comparatif des trois méthodes d'association d'événements. Seule `addEventListener` offre la robustesse nécessaire aux applications professionnelles.

5.5 La méthode recommandée : `addEventListener()`

PROPRIÉTÉ Syntaxe `addEventListener`

```
element.addEventListener(type, listener[, options]);
```

- **type (chaîne)** : nom de l'événement sans préfixe « on » ('click', 'submit', 'input', 'keyup'...).
- **listener (fonction)** : la fonction callback à exécuter. Sans parenthèses d'appel.
- **options (objet, optionnel)** : { once: true } (auto-suppression après 1 exécution), { passive: true } (optimisation scroll), { capture: true } (phase de capture).

MÉTHODE Réagir à un clic sur un bouton — Démarche standard

1. Sélectionner l'élément déclencheur (bouton, lien, etc.) via `document.querySelector` ou `getElementById`.
2. Définir la fonction de traitement (callback) — soit nommée, soit fléchée anonyme.
3. Enregistrer l'écouteur avec `addEventListener('click', callback)`.
4. (Optionnel) Exploiter l'objet `event` passé en argument pour accéder à la cible (`event.target`) ou annuler un comportement par défaut (`event.preventDefault()`).

EXEMPLE Bouton « Calculer » — Affichage dynamique sans rechargement

Contexte : Un simulateur de crédit simple. L'utilisateur saisit un montant, clique sur « Calculer », le montant des mensualités s'affiche instantanément.

```
<!-- index.html -->
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Simulateur Crédit - OnBuch+</title>
  <style>
    .resultat { color : #006400; font-weight : bold; margin-top : 1rem; }
    .erreur { color : #cc0000; margin-top : 1rem; }
  </style>
</head>
<body>
  <h1>Simulateur de crédit (Banque Camerounaise)</h1>
  <form id="form-credit" novalidate> <!-- novalidate : on gère la validation en JS -->
    <div>
      <label for="montant">Montant souhaité (FCFA) :</label>
      <input type="number" id="montant" name="montant" min="10000" step="1000" required
        placeholder="Ex : 500000">
    </div>
    <div>
      <label for="duree">Durée (mois) :</label>
      <select id="duree" name="duree" required>
        <option value="">-- Choisir --</option>
        <option value="12">12 mois</option>
        <option value="24">24 mois</option>
        <option value="36">36 mois</option>
      </select>
    </div>
    <button type="button" id="btn-calculer">Calculer la mensualité</button>
    <button type="reset" id="btn-reset">Réinitialiser</button>
  </form>
  <div id="zone-resultat" class="resultat" aria-live="polite"></div>
  <div id="zone-erreur" class="erreur" aria-live="assertive"></div>

  <script src="app.js"></script>
</body>
</html>
```

```
// app.js
// 1. Sélection des éléments du DOM
const form = document.getElementById('form-credit');
const inputMontant = document.getElementById('montant');
const selectDuree = document.getElementById('duree');
const btnCalculer = document.getElementById('btn-calculer');
const btnReset = document.getElementById('btn-reset');
const zoneResultat = document.getElementById('zone-resultat');
const zoneErreur = document.getElementById('zone-erreur');

// Constantes métier (taux annuel fictif)
const TAUX_ANNUEL = 0.075; // 7,5 %
```

```
// 2. Définition du gestionnaire (callback) pour le clic
function calculerMensualite() {
  // Nettoyage affichages précédents
  zoneResultat.textContent = '';
  zoneErreur.textContent = '';

  // 3. Récupération et validation basique des valeurs
  const montant = Number(inputMontant.value);
  const duree = Number(selectDuree.value);

  if (isNaN(montant) || isNaN(duree)) {
    zoneErreur.textContent = 'Veuillez remplir tous les champs correctement.';
    return; // Sortie anticipée
  }

  // 4. Calcul (formule standard mensualité constante)
  //  $M = C * (t/12) / (1 - (1 + t/12)^{-n})$ 
  const tauxMensuel = TAUX_ANNUEL / 12;
  const mensualite = montant * tauxMensuel / (1 - Math.pow(1 + tauxMensuel, -duree));

  // 5. Mise à jour du DOM (affichage)
  zoneResultat.innerHTML =
    'Mensualité estimée : <strong>${mensualite.toLocaleString('fr-CM', {style: 'currency', currency: 'XAF', minimumFractionDigits: 0})}</strong><br>' +
    'Coût total du crédit : <strong>${(mensualite * duree).toLocaleString('fr-CM', {style: 'currency', currency: 'XAF', minimumFractionDigits: 0})}</strong>';
}

// 6. Enregistrement de l'écouteur (BONNE PRATIQUE : sans parenthèses !)
btnCalculer.addEventListener('click', calculerMensualite);

// 7. Gestion du bouton Reset (nettoyage zones de texte)
btnReset.addEventListener('click', () => {
  zoneResultat.textContent = '';
  zoneErreur.textContent = '';
});

// 8. Bonus : validation en temps réel sur le champ montant (événement input)
inputMontant.addEventListener('input', () => {
  const val = Number(inputMontant.value);
  if (val < 10000) {
    inputMontant.setCustomValidity('Minimum 10 000 FCFA');
    inputMontant.reportValidity(); // Affiche bulle navigateur
  } else {
    inputMontant.setCustomValidity(''); // Valide
  }
});
```

ATTENTION Le piège des parenthèses : `maFonction()` vs `maFonction`

C'est l'erreur numéro 1 des débutants.

- `btn.addEventListener('click', calculerMensualite);` → Correct. On passe la référé-

rence de la fonction. Le navigateur l'appellera plus tard.

- `btn.addEventListener('click', calculerMensualite());` → Incorrect. Les parenthèses exécutent la fonction immédiatement au chargement de la page. `addEventListener` reçoit le résultat de la fonction (souvent `undefined`) au lieu de la fonction elle-même. Le clic ne fera ensuite plus rien (erreur silencieuse).

Règle d'or : Dans `addEventListener`, jamais de parenthèses après le nom de la fonction callback.

5.6 L'objet `event` : informations contextuelles et contrôle

Lorsqu'un gestionnaire est appelé, le navigateur lui transmet automatiquement un unique argument : l'**objet Event** (souvent nommé `e`, `evt` ou `event`). Cet objet est une mine d'informations.

PROPRIÉTÉ Propriétés et méthodes clés de l'objet `Event`

Propriété / Méthode	Utilité
<code>event.target</code>	L'élément exact qui a déclenché l'événement (celui sur lequel on a cliqué, dans lequel on a tapé). Différent de <code>this</code> (ou <code>event.currentTarget</code>) qui est l'élément sur lequel l'écouteur est attaché.
<code>event.currentTarget</code>	L'élément sur lequel <code>addEventListener</code> a été appelé (équivalent à <code>this</code> dans une fonction classique, pas dans une fléchée).
<code>event.type</code>	Nom de l'événement ('click', 'submit'...).
<code>event.preventDefault()</code>	Annule le comportement par défaut du navigateur pour cet événement. Ex : empêcher l'envoi du formulaire (<code>submit</code>), empêcher le suivi d'un lien (clic sur <a>), empêcher la saisie d'un caractère (<code>keydown</code>).
<code>event.stopPropagation()</code>	Empêche l'événement de remonter vers les parents (bubbling). Utile pour éviter de déclencher un clic sur un conteneur quand on clique sur un bouton à l'intérieur.
<code>event.key</code> / <code>event.code</code>	(Clavier) Touche pressée.
<code>event.clientX</code> / <code>event.clientY</code>	(Souris) Position du curseur par rapport au viewport.

EXEMPLE Délégation d'événements — `event.target` en action

Plutôt que de mettre un écouteur sur chaque ligne d'un tableau dynamique (coûteux en mémoire), on met un seul écouteur sur le parent (`<tbody>`) et on inspecte `event.target`.

```
// HTML : <table id="tab-utilisateurs"><tbody>...<tr><td>...</td><td><button class="btn-suppr">X</button></td></tr>...</tbody></table>

const tbody = document.querySelector('#tab-utilisateurs tbody');

tbody.addEventListener('click', (e) => {
  // e.target est l'élément cliqué (le <button> ou le <td> si on rate le bouton)
  // On remonte au bouton s'il a la classe visée (closest)
  const btn = e.target.closest('.btn-suppr');

  if (btn) {
    // Trouver la ligne <tr> parente
    const ligne = btn.closest('tr');
    if (ligne && confirm('Supprimer cet utilisateur ?')) {
      ligne.remove(); // Suppression DOM immédiate
      // Ici : appel AJAX fetch() pour suppression BDD réelle
    }
  }
  // Si clic ailleurs dans le tbody (sur un <td>), btn est null -> ne fait rien
});
```

5.7 Cas critique : Intercepter la soumission (`submit`)

C'est le point central du **contrôle de saisie côté client**. Par défaut, un formulaire (`<form>`) avec un bouton `type="submit"` recharge la page (ou navigue vers l'action) en envoyant les données. Pour valider en JavaScript **avant** l'envoi (ou envoyer via AJAX/Fetch), il faut **intercepter l'événement** `submit` sur le `<form>` (pas sur le bouton!) et appeler `event.preventDefault()`.

MÉTHODE Valider et traiter un formulaire sans rechargement (AJAX/Fetch)

1. Sélectionner le `<form>` (pas le bouton).
2. Attacher `addEventListener('submit', callback)`.
3. Dans le callback : `event.preventDefault()` immédiatement.
4. Récupérer les données : `new FormData(form)`.
5. Valider côté client (voir Section 6).
6. Si valide : envoyer via `fetch()` (API moderne) ou `XMLHttpRequest`.
7. Gérer la réponse (succès/erreur) et mettre à jour le DOM.

EXERCICE RÉSOLU 5 Formulaire d'inscription newsletter — Interception submit + Fetch

Énoncé : Créer un formulaire d'inscription (email). Au soumission, vérifier le format email côté client. Si valide, envoyer via fetch vers /api/inscrire (simulé). Afficher le résultat sans recharger la page.

Solution détaillée :

```
<!-- newsletter.html -->
<form id="form-newsletter" action="/api/inscrire" method="POST">
  <label for="email">Votre email (ex : jean@camtel.cm) :</label>
  <input type="email" id="email" name="email" required placeholder="votre@email.cm">
  <button type="submit">S'inscrire</button>
</form>
<div id="message-feedback" aria-live="polite"></div>
```

```
// newsletter.js
const form = document.getElementById('form-newsletter');
const feedback = document.getElementById('message-feedback');

form.addEventListener('submit', async (e) => {
  // 1. ANNULER le comportement par défaut (rechargement page)
  e.preventDefault();

  // 2. Récupération données (FormData gère l'encodage multipart/urlencoded)
  const formData = new FormData(form);
  const email = formData.get('email').trim();

  // 3. Validation côté client (double sécurité : HTML5 'required' + 'type=email' + JS)
  // Regex simplifiée pour l'exemple (RFC 5322 est trop complexe)
  const regexEmail = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!regexEmail.test(email)) {
    feedback.textContent = 'Format d\'email invalide.';
    feedback.className = 'erreur'; // CSS .erreur { color : red; }
    return;
  }

  // 4. Préparation UI (état chargement)
  feedback.textContent = 'Inscription en cours...';
  feedback.className = 'info';
  const btnSubmit = form.querySelector('button[type="submit"]');
  btnSubmit.disabled = true; // Éviter double clic

  try {
    // 5. Envoi asynchrone (Fetch API - Standard moderne)
    // Note : En TP réel, l'URL pointerait vers un backend (Node/PHP/Python)
    const response = await fetch(form.action, { // form.action = "/api/inscrire"
      method : 'POST',
      body : formData, // Envoie automatiquement en multipart/form-data
      headers : {
        'Accept' : 'application/json' // On attend du JSON en retour
      }
    });

    const result = await response.json(); // Parse JSON réponse
```

```
if (response.ok && result.success) {
    feedback.textContent = '[OK] ${result.message} (${email})';
    feedback.className = 'succes';
    form.reset(); // Vide le formulaire
} else {
    // Erreur métier (ex : email déjà existant)
    feedback.textContent = '[ERREUR] ${result.message || 'Erreur serveur'}';
    feedback.className = 'erreur';
}
} catch (err) {
    // Erreur réseau / CORS / JSON invalide
    console.error('Erreur fetch :', err);
    feedback.textContent = '[ATTENTION] Erreur de connexion. Réessayez plus tard.';
    feedback.className = 'erreur';
} finally {
    btnSubmit.disabled = false; // Réactive le bouton
}
});
```

ATTENTION Pourquoi écouter `submit` sur le `<form>` et non `click` sur le bouton ?

- Un formulaire peut être soumis de trois façons : clic sur bouton `type="submit"`, touche Entrée dans un champ, `form.submit()` en JS.
- L'événement `submit` sur le `<form>` capture tous ces cas. L'événement `click` sur le bouton rate la touche Entrée et l'appel JS.
- `e.preventDefault()` sur `submit` annule l'envoi HTTP natif. Sur `click`, il n'annule que le clic, le formulaire peut quand même partir si `form.submit()` est appelé ailleurs.

5.8 Exemple complet : Validateur de mot de passe en temps réel

Cet exemple combine `input` (réactivité), `submit` (blocage final), `event.target`, `classList` et messages accessibles.

EXPÉRIENCE TP Guidé : Indicateur de force de mot de passe

Objectif : Créer un champ mot de passe avec une barre de progression visuelle (faible/moyen/fort) qui se met à jour à chaque frappe, et bloquer l'envoi si le mot de passe est « Faible ».

Mise en place : Fichier `mdp.html` + `mdp.js`.

```
<!-- mdp.html -->
<!DOCTYPE html>
<html lang="fr">
<head>
```

```

<meta charset="UTF-8">
<title>Force du mot de passe</title>
<style>
  .strength-bar { height : 8px; border-radius : 4px; background : #eee; overflow : hidden; margin :
    0.5rem 0; }
  .strength-bar div { height : 100%; width : 0%; transition : width 0.3s, background 0.3s; }
  .faible div { width : 33%; background : #e74c3c; }
  .moyen div { width : 66%; background : #f39c12; }
  .fort div { width : 100%; background : #27ae60; }
  .feedback { font-size : 0.9rem; min-height : 1.2em; }
  .erreur { color : #c0392b; }.succes { color : #27ae60; }
  button:disabled { opacity : 0.6; cursor : not-allowed; }
</style>
</head>
<body>
  <h1>Création de compte – Sécurité</h1>
  <form id="form-mdp" novalidate>
    <div>
      <label for="pwd">Mot de passe :</label>
      <input type="password" id="pwd" name="pwd" required minlength="8"
        aria-describedby="pwd-feedback pwd-strength">
      <div id="pwd-strength" class="strength-bar" aria-hidden="true"><div></div></div>
      <div id="pwd-feedback" class="feedback" aria-live="polite"></div>
    </div>
    <button type="submit" id="btn-valider" disabled>Valider</button>
  </form>
  <script src="mdp.js"></script>
</body>
</html>

```

```

// mdp.js
const form = document.getElementById('form-mdp');
const input = document.getElementById('pwd');
const barContainer = document.getElementById('pwd-strength');
const feedback = document.getElementById('pwd-feedback');
const btn = document.getElementById('btn-valider');

// 1. Événement INPUT : Analyse à chaque frappe
input.addEventListener('input', (e) => {
  const pwd = e.target.value; // event.target = l'input
  const strength = calculerForce(pwd); // 0, 1, 2, 3

  // MAJ Barre visuelle (classes CSS)
  barContainer.className = 'strength-bar'; // Reset
  if (strength > 0) barContainer.classList.add(['faible', 'moyen', 'fort'][strength - 1]);

  // MAJ Texte feedback
  const messages = ['', 'Faible : ajoutez longueur/chiffres/maj', 'Moyen : ajoutez symboles/longueur', 'Fort !'];
  feedback.textContent = messages[strength];
  feedback.className = 'feedback' + (strength >= 2 ? 'succes' : 'erreur');

  // Activation bouton seulement si Fort (niveau 3)
  btn.disabled = (strength < 3);
});

```

```
// 2. Fonction pure (testable, réutilisable) – Algorithme de scoring
function calculerForce(pwd) {
  if (pwd.length === 0) return 0;
  let score = 0;
  // Critères (inspirés zxcvbn / NIST)
  if (pwd.length >= 8) score++;
  if (pwd.length >= 12) score++;
  if (/[A-Z]/.test(pwd)) score++; // Majuscule
  if (/[0-9]/.test(pwd)) score++; // Chiffre
  if (/[^A-Za-z0-9]/.test(pwd)) score++; // Symbole

  // Normalisation sur 3 niveaux
  if (score <= 2) return 1; // Faible
  if (score <= 4) return 2; // Moyen
  return 3; // Fort
}

// 3. Événement SUBMIT : Dernière barrière
form.addEventListener('submit', (e) => {
  // La validation 'input' a déjà géré l'UI, mais on revérifie SÉCURITAIREMENT
  if (calculerForce(input.value) < 3) {
    e.preventDefault(); // Bloquer l'envoi définitif
    feedback.textContent = 'Mot de passe trop faible. Renforcez-le.';
    feedback.className = 'feedback erreur';
    input.focus();
  } else {
    // Succès : envoi réel (ici simulation)
    e.preventDefault(); // On empêche quand même pour la démo
    alert('Compte créé avec succès (simulation) !');
    form.reset();
    barContainer.className = 'strength-bar';
    feedback.textContent = '';
    btn.disabled = true;
  }
});
```

Observations attendues :

1. Tapez « abc » → Barre rouge 33%, bouton grisé.
2. Tapez « ABCDEF12 » → Barre orange 66%, bouton grisé.
3. Tapez « ABCDEF12!@ » → Barre verte 100%, bouton actif.
4. Cliquez « Valider » sur mot de passe faible → Bloqué par `preventDefault()`.

Compétence validée : `input` pour UX temps réel, `submit` pour sécurité finale, `event.target`, `classList`, `disabled`, `preventDefault()`.

5.9 Synthèse : Les bons réflexes

À RETENIR Check-list « Gestion des événements »

- Séparer HTML (structure) et JS (comportement) : jamais `onclick="..."` dans le HTML.
- Utiliser exclusivement `element.addEventListener('type', fonction)`.
- Ne pas mettre de parenthèses après le nom de la fonction callback.
- Pour les formulaires : écouter `submit` sur le `<form>`, appeler `e.preventDefault()` tout de suite.
- Utiliser `e.target` (cible réelle) et `e.currentTarget` (porteur de l'écouteur) à bon escient (délégation).
- `input` = instantané (UX) ; `change` = final (validation/sauvegarde) ; `submit` = critique (sécurité/envoi).
- Penser `FormData(form)` pour récupérer les données proprement avant `fetch()`.

EXERCICE 4 Entraînement — Gestion d'événements

1. Créer une page avec une liste `<ul id="taches">` et un formulaire (input text + bouton « Ajouter »). Au `submit`, ajouter une `<li>` avec le texte saisi, vider l'input, sans recharger la page.
2. Ajouter sur chaque `<li>` (via délégation sur le `<ul>`) un clic qui barre le texte (`classList.toggle('faite')`) et un double-clic (`dblclick`) qui supprime la `<li>` après confirmation.
3. Ajouter un compteur « Tâches restantes : X » mis à jour à chaque ajout/suppression/-modification.

CORRIGÉ Exercice 1 – Entraînement

```
// solution.js
const form = document.getElementById('form-tache');
const input = document.getElementById('input-tache');
const liste = document.getElementById('taches');
const compteur = document.getElementById('compteur'); // <span id="compteur">0</span>

function majCompteur() {
  const restantes = liste.querySelectorAll('li:not(.faite)').length;
  compteur.textContent = restantes;
}

// 1. Ajout (submit sur form)
form.addEventListener('submit', (e) => {
```

```
e.preventDefault();
const texte = input.value.trim();
if (!texte) return;

const li = document.createElement('li');
li.textContent = texte;
liste.appendChild(li);

input.value = '';
majCompteur();
});

// 2. Délégation : clic (basculer) / dblclick (supprimer)
liste.addEventListener('click', (e) => {
  const li = e.target.closest('li');
  if (li && e.target === li) { // Clic direct sur li (pas sur un enfant hypothétique)
    li.classList.toggle('faite');
    majCompteur();
  }
});

liste.addEventListener('dblclick', (e) => {
  const li = e.target.closest('li');
  if (li && confirm('Supprimer cette tâche ?')) {
    li.remove();
    majCompteur();
  }
});
```

5.10 Transition vers la Section 6 : Validation de la saisie côté client

Nous maîtrisons désormais le **quand** (événements `input`, `change`, `submit`) et le **comment** (sélection DOM, `preventDefault`, `fetch`, mise à jour UI). La Section 6 va approfondir le **quoi** : les stratégies de **validation de données** (attributs HTML5 `required`, `pattern`, `min/max`, API `Constraint Validation` `checkValidity()`, `setCustomValidity()`, `regex`, messages d'erreur accessibles `aria-live`) pour garantir l'intégrité des données avant qu'elles ne quittent le navigateur.

6

VALIDATION DE LA SAISIE CÔTÉ CLIENT

Après avoir maîtrisé la sélection d'éléments dans le DOM et la gestion des événements (clic, saisie, survol), nous abordons maintenant l'une des compétences les plus concrètes et les plus valorisantes au Baccalauréat : **contrôler la saisie utilisateur avant l'envoi au serveur**. Tu vas apprendre à transformer un formulaire passif en une interface intelligente qui guide l'utilisateur, évite les erreurs bêtes et allège la charge du serveur.

6.1 Pourquoi valider côté client ? Confort, rapidité et économie de ressources

Imagine un élève de Terminale à Douala qui remplit un formulaire d'inscription en ligne pour le concours de l'ENS. Il oublie son numéro de téléphone, tape des lettres dans le champ « Âge »

et clique sur « Valider ». Sans validation côté client :

1. La requête HTTP part vers le serveur (latence réseau, coût data).
2. Le serveur (PHP, Node.js, Python) reçoit la requête, analyse les données, détecte les erreurs.
3. Le serveur renvoie une nouvelle page HTML avec les messages d'erreur (rechargement complet, perte de contexte, frustration).

Avec une validation côté client (JavaScript dans le navigateur) :

1. L'utilisateur clique sur « Valider ».
2. JavaScript intercepte l'événement `submit`, analyse les champs instantanément (0 ms réseau).
3. Les erreurs s'affichent dynamiquement sous les champs concernés, le formulaire n'est pas envoyé.
4. L'utilisateur corrige et renvoie : cette fois, c'est bon, le `submit` part vers le serveur.

DÉFINITION**Validation côté client (Client-side validation)**

Ensemble de vérifications effectuées par le navigateur web (via JavaScript) sur les données d'un formulaire avant l'envoi de la requête HTTP au serveur. Elle vise à améliorer l'expérience utilisateur (UX) en donnant un retour immédiat, sans aller-retour réseau.

PROPRIÉTÉ**Principe de non-confiance absolue — Règle d'or de la sécurité web**

Toute donnée provenant du navigateur (côté client) doit être considérée comme non fiable, potentiellement modifiée, falsifiée ou malveillante. La validation côté client est une commodité pour l'utilisateur honnête, jamais une protection pour le serveur. La validation côté serveur (PHP, Node.js, Java, etc.) est indispensable et systématique : elle est la seule barrière réelle contre les injections SQL, XSS, contournements de logique métier, etc. On parle de défense en profondeur : client (confort) + serveur (sécurité).

6.2 Les briques élémentaires : tester un champ en JavaScript

Avant d'écrire l'algorithme global, maîtrisons les tests unitaires sur un champ `<input>` récupéré via `document.getElementById` ou `querySelector`. Rappel : `input.value` renvoie **toujours une chaîne de caractères** (String), même pour `type="number"`.

MÉTHODE Tester les trois cas classiques au Bac

1. Champ non vide (texte, email, sélection)

```
// Supposons : const champ = document.getElementById('nom');
const valeur = champ.value.trim(); // Supprime espaces début/fin
if (valeur === '') {
    // Erreur : champ vide ou seulement des espaces
}
```

2. Format numérique et bornes (âge, quantité, prix)

```
// Supposons : const champ = document.getElementById('quantite');
const valBrute = champ.value.trim();
if (valBrute === '') { /* erreur vide */ }
else {
    // isNaN() renvoie true si ce n'est PAS un nombre
    if (isNaN(valBrute)) {
        // Erreur : "abc", "12a" ne sont pas des nombres
    } else {
        const nombre = Number(valBrute); // Conversion sûre
        if (nombre < 1 || nombre > 500) {
            // Erreur : hors bornes [1, 500]
        }
    }
}
```

3. Longueur exacte (numéro téléphone Cameroun : 9 chiffres)

```
// Supposons : const champ = document.getElementById('telephone');
const val = champ.value.trim();
// On vérifie : 9 caractères ET que des chiffres (via regex simple ou boucle)
// Version sans regex (programme de base) :
let estChiffres = true;
for (let i = 0; i < val.length; i++) {
    if (val[i] < '0' || val[i] > '9') { estChiffres = false; break; }
}
if (val.length !== 9 || !estChiffres) {
    // Erreur : format invalide (ex : 6XXXXXXXX ou 2XXXXXXXX)
}
```

ATTENTION Piège classique : comparer des chaînes au lieu de nombres !

En JavaScript, '9' > '10' renvoie true (comparaison lexicographique : '9' > '1').

```
const a = '9'; const b = '10';
console.log(a > b); // true <-- PIÈGE !
console.log(Number(a) > Number(b)); // false <-- CORRECT
```

Toujours convertir avec `Number()` ou `parseInt()` / `parseFloat()` avant toute comparaison numérique (<, >, <=, >=). L'opérateur `===` compare valeurs et types, donc '9' === 9 est false.

6.3 Signaler l'erreur à l'utilisateur : feedback visuel immédiat

Un message d'erreur dans `console.log()` ne sert à rien pour l'utilisateur. Il faut modifier le DOM : créer ou afficher un élément dédié (souvent `<span class="erreur">` ou `<p class="erreur">`) près du champ incriminé, et colorer la bordure du champ.

EXEMPLE Affichage dynamique d'une erreur sous un champ

HTML de base :

```
<div class="champ-groupe">
  <label for="telephone">Téléphone (9 chiffres) :</label>
  <input type="text" id="telephone" name="telephone">
  <span class="message-erreur" id="err-telephone" aria-live="polite"></span>
</div>
```

CSS minimal (à mettre dans `<style>` ou fichier `.css`) :

```
.message-erreur { color : #c0392b; font-size : 0.85em; margin-top : 0.25rem; display : block; min-
  height : 1.2em; }
.input-erreur { border-color : #c0392b !important; background-color : #fdf2f2; }
.input-valide { border-color : #27ae60 !important; }
</style>
```

JavaScript (fonction utilitaire réutilisable) :

```
function afficherErreur(idChamp, idMessage, texte) {
  const champ = document.getElementById(idChamp);
  const msg = document.getElementById(idMessage);
  champ.classList.add('input-erreur');
  champ.classList.remove('input-valide');
  msg.textContent = texte;
}

function effacerErreur(idChamp, idMessage) {
  const champ = document.getElementById(idChamp);
  const msg = document.getElementById(idMessage);
  champ.classList.remove('input-erreur');
  champ.classList.add('input-valide');
  msg.textContent = '';
}
```

6.4 Bloquer l'envoi : l'arme fatale `preventDefault()`

C'est le point clé : tant qu'il y a une erreur, le formulaire **ne doit pas partir**. On place un écouteur sur l'événement `submit` du `<form>` (pas sur le clic du bouton!), et on appelle `event.preventDefault()` si la validation échoue.

PROPRIÉTÉ Syntaxe de l'interception du submit

```
const form = document.getElementById('monFormulaire');
form.addEventListener('submit', function (event) {
    // 1. Réinitialiser l'affichage des erreurs précédentes
    // 2. Valider chaque champ, cumuler les erreurs
    // 3. Si au moins une erreur :
    if (ilYADesErreurs) {
        event.preventDefault(); // ANNULE l'envoi HTTP
        // Les messages d'erreur sont déjà affichés dans le DOM
    }
    // Si pas d'erreur : on ne fait RIEN, le submit continue normalement vers le serveur
});
```

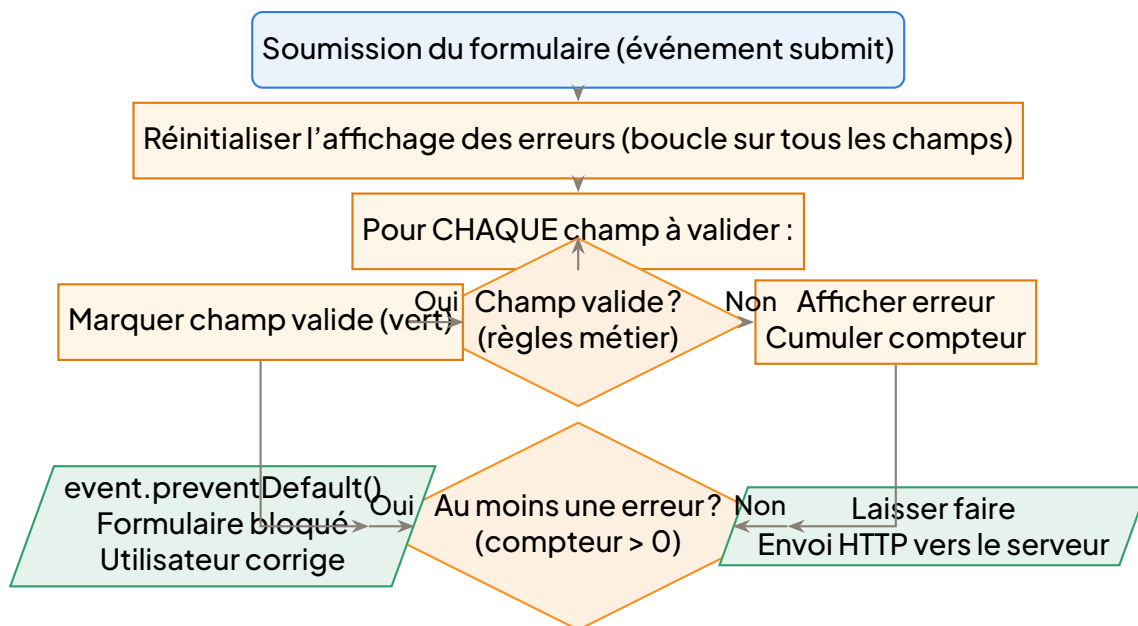
6.5 Stratégie globale : l'algorithme de validation complet

Ne valide pas champ par champ en envoyant le formulaire à chaque fois. La méthode professionnelle : **intercepter le submit unique, tout tester, tout afficher, décider.**

MÉTHODE

Valider un formulaire en 4 étapes (Méthode Bac)

1. **Intercepter** : `form.addEventListener('submit', handler).`
2. **Tester** : Pour chaque champ obligatoire, appliquer les règles (vide, type, bornes, longueur, format). Stocker les résultats (booléen) et les messages d'erreur.
3. **Afficher** : Pour chaque champ en erreur, appeler `afficherErreur()`. Pour les champs valides, appeler `effacerErreur()` (important si l'utilisateur corrige et re-soumet).
4. **Décider** : Si `nbErreurs > 0` alors `event.preventDefault()`; sinon laisser faire (envoi effectif).



Organigramme de l'algorithme de validation globale au submit

6.6 Exemple complet : Formulaire de la Coopérative Agricole « Espoir Vert »

Contexte : Une coopérative de producteurs de cacao à Ebolowa veut un formulaire de commande de sacs d'engrais. Règles métier :

- **Nom du producteur** : obligatoire, non vide.
- **Téléphone** : obligatoire, exactement 9 chiffres (format Cameroun : 6XXXXXXX ou 2XXXXXXX).
- **Quantité de sacs** : obligatoire, entier entre 1 et 500 inclus.

EXEMPLE Code source complet : formulaire + validation JS (fichier unique pour clarté)

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Commande Engrais – Coopérative Espoir Vert</title>
  <style>
    body { font-family : 'Segoe UI', sans-serif; max-width : 600px; margin : 2rem auto; padding : 0 1rem; }
    .champ-groupe { margin-bottom : 1.2rem; }
    label { display : block; margin-bottom : 0.3rem; font-weight : 600; color : #2c3e50; }
    input[type="text"], input[type="number"] {
      width : 100%; padding : 0.6rem; border : 2 solid #bdc3c7; border-radius : 4px;
      font-size : 1rem; box-sizing : border-box; transition : border-color 0.2s, background 0.2s;
    }
    .message-erreur { color : #c0392b; font-size : 0.85em; margin-top : 0.3rem; display : block; min-height : 1.2em; }
```

```

    .input-erreur { border-color : #c0392b !important; background-color : #fdf2f2; }
    .input-valide { border-color : #27ae60 !important; }
    button { background : #2980b9; color : white; border : none; padding : 0.8rem 2rem; font-size : 1.1
rem; border-radius : 4px; cursor : pointer; transition : background 0.2s; }
    button :hover { background : #1c5980; }
    .succes-global { background : #e8f8f5; border : 2px solid #27ae60; color : #1e8e49; padding : 1
rem; border-radius : 4px; margin-top : 1rem; display : none; }
</style>
</head>
<body>
  <h1>Commande d'engrais – Coopérative Espoir Vert</h1>
  <form id="formCommande" novalidate> <!-- novalidate désactive la validation HTML5 native pour
laisser faire notre JS -->
    <div class="champ-groupe">
      <label for="nom">Nom du producteur *</label>
      <input type="text" id="nom" name="nom" placeholder="Ex : MBARGA Jean">
      <span class="message-erreur" id="err-nom" aria-live="polite"></span>
    </div>

    <div class="champ-groupe">
      <label for="telephone">Téléphone (9 chiffres) *</label>
      <input type="text" id="telephone" name="telephone" placeholder="Ex : 677123456">
      <span class="message-erreur" id="err-telephone" aria-live="polite"></span>
    </div>

    <div class="champ-groupe">
      <label for="quantite">Quantité de sacs (1 à 500) *</label>
      <input type="number" id="quantite" name="quantite" min="1" max="500" placeholder="Ex :
50">
      <span class="message-erreur" id="err-quantite" aria-live="polite"></span>
    </div>

    <button type="submit">Valider la commande</button>
  </form>

  <div class="succes-global" id="msg-succes" role="alert"></div>

  <script>
    // --- FONCTIONS UTILITAIRES ---
    function afficherErreur(idChamp, idMessage, texte) {
      const champ = document.getElementById(idChamp);
      const msg = document.getElementById(idMessage);
      champ.classList.add('input-erreur');
      champ.classList.remove('input-valide');
      msg.textContent = texte;
    }

    function effacerErreur(idChamp, idMessage) {
      const champ = document.getElementById(idChamp);
      const msg = document.getElementById(idMessage);
      champ.classList.remove('input-erreur');
      champ.classList.add('input-valide');
      msg.textContent = '';
    }

    // --- VALIDATEURS SPÉCIFIQUES ---
    function validerNom() {

```

```

const val = document.getElementById('nom').value.trim();
if (val === '') {
    afficherErreur('nom', 'err-nom', 'Le nom est obligatoire.');
```

return false;

```

}
if (val.length < 2) {
    afficherErreur('nom', 'err-nom', 'Le nom est trop court (min 2 caractères.);
    return false;
}
effacerErreur('nom', 'err-nom');
return true;
}

function validerTelephone() {
    const val = document.getElementById('telephone').value.trim();
    if (val === '') {
        afficherErreur('telephone', 'err-telephone', 'Le téléphone est obligatoire.');
```

return false;

```

    }
    // Vérification : 9 chiffres exactement, commençant par 6 ou 2 (format CM)
    const regexTel = /^[26][0-9]{8}$/; // Complement Bac : Regex
    // Version sans regex (programme de base) :
    // let ok = (val.length === 9 && (val[0] === '6' || val[0] === '2'));
    // if (ok) { for(let i=0; i<9; i++) if (val[i] < '0' || val[i] > '9') ok = false; }

    if (!regexTel.test(val)) {
        afficherErreur('telephone', 'err-telephone', 'Format invalide : 9 chiffres commençant par 6
ou 2 (ex : 677123456).');
```

return false;

```

    }
    effacerErreur('telephone', 'err-telephone');
    return true;
}

function validerQuantite() {
    const valBrute = document.getElementById('quantite').value.trim();
    if (valBrute === '') {
        afficherErreur('quantite', 'err-quantite', 'La quantité est obligatoire.');
```

return false;

```

    }
    if (isNaN(valBrute)) {
        afficherErreur('quantite', 'err-quantite', 'Veuillez entrer un nombre valide.');
```

return false;

```

    }
    const qte = Number(valBrute);
    if (!Number.isInteger(qte) || qte < 1 || qte > 500) {
        afficherErreur('quantite', 'err-quantite', 'Quantité invalide : entier entre 1 et 500 sacs.');
```

return false;

```

    }
    effacerErreur('quantite', 'err-quantite');
    return true;
}

// --- GESTIONNAIRE PRINCIPAL (SOUMISSION) ---
document.getElementById('formCommande').addEventListener('submit', function (event) {
    // 1. Réinitialiser le message de succès global
    document.getElementById('msg-succes').style.display = 'none';

```

```

document.getElementById('msg-succes').textContent = '';

// 2. Lancer toutes les validations (on ne s'arrête pas à la première erreur)
const okNom = validerNom();
const okTel = validerTelephone();
const okQte = validerQuantite();

// 3. Décision globale
if (!(okNom && okTel && okQte)){
    event.preventDefault(); // BLOQUER L'ENVOI
    // Les erreurs sont déjà affichées par les fonctions ci-dessus
} else {
    // Succès côté client : on laisse le formulaire partir vers le serveur
    // Ici, pour la démo, on bloque quand même et on affiche un message local
    event.preventDefault(); // À RETIRER en production réelle !
    document.getElementById('msg-succes').textContent = 'Formulaire valide ! Envoi au
serveur simulé...';
    document.getElementById('msg-succes').style.display = 'block';
    // En vrai : this.submit() ou ne rien faire (le submit continue)
}
});
</script>
</body>
</html>

```

Commande d'engrais – Coopérative Espoir Vert

Nom du producteur *

MBARGA

Téléphone (9 chiffres)

67712345

Format invalide : 9 chiffres commençant par 6 ou 2

Quantité de sacs (1 à 50)

50

Valider la commande

Formulaire valide ! Envoi au serveur simulé... (démo)

Classe CSS: .input-erreur (bordure rouge, fond rose)
 Span .message-erreur: display:block, color:#c0392b
 event.preventDefault() appelé car compteur erreurs > 0

Capture simulée : formulaire avec erreur de validation sur le téléphone (8 chiffres au lieu de 9). Le bouton a été cliqué, le submit intercepté, l'erreur affichée, l'envoi bloqué.

6.7 Complément utile au Bac : Les expressions régulières (Regex)

Le programme mentionne la validation de format. Si `isNaN` et `length` suffisent pour des cas simples, les **expressions régulières** (RegExp) sont l'outil standard pour décrire des motifs complexes (email, téléphone, code postal, mot de passe fort). C'est un complément très valorisant à l'oral ou à l'écrit.

PROPRIÉTÉ Syntaxe de base des Regex en JavaScript

Un objet `RegExp` s'écrit entre slashes : `/motif/flags`. La méthode `test(chaine)` renvoie `true/false`.

Motif	Signification	Exemple match
<code>^[0-9]{9}\$</code>	Exactement 9 chiffres (début ^ fin \$)	677123456
<code>^[26][0-9]{8}\$</code>	9 chiffres commençant par 2 ou 6	233456789
<code>^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+[a-zA-Z]{2,}\$</code>	Email simplifié	user@mtn.cm
<code>^(?=.*[a-z])(?=.*[A-Z])(?=.*\d){8,}\$</code>	Mot de passe fort (1 maj, 1 min, 1 chif, 8+)	Passw0rd

Utilisation : `const regex = /^[26][0-9]{8}$/; if (!regex.test(valeur)) { erreur }`

ATTENTION Regex : pièges fréquents

- Oublier les ancrs `^` (début) et `$` (fin) : `/[0-9]9/` valide "abc123456789xyz" car il trouve 9 chiffres quelque part.
- Échapper le point `\.` pour matcher un vrai point (sinon `.` = n'importe quel caractère).
- En JavaScript, le backslash dans une chaîne littérale "... " doit être doublé : `new RegExp("^ [0-9]9$")` (mais la notation littérale `/.../` est préférée).

6.8 Le savais-tu ? : L'histoire de la défense en profondeur

LESAVAIS-TU ? Pourquoi ton prof insiste sur la validation serveur ?

Dans les années 2000, de nombreux sites camerounais (et d'ailleurs) ne validaient que côté client. Les attaquants utilisaient des outils (comme `curl`, `Postman`, ou la console navigateur

F12) pour supprimer le JavaScript de validation et envoyer des charges malveillantes directement au serveur :

- Injection SQL : ' OR '1'='1 dans un champ login → contournement authentification, vol de base de données.
- XSS (Cross-Site Scripting) : <script>volerCookies()</script> dans un commentaire → vol de session admin.
- Contournement métier : Prix négatif, quantité 999999, ID utilisateur modifié.

Aujourd'hui, l'ANTIC (Agence Nationale des Technologies de l'Information et de la Communication) impose aux administrations et opérateurs (MTN, Orange, CAMTEL) des audits de sécurité incluant la validation serveur systématique. La validation client que tu apprends ici est la première ligne (confort), la validation serveur est le dernier rempart (survie). C'est ça, la défense en profondeur.

6.9 Exercice résolu : Validation d'un code promo

EXERCICE RÉSOLU 6 Valider un code promotionnel

Énoncé : Un site e-commerce camerounais (style Jumia CM ou Kaymu) propose un champ « Code Promo ». Le code doit respecter le format : PROMO-XXXX où XXXX sont exactement 4 chiffres (ex : PROMO-1234). Écrire la fonction de validation JavaScript et l'intégrer dans un gestionnaire submit.

Solution détaillée :

1. Analyse du format :

- Préfixe fixe : "PROMO-" (6 caractères, majuscules, tiret).
- Suffixe : 4 chiffres [0-9]{4}.
- Longueur totale : 10 caractères.
- Sensible à la casse (majuscules imposées).

2. Choix de la méthode : Regex (efficace) ou parsing manuel (programme de base).

Version Regex (recommandée au Bac pour la clarté) :

```
function validerCodePromo(valeur) {  
  const v = valeur.trim().toUpperCase(); // Tolérance : on accepte minuscules saisies  
  const regex = /^PROMO-[0-9]{4}$/;  
  return regex.test(v);  
}
```

Version sans Regex (algorithme pur, programme de base) :

```
function validerCodePromo(valeur) {  
  const v = valeur.trim().toUpperCase();  
  if (v.length !== 10) return false;  
  if (v.substring(0, 6) !== 'PROMO-') return false;  
}
```

```
const suffixe = v.substring(6); // les 4 derniers
for (let i = 0; i < 4; i++) {
  if (suffixe[i] < '0' || suffixe[i] > '9') return false;
}
return true;
}
```

3. Intégration dans le formulaire :

```
<form id="formPromo" novalidate>
  <label for="code">Code Promo :</label>
  <input type="text" id="code" name="code" placeholder="PROMO-1234">
  <span class="message-erreur" id="err-code"></span>
  <button type="submit">Appliquer</button>
</form>

<script>
document.getElementById('formPromo').addEventListener('submit', function(e) {
  const champ = document.getElementById('code');
  const val = champ.value;
  if (!validerCodePromo(val)) {
    e.preventDefault();
    afficherErreur('code', 'err-code', 'Format attendu : PROMO-XXXX (4 chiffres).');
  } else {
    effacerErreur('code', 'err-code');
    // Envoi autorisé
  }
});
</script>
```

6.10 Exercice d'entraînement : Formulaire d'inscription à un hackathon

EXERCICE 5 Inscription Hackathon « Silicon Mountain »

Réalise la page HTML/JS complète pour l'inscription à un hackathon à Buea. Le formulaire contient :

1. Nom d'équipe : obligatoire, 3 à 30 caractères.
2. Email chef d'équipe : obligatoire, format email valide (utilise une regex simple).
3. Nombre de membres : obligatoire, entier entre 2 et 5 inclus.
4. Thème choisi : <select> obligatoire (value non vide) : FinTech, AgriTech, HealthTech, EdTech, Autre.

Contraintes techniques :

- Utilise `novalidate` sur le `<form>`.
- Une seule fonction `validerTout()` appelée au `submit`.
- Affiche les erreurs sous chaque champ (`span .message-erreur`).

- Colore les champs invalides (bordure rouge) et valides (bordure verte).
- Bloque l'envoi si erreur (`preventDefault()`).
- Si tout est valide : affiche une alerte "Inscription envoyée au serveur !" et laisse le formulaire partir (ne pas `preventDefault()`).

Indice : Pour le `<select>`, teste `select.value === ''` (si la première option a `value=""`).

CORRIGÉ Exercice Inscription Hackathon

```
<!DOCTYPE html>
<html lang="fr">
<head>
<meta charset="UTF-8">
<title>Inscription Hackathon Silicon Mountain</title>
<style>
  body { font-family : system-ui, sans-serif; max-width : 500px; margin : 2rem auto; padding : 0 1rem; }
  .groupe { margin-bottom : 1rem; }
  label { display : block; margin-bottom : .3rem; font-weight : 600; }
  input, select { width : 100%; padding : .5rem; border : 2px solid #ccc; border-radius : 4px; box-sizing : border-box; }
  .err { color : #c00; font-size : .85em; margin-top : .2rem; display : block; min-height : 1.2em; }
  .inp-err { border-color : #c00 !important; background : #fee; }
  .inp-ok { border-color : #090 !important; }
  button { width : 100%; padding : .7rem; background : #27ae60; color : white; border : none; border-radius : 4px; font-size : 1.1rem; cursor : pointer; }
  button :hover { background : #219150; }
</style>
</head>
<body>
<h1>Inscription Hackathon Silicon Mountain</h1>
<form id="formHack" novalidate>
  <div class="groupe">
    <label for="equipe">Nom d'équipe (3-30 car.) *</label>
    <input type="text" id="equipe" name="equipe" maxlength="30">
    <span class="err" id="err-equipe"></span>
  </div>
  <div class="groupe">
    <label for="email">Email chef d'équipe *</label>
    <input type="email" id="email" name="email">
    <span class="err" id="err-email"></span>
  </div>
  <div class="groupe">
    <label for="membres">Nombre de membres (2-5) *</label>
    <input type="number" id="membres" name="membres" min="2" max="5">
    <span class="err" id="err-membres"></span>
  </div>
  <div class="groupe">
    <label for="theme">Thème *</label>
    <select id="theme" name="theme">
      <option value="">-- Choisir --</option>
      <option value="FinTech">FinTech</option>
    </select>
  </div>
  <button type="submit">Inscrire</button>
</form>
</body>
</html>
```

```
<option value="AgriTech">AgriTech</option>
<option value="HealthTech">HealthTech</option>
<option value="EdTech">EdTech</option>
<option value="Autre">Autre</option>
</select>
<span class="err" id="err-theme"></span>
</div>
<button type="submit">S'inscrire</button>
</form>

<script>
// Utils
function showErr(id, msg) {
  const el = document.getElementById(id);
  const err = document.getElementById('err-' + id);
  el.classList.add('inp-err'); el.classList.remove('inp-ok');
  err.textContent = msg;
}
function clearErr(id) {
  const el = document.getElementById(id);
  const err = document.getElementById('err-' + id);
  el.classList.remove('inp-err'); el.classList.add('inp-ok');
  err.textContent = '';
}

// Validateurs
function checkEquipe() {
  const v = document.getElementById('equipe').value.trim();
  if (v.length < 3 || v.length > 30) { showErr('equipe', 'Entre 3 et 30 caractères.');
```

```
  return false; }
  clearErr('equipe'); return true;
}
function checkEmail() {
  const v = document.getElementById('email').value.trim();
  // Regex email simple mais efficace pour le Bac
  if (!/^[^\\s@]+@[^\\s@]+\\. [^\\s@]+$/ .test(v)) { showErr('email', 'Format email invalide.');
```

```
  return false; }
  clearErr('email'); return true;
}
function checkMembres() {
  const v = document.getElementById('membres').value.trim();
  if (v === '' || isNaN(v)) { showErr('membres', 'Nombre requis.');
```

```
  return false; }
  const n = Number(v);
  if (!Number.isInteger(n) || n < 2 || n > 5) { showErr('membres', 'Entier entre 2 et 5.');
```

```
  return false; }
  clearErr('membres'); return true;
}
function checkTheme() {
  const v = document.getElementById('theme').value;
  if (v === '') { showErr('theme', 'Veuillez choisir un thème.');
```

```
  return false; }
  clearErr('theme'); return true;
}

// Gestionnaire principal
document.getElementById('formHack').addEventListener('submit', function(e) {
  const ok = checkEquipe() & checkEmail() & checkMembres() & checkTheme(); // & (bitwise) force éval
  de tous
  if (!ok) {
    e.preventDefault(); // Bloquer
  } else {
```

```
        alert('Inscription envoyée au serveur !'); // Demo
        // Pas de preventDefault -> envoi réel
    }
});
</script>
</body>
</html>
```

À RETENIR**Check-list de validation côté client pour le Bac**

- **Structure** : `<form novalidate> + addEventListener('submit', ...)`.
- **Récupération** : `input.value.trim()` (toujours chaîne).
- **Tests** : `=== ''` (vide), `isNaN()` / `Number()` (numérique), `.length` / `regex` (format/longueur), `select.value === ''` (liste).
- **Comparaison numérique** : Jamais sur des chaînes (`'9' > '10'`). Toujours `Number(val)`.
- **Feedback** : `span.erreur` dynamique + classes CSS `.input-erreur` / `.input-valide`.
- **Blocage** : `event.preventDefault()` uniquement si erreur(s).
- **Sécurité** : // RAPPEL : La validation serveur est OBLIGATOIRE.

7 ACTIVITÉ D'INTÉGRATION

8 RAPPELS DE PROGRAMMATION WEB : HTML ET JAVASCRIPT

8.1 Structure d'une page HTML5

DÉFINITION Page HTML5 — Squelette minimal

Une page web valide HTML5 commence par la déclaration de type `<!DOCTYPE html>`, suivie de l'élément racine `<html lang="fr">` (langue du contenu). Elle contient deux enfants directs :

- `<head>` : métadonnées invisibles (encodage, titre, viewport, liens CSS/JS).
- `<body>` : contenu visible (texte, images, formulaires, tableaux, scripts).

L'encodage UTF-8 (`<meta charset="UTF-8">`) est obligatoire pour afficher correctement les accents, le symbole FCFA (ou FCFA) et les caractères spéciaux camerounais.

PROPRIÉTÉ Balises sémantiques HTML5 principales

HTML5 introduit des balises qui décrivent le sens du contenu (accessibilité, SEO, maintenance) plutôt que son apparence :

Balise	Rôle sémantique
<code><header></code>	En-tête de page ou de section (logo, navigation principale).
<code><nav></code>	Zone de navigation majeure (menu principal).
<code><main></code>	Contenu principal unique de la page (exclut header/footer/-nav/aside).
<code><section></code>	Regroupement thématique avec un titre (<code><h1></code> - <code><h6></code>).
<code><article></code>	Contenu autonome, syndiquable (article de blog, fiche produit).
<code><aside></code>	Contenu annexe (barre latérale, publicité, définition).
<code><footer></code>	Pied de page (mentions légales, contact, réseaux sociaux).
<code><figure>/<figcaption></code>	Illustration (image, code, schéma) avec sa légende.

Utilise ces balises au lieu de `<div>` génériques dès que le sens correspond.

EXEMPLE Squelette HTML5 de base pour un projet TI

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mon Application – Coopérative Espoir Bafoussam</title>
  <link rel="stylesheet" href="style.css">
  <!-- Script chargé en fin de body ou avec defer (voir §4) -->
</head>
<body>
  <header>
    <h1>Coopérative Agricole « Espoir de Bafoussam »</h1>
  </header>
  <main>
    <!-- Contenu principal : formulaires, tableaux, etc. -->
  </main>
  <footer>
    <p>&copy; 2025 – AgriTech Cameroon – Tous droits réservés</p>
  </footer>
  <script src="script.js" defer></script>
</body>
</html>
```

ATTENTION Pièges classiques : Doctype, Charset, Lang

- Oublier `<!DOCTYPE html>` : le navigateur passe en mode Quirks (rendu imprévisible, box-model ancien).
- Placer `<meta charset>` après le `<title>` : le titre peut contenir des accents mal interprétés. Toujours en premier dans `<head>`.
- Omettre `lang="fr"` : nuit à l'accessibilité (lecteurs d'écran) et au référencement (SEO local Cameroun).
- Encodage du fichier source : ton éditeur (VS Code, Notepad++) doit sauver en UTF-8 sans BOM.

8.2 Les formulaires HTML

DÉFINITION Formulaire web — Rôle et attributs

Un formulaire (`<form>`) regroupe des contrôles interactifs pour collecter des données utilisateur. Ses attributs clés :

- `action` : URL de traitement (ici # ou absent car traitement 100% client/JS).

- `method` : GET (données en URL, idempotent) ou POST (corps HTTP, modification serveur). En SPA côté client, on n'envoie pas le formulaire : on intercepte `submit`.
- `novalidate` : Indispensable pour désactiver la validation native du navigateur (bulles UI non personnalisables) et imposer notre validation JavaScript centralisée.

PROPRIÉTÉ**Champs de saisie (<input>, <select>, <textarea>) — Types et attributs courants**

Type / Balise	Attributs clés
<input type="text">	required, minlength, maxlength, placeholder, autocomplete
<input type="email">	required, pattern
<input type="tel">	pattern, placeholder
<input type="number">	min, max, step, required
<input type="date">	min, max

Usage / Exemple

ca-me-rou-nais

Nom, pré-nom, ville (ex: autocomplete)

Validation native basique (ex: contact@ag)

Téléphone CM: pattern=" [(9 chiffres, 6 ou 2).

Quantité sacs: min="1" max="500" step="1".

Date de livraison (ca-len-drier natif mobile).

PROPRIÉTÉ Accessibilité obligatoire : `<label>` et `for / id`

Chaque champ doit être associé à une étiquette explicite :

```
<label for="telephone">Téléphone (9 chiffres, 6 ou 2) <span aria-hidden="true">*</span></label>
<input type="tel" id="telephone" name="telephone" required pattern="[26][0-9]{8}" placeholder="Ex :
690123456">
```

- `for="id_du_champ"` lie le label au champ (clic sur label = focus champ).
- `aria-hidden="true"` sur l'astérisque * évite de l'annoncer deux fois aux lecteurs d'écran.
- Regroupe les champs liés dans `<fieldset>` avec `<legend>` (ex : « Nouvel Apport »).

MÉTHODE Construire un formulaire robuste et accessible

1. Balise `<form id="monForm" novalidate>`.
2. `<fieldset>` + `<legend>` pour regrouper logiquement.
3. Pour chaque champ : `<label for="id">` + `<input id="id" name="nom" ...>`.
4. Ajouter `required` côté HTML (filet de sécurité) + attributs de contrainte (`min`, `max`, `pattern`, `minlength`).
5. Prévoir un `<div id="message" aria-live="polite">` pour les retours JS (erreurs/succès).
6. Bouton `<button type="submit">` (par défaut `type="submit"`, mais explicite est mieux).

LESAVAIS-TU ? Mobile Money au Cameroun : MTN MoMo & Orange Money

Les numéros de téléphone sont la clé d'identification pour les paiements Mobile Money (MTN MoMo, Orange Money, Moov Money). Le format national à 9 chiffres (sans +237) commence par :

- 6 : Mobile (MTN : 67x, 68x, 65x, 69x | Orange : 69x, 68x, 67x | Nexttel : 66x, 65x).
- 2 : Fixe CAMTEL / CDMA (ex : 233, 222, 242...).

Les préfixes 3, 5, 7, 8, 9 ne sont pas attribués au grand public. D'où la regex stricte `/^[26][0-9]{8}$/`.

8.3 Les tableaux HTML

DÉFINITION Tableau de données — Structure sémantique

Un tableau (<table>) présente des données tabulaires (grille lignes/colonnes), jamais pour la mise en page (CSS Grid/Flexbox pour ça). Structure complète :

```
<table>
  <caption>Registre des apports – Juin 2025</caption>
  <thead>
    <tr>
      <th scope="col">N°</th>
      <th scope="col">Nom</th>
      <th scope="col">Montant (FCFA)</th>
    </tr>
  </thead>
  <tbody>
    <tr><td>1</td><td>Marie Nono</td><td style="text-align:right;">1 250 000</td></tr>
  </tbody>
  <tfoot>
    <tr>
      <td colspan="2">TOTAL</td>
      <td style="text-align:right;">1 250 000 FCFA</td>
    </tr>
  </tfoot>
</table>
```

PROPRIÉTÉ Attributs essentiels pour l'accessibilité et la structure

- <caption> : Titre du tableau (lu par lecteurs d'écran, placé juste après <table>).
- <th scope="col"> / scope="row" : Définit si l'en-tête s'applique à une colonne ou une ligne. Obligatoire pour accessibilité.
- colspan="n" / rowspan="n" : Fusion de cellules (ex : pied de tableau « Total » sur plusieurs colonnes).
- <thead>, <tbody>, <tfoot> : Permettent au navigateur de scroller le corps indépendamment (impression multi-pages, CSS tbody { display:block; max-height:300px; overflow:auto;}).

À RETENIR Bonnes pratiques CSS pour tableaux de données

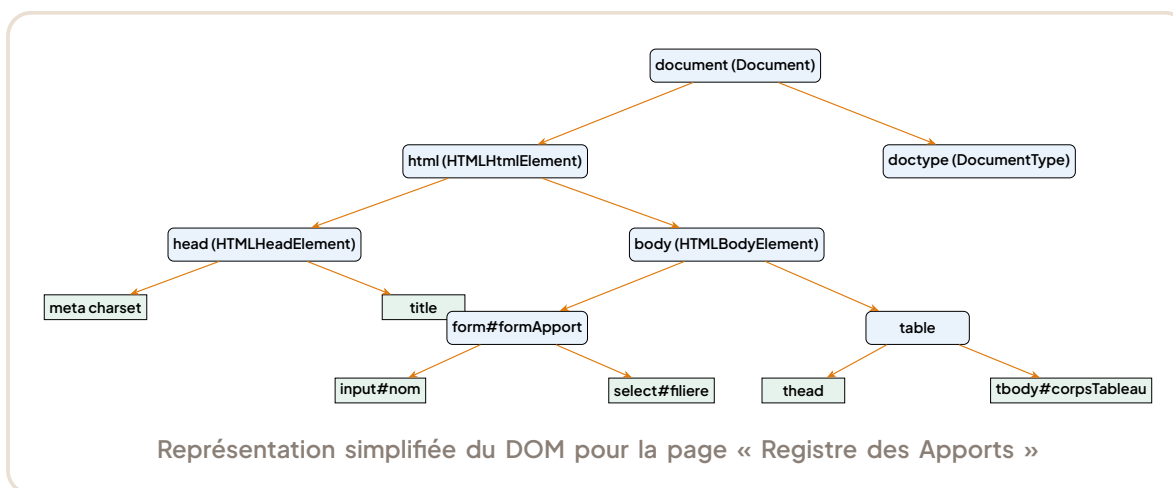
- table { border-collapse: collapse; width: 100%; } : bordures fusionnées, largeur fluide.
- th, td { padding: 0.5rem 1rem; border: 1px solid #ddd; } : espace lisible.
- th { background: #2c3e50; color: white; text-align: left; } : en-tête visible.

- **Alignement des nombres à droite** : `td:nth-child(n) { text-align: right; font-variant-numeric: tabular-nums; }` (chiffres alignés sur la largeur).
- `tbody tr:hover { background: #f5f5f5; }` : feedback visuel ligne survolée.

8.4 JavaScript et le DOM (Document Object Model)

DÉFINITION Le DOM — Arbre d'objets du navigateur

Quand le navigateur charge une page HTML, il la parse et construit une arborescence d'objets JavaScript : le **DOM**. Chaque balise devient un `Element`, chaque texte un `TextNode`, chaque attribut un `Attr`. L'objet racine est `document`.



PROPRIÉTÉ Sélection d'éléments — API modernes (ES6+)

Méthode	Usage / Retour
<code>document.getElementById('id')</code>	Unique, rapide. Retourne <code>Element</code> ou <code>null</code> . Préféré pour un ID.
<code>document.querySelector('selector')</code>	Premier élément correspondant au sélecteur CSS (<code>#id</code> , <code>.class</code> , <code>tag</code> , <code>[attr]</code>). Retourne <code>Element</code> ou <code>null</code> .
<code>document.querySelectorAll('selector')</code>	<code>NodeList</code> statique (pas live) de tous les éléments correspondants. Itérable avec <code>forEach</code> .
<code>element.querySelector(...)</code>	Recherche descendants de <code>element</code> (encapsulation).

Évite `getElementsByName` / `getElementsByClassName` (`HTMLCollection` live, source de bugs).

PROPRIÉTÉ
Manipulation du DOM — Création, insertion, contenu

Opération	Syntaxe / Exemple
Créer un élément	<code>const tr = document.createElement('tr');</code>
Créer nœud texte	<code>const txt = document.createTextNode('Marie');</code> (ou <code>td.textContent = 'Marie';</code>)
Insérer enfant (fin)	<code>parent.appendChild(enfant);</code>
Insérer enfant (début)	<code>parent.prepend(enfant);</code>
Insérer avant un nœud	<code>parent.insertBefore(nouveau, reference);</code>
Remplacer	<code>parent.replaceChild(nouveau, ancien);</code>
Supprimer	<code>enfant.remove();</code> (moderne) ou <code>parent.removeChild(enfant);</code>
Contenu texte (sécurisé)	<code>element.textContent = 'Valeur <script>';</code> // Affiche littéralement
Contenu HTML (DANGEREUX)	<code>element.innerHTML = 'Gras';</code> // Interdit avec données utilisateur (XSS)
Attributs	<code>el.setAttribute('name', 'val');</code> <code>el.getAttribute('name');</code> , <code>el.id</code> , <code>el.value</code> , <code>el.classList.add('classe')</code>

ATTENTION Faille XSS (Cross-Site Scripting) — `innerHTML` vs `textContent`

Si tu insères une saisie utilisateur via `innerHTML`, le navigateur exécute le code HTML/JS qu'elle contient.

```
// DONNÉE MALVEILLANTE : nom = "<img src=x onerror=alert('Hack!')>"
// DANGEREUX :
div.innerHTML = nom; // Exécute le script ! Vol de session, défiguration.
// SÉCURISÉ :
div.textContent = nom; // Affiche littéralement : <img src=x onerror=alert('Hack!')>
```

Règle d'or TI : Jamais `innerHTML` (ni `outerHTML`, `insertAdjacentHTML`) avec une donnée non sanitisée. Toujours `textContent` pour du texte, `value` pour les champs, `setAttribute` pour les attributs.

8.5 Gestion des événements en JavaScript

DÉFINITION Modèle événementiel — Écouteurs et propagation

Une interaction utilisateur (clic, saisie, soumission, chargement) génère un objet `Event` qui propage dans le DOM :

1. Phase de capture : de `window` vers la cible (rarement utilisée).
2. Phase cible : sur l'élément déclencheur.
3. Phase de remontée (bubbling) : de la cible vers `window` (par défaut pour `addEventListener`).

On enregistre un gestionnaire (handler) avec `addEventListener(type, handler, options)`.

PROPRIÉTÉ Syntaxe `addEventListener` et objet `Event`

```
// Syntaxe complète
element.addEventListener('type', function(event) {
  // event.target : élément qui a déclenché l'événement
  // event.currentTarget : élément sur lequel l'écouteur est attaché (=== this)
  // event.preventDefault() : annule l'action par défaut du navigateur
  // event.stopPropagation() : arrête la remontée (bubbling)
}, { once: false, passive: false, capture: false }); // options par défaut
```

PROPRIÉTÉ Événements formulaires essentiels

Type	Déclencheur / Usage typique
submit	Sur <code><form></code> : validation globale avant envoi. TOUJOURS <code>e.preventDefault()</code> en SPA.
input	Sur <code><input></code> , <code><select></code> , <code><textarea></code> : à chaque modification (clavier, coller, voix). Idéal pour validation temps réel.
change	Sur <code><select></code> , <code><input type="checkbox/radio/date/file"></code> : quand la valeur est validée (perte focus ou choix).
focus / blur	Entrée / sortie du champ. Style, aide contextuelle, validation au blur.
keydown / keyup	Touche pressée / relâchée. Filtrer saisie (ex : n'autoriser que chiffres).

MÉTHODE Intercepter la soumission d'un formulaire (SPA)

1. Sélectionner le formulaire : `const form = document.getElementById('monForm');`
2. Ajouter l'écouteur sur submit : `form.addEventListener('submit', handler);`
3. Dans `handler(e)` :
 - (a) `e.preventDefault();` // Bloque rechargement / envoi HTTP.
 - (b) Récupérer valeurs : `const val = champ.value.trim();`
 - (c) Valider séquentiellement (voir §6).
 - (d) Si OK : Mettre à jour DOM (créer éléments, `appendChild`), maj état (totaux), feedback utilisateur, `form.reset()`, `focus()` premier champ.

LESAVAIS-TU ? Single Page Application (SPA) légère — Côté client uniquement

Notre application « Registre des Apports » est une SPA légère : une seule page HTML (`registre.html`), pas de rechargement, tout le traitement (validation, calcul, affichage) se fait en JavaScript dans le navigateur. Les données vivent en mémoire (variables JS `totalSacs`, `totalMontant`). Limite : rafraîchir la page (F5) perd tout. Évolution naturelle : persistance via `localStorage` (côté client) ou API REST + Base de données (côté serveur — PHP/Node.js + MySQL/PostgreSQL).

8.6 Validation de la saisie côté client

DÉFINITION Validation client vs serveur — Rôles complémentaires

- Côté client (JavaScript) : Confort utilisateur (UX). Retour instantané, pas de latence réseau, économie bande passante. Pas de sécurité : l'utilisateur peut désactiver JS, modifier le code (DevTools), envoyer des requêtes `curl`/Postman.
- Côté serveur (PHP, Node, Python...) : Sécurité et intégrité. Unique rempart contre injections SQL, XSS stocké, corruption données. Les deux sont obligatoires en production.

PROPRIÉTÉ Algorithme de validation séquentielle (court-circuit)

On teste les règles une par une, dans l'ordre logique. À la première erreur : on affiche, on focus le champ, on `return` (sort de la fonction). Pas de `else if` imbriqués inutiles.

```
function valider(nom, filiere, qteStr, tel) {  
  // 1. Nom  
  if (nom.length < 2) return { ok: false, msg: 'Nom invalide (min 2 car.)', champ: 'nom' };  
  // 2. Filière  
  if (! filiere ) return { ok: false, msg: 'Choisir une filière', champ: 'filiere' };  
  // 3. Quantité : entier 1..500  
  const qte = parseInt(qteStr, 10);  
  if (isNaN(qte) || qte < 1 || qte > 500 || !Number.isInteger(qte))  
    return { ok: false, msg: 'Quantité invalide (1 à 500 sacs)', champ: 'quantite' };  
  // 4. Téléphone Cameroun  
  if (!/^ [26][0-9]{8}$/.test(tel))  
    return { ok: false, msg: 'Tél invalide (9 ch., 6 ou 2)', champ: 'telephone' };  
  // Succès  
  return { ok: true, qte: qte };  
}
```

PROPRIÉTÉ Regex téléphone Cameroun — Détail

`/^ [26][0-9]{8}$` / décomposé :

- `^` : Début de chaîne (ancrage).
- `[26]` : Classe de caractères : 2 (fixe CAMTEL) OU 6 (mobile MTN/Orange/Nexttel).
- `[0-9]{8}` : Exactement 8 chiffres (0 à 9).
- `$` : Fin de chaîne (ancrage).

Rejette : +237... , espaces, tirets, 8 ou 10 chiffres, début par 3/5/7/8/9.

PROPRIÉTÉ Vérification nombre entier — Piège `parseInt`

`parseInt("10.5", 10)` renvoie 10 (tronque). `parseInt("abc")` renvoie NaN. Validation stricte :

```
const val = parseInt(chaine, 10); // Base 10 EXPLICITE (évite octal ancien)
if (isNaN(val) || !Number.isInteger(val) || val < 1 || val > 500) { /* erreur */ }
```

`Number.isInteger(val)` renvoie false pour NaN, Infinity, 10.5, "10". C'est le garde-fou fiable.

PROPRIÉTÉ Formatage monétaire FCFA — `toLocaleString`

```
const montant = 1250000;
const formate = montant.toLocaleString('fr-FR') + ' FCFA'; // "1 250 000 FCFA"
// 'fr-FR' utilise l'espace insécable (U+202F) comme séparateur de milliers.
// Optionnel : { style: 'currency', currency: 'XAF' } -> "1 250 000 FCFA" (symbole normalisé ISO 4217)
```

ATTENTION Top 5 des bugs classiques en validation JS (Bac / Projet)

1. Oublier `e.preventDefault()` : Rechargement page → variables JS remises à 0, tableau vidé, utilisateur perdu.
2. Variable d'état locale : `let total = 0` dans le handler → remis à 0 à chaque soumission. Déclarer hors handler (portée module/IIFE).
3. `innerHTML` avec données utilisateur : Faille XSS critique. TOUJOURS `textContent`.
4. `parseInt` sans `isNaN` / `Number.isInteger` : `parseInt("abc") = NaN`, `NaN > 500` est false, `NaN < 1` est false → passe la validation !
5. Ne pas nettoyer le message précédent : L'erreur persiste après une saisie correcte. `messageDiv.textContent = ''`; `messageDiv.className = ''`; au début du handler.

8.7 Activité d'intégration : Registre des apports de la Coopérative « Espoir de Bafous-sam »

EXPÉRIENCE TP Complet — Application SPA « Registre des Apports »

Objectif : Mobiliser l'ensemble des savoirs (HTML sémantique, CSS, DOM, Événements, Validation JS) dans un projet complet, réaliste, testé et revu par les pairs.

Situation : La Coopérative Agricole « Espoir de Bafoussam » (Ouest, Cameroun) numérise son registre des apports quotidiens. Prix fixe : 25 000 FCFA / sac (50 kg). Paiement Mobile Money (MTN MoMo / Orange Money). Tu es développeur junior chez AgriTech Cameroun. Tu réalises la page « Registre des apports » (SPA légère, intranet localhost via WampServer/XAMPP).

Spécifications fonctionnelles :

1. Formulaire (`<form id="formApport" novalidate>`):

- `<input type="text" id="nom" required minlength="2">` Nom membre (obligatoire, ≥ 2 car.).
- `<select id="filiere" required>` Options : Cacao, Café, Maïs, Banane plantain. Défaut : `<option value="" disabled selected>-- Choisir --</option>`.
- `<input type="number" id="quantite" required min="1" max="500" step="1">` Sacs (entier [1, 500]).
- `<input type="tel" id="telephone" required pattern="[26][0-9]{8}">` 9 chiffres, commence par 6 ou 2.
- `<button type="submit">Enregistrer</button>`.

2. Zone message : `<div id="message" aria-live="polite">` (vide au départ). Erreur = rouge (`.erreur`), Succès = vert (`.succes`).

3. Tableau récapitulatif (`<table>`):

- `<thead>` : N°, Nom, Filière, Quantité (sacs), Téléphone, Montant (FCFA).
- `<tbody id="corpsTableau">` : Vide initialement. Lignes ajoutées dynamiquement.
- `<tfoot>` : 2 lignes totaux — `<td id="totalSacs">`, `<td id="totalMontant">`.

Règles de validation JS (côté client) :

- Nom < 2 car. → « Nom invalide (min 2 caractères) ».
- Filière vide → « Veuillez sélectionner une filière ».
- Quantité vide, < 1 , > 500 , non entier → « Quantité invalide (1 à 500 sacs) ».
- Téléphone ne match pas `/^[26][0-9]{8}$/` → « Téléphone invalide (9 chiffres, commence par 6 ou 2) ».
- Erreur : Afficher dans `#message` (classe `.erreur`), pas d'ajout ligne, pas de maj totaux, `focus()` champ incriminé.
- Succès : « Apport enregistré », ajout ligne `<tr>` (6 `<td>` via `createElement/appendChild/textContent`), maj totaux (`toLocaleString('fr-FR')`), `form.reset()`, `focus()` sur `#nom`.

Phases de réalisation (durée indicative 1h10) :

1. **HTML (20 min) :** registre.html complet (doctype, head, body, form fieldset/legend, label/for, table thead/tbody/tfoot, div message). Lier style.css (head) et script.js (fin body avec defer).
2. **CSS (15 min) :** style.css. Conteneur max-width: 900px; margin: auto. Police system-ui. Tableau border-collapse, alignement nombres à droite. Classes .erreur (fond #fdecea, texte #c0392b) et .succes (fond #eafaf1, texte #27ae60). input:invalid optionnel.
3. **JavaScript (35 min) :** script.js. IIFE. Constantes (PRIX_SAC=25000, REGEX_TEL_CM). État (let totalSacs=0, totalMontant=0, compteur=0). Sélection DOM hors handler. form.addEventListener('submit', handler). Handler : preventDefault(), reset message, récup .value.trim(), validation séquentielle (retour précoce), calcul montant, création ligne DOM (textContent), maj totaux, feedback, reset form.
4. **Tests & Relecture croisée (20 min) :** Grille de test fournie (cas nominaux, limites, erreurs). Échange de poste, rôle « testeur malveillant », correction bugs.

Grille de relecture croisée (Pair Review) — Cas limites à tester :

Cas de test	Données entrées	Résultat attendu
Nom vide	"" , Cacao, 10, 6900000000	Erreur nom, tableau inchangé
Nom court	"A" , Cacao, 10, 6900000000	Erreur nom
Filière non choisie	"Jean" , "" , 10, 6900000000	Erreur filière
Quantité 0	"Jean" , Cacao, 0, 6900000000	Erreur quantité
Quantité 501	"Jean" , Cacao, 501, 6900000000	Erreur quantité
Quantité décimale	"Jean" , Cacao, 10.5, 6900000000	Erreur quantité (entier requis)
Téléphone 8 chiffres	"Jean" , Cacao, 10, 690000000	Erreur téléphone
Téléphone 10 chiffres	"Jean" , Cacao, 10, 69000000000	Erreur téléphone
Téléphone commence par 5	"Jean" , Cacao, 10, 5900000000	Erreur téléphone
Cas nominal 1	"Marie Nono" , Café, 50, 677889900	Ligne + Total : 50 sacs, 1 250 000 FCFA
Cas nominal 2	"Paul Biya" , Maïs, 200, 233445566	Ligne + Total : 250 sacs, 6 250 000 FCFA

EXEMPLE Solution complète commentée : registre.html / style.css / script.js

Stratégie : Fichier unique registre.html pour test immédiat (copier-coller dans navigateur). En production : séparer en 3 fichiers (.html, .css, .js).

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Registre des Apports – Coopérative Espoir Bafoussam</title>
  <style>
    /* --- CSS INTÉGRÉ POUR L'EXEMPLE (à extraire dans style.css) --- */
    body { font-family : system-ui, sans-serif; max-width : 960px; margin : 2rem auto; padding : 0 1rem;
    background : #f9f9f9; line-height : 1.5; }
    h1 { color : #2c3e50; text-align : center; border-bottom : 2px solid #27ae60; padding-bottom : .5rem;
    margin-bottom : 1.5rem; }
```

```

    fieldset { border : 1px solid #ddd; border-radius : 8px; padding : 1.5rem; background : white; box-shadow
: 0 2px 4px rgba(0,0,0,.05); margin-bottom : 2rem; }
    legend { font-weight : bold; color : #27ae60; padding : 0 .5rem; font-size : 1.1rem; }
    .form-group { margin-bottom : 1rem; display : flex; flex-direction : column; gap : .3rem; }
    label { font-weight : 600; font-size : .95rem; color : #34495e; }
    input, select { padding : .6rem .8rem; border : 1px solid #ccc; border-radius : 4px; font-size : 1rem; font-
family : inherit; }
    input :focus, select :focus { outline : none; border-color : #27ae60; box-shadow : 0 0 3px rgba
(39,174,96,.2); }
    button[type="submit"] { background : #27ae60; color : white; border : none; padding : .8rem 1.5rem; font-
size : 1rem; border-radius : 4px; cursor : pointer; font-weight : bold; transition : background .2s; width :
100%; margin-top : .5rem; }
    button[type="submit"] :hover { background : #219653; }
    #message { margin-bottom : 1rem; padding : 1rem; border-radius : 4px; font-weight : bold; text-align :
center; display : none; }
    #message.erreur { display : block; background : #fdecea; color : #c0392b; border : 1px solid #fadbd8; }
    #message.succes { display : block; background : #eafaf1; color : #27ae60; border : 1px solid #a9dfbf; }
    table { width : 100%; border-collapse : collapse; background : white; box-shadow : 0 2px 4px rgba
(0,0,0,.05); border-radius : 8px; overflow : hidden; }
    th, td { padding : .8rem 1rem; border-bottom : 1px solid #eee; text-align : left; }
    th { background : #2c3e50; color : white; font-weight : 600; white-space : nowrap; }
    td :nth-child(4), td :nth-child(6), th :nth-child(4), th :nth-child(6) { text-align : right; font-variant-numeric :
tabular-nums; }
    tbody tr :hover { background : #f5f5f5; }
    tfoot { font-weight : bold; background : #f8f9fa; }
    tfoot td { border-top : 2px solid #2c3e50; padding-top : 1rem; }
    .sr-only { position : absolute; width : 1px; height : 1px; padding : 0; margin : -1px; overflow : hidden; clip :
rect(0.0,0.0,0); white-space : nowrap; border : 0; }
</style>

```

```
</head>
```

```
<body>
```

```
<h1>≡ Registre des Apports — Coopérative Espoir de Bafoussam</h1>
```

```
<!-- Zone de feedback utilisateur (ARIA live pour accessibilité) -->
```

```
<div id="message" aria-live="polite"></div>
```

```
<!-- FORMULAIRE -->
```

```
<form id="formApport" novalidate>
```

```
  <fieldset>
```

```
    <legend>Nouvel Apport</legend>
```

```
    <div class="form-group">
```

```
      <label for="nom">Nom du membre <span aria-hidden="true">*</span></label>
```

```
      <input type="text" id="nom" name="nom" required minlength="2" autocomplete="name" placeholder
="Ex : Marie Nono">
```

```
    </div>
```

```
    <div class="form-group">
```

```
      <label for="filier" >Filière <span aria-hidden="true">*</span></label>
```

```
      <select id="filier" name="filier" required>
```

```
        <option value="" disabled selected>-- Choisir une filière --</option>
```

```
        <option value="Cacao">Cacao</option>
```

```
        <option value="Café">Café</option>
```

```
        <option value="Maïs">Maïs</option>
```

```
        <option value="Banane plantain">Banane plantain</option>
```

```
      </select>
```

```
    </div>
```

```
  <div class="form-group">
```

```

    <label for="quantite">Quantité (sacs de 50 kg) <span aria-hidden="true">*</span></label>
    <input type="number" id="quantite" name="quantite" required min="1" max="500" step="1"
placeholder="Ex : 50" autocomplete="off">
  </div>

  <div class="form-group">
    <label for="telephone">Téléphone (9 chiffres, 6 ou 2) <span aria-hidden="true">*</span></label>
    <input type="tel" id="telephone" name="telephone" required pattern="[26][0-9]{8}" placeholder="Ex :
690123456" autocomplete="tel">
  </div>

  <button type="submit">≡ Enregistrer l'apport</button>
</fieldset>
</form>

<!-- TABLEAU RÉCAPITULATIF -->
<table>
  <caption class="sr-only">Liste des apports enregistrés</caption>
  <thead>
    <tr>
      <th scope="col">N°</th>
      <th scope="col">Nom</th>
      <th scope="col">Filière</th>
      <th scope="col">Quantité (sacs)</th>
      <th scope="col">Téléphone</th>
      <th scope="col">Montant (FCFA)</th>
    </tr>
  </thead>
  <tbody id="corpsTableau">
    <!-- Lignes injectées par JavaScript -->
  </tbody>
  <tfoot>
    <tr>
      <td colspan="3">TOTAL SACS</td>
      <td id="totalSacs" style="text-align:right;">0</td>
      <td colspan="2"></td>
    </tr>
    <tr>
      <td colspan="3">TOTAL MONTANT</td>
      <td colspan="2"></td>
      <td id="totalMontant" style="text-align:right;">0 FCFA</td>
    </tr>
  </tfoot>
</table>

<script>
// --- JAVASCRIPT INTÉGRÉ (à extraire dans script.js) ---
(() => { // IIFE : encapsulation, pas de pollution globale (window)

  // 1. CONSTANTES MÉTIER & ÉTAT DE L'APPLICATION
  const PRIX_SAC = 25000;
  let totalSacs = 0;
  let totalMontant = 0;
  let compteurLignes = 0;

  // 2. SÉLECTION DOM (Une seule fois au chargement – Performance)
  const form = document.getElementById('formApport');
  const messageDiv = document.getElementById('message');
  const corpsTableau = document.getElementById('corpsTableau');

```

```
const totalSacsEl = document.getElementById('totalSacs');
const totalMontantEl = document.getElementById('totalMontant');

// Références aux champs (pour .value et .focus())
const nomInput = document.getElementById('nom');
const filiereSelect = document.getElementById('filiere');
const quantiteInput = document.getElementById('quantite');
const telephoneInput = document.getElementById('telephone');

// Regex stricte téléphone Cameroun : 9 chiffres, commence par 2 (fixe) ou 6 (mobile)
const REGEX_TEL_CM = /^[26][0-9]{8}$/;

// 3. FONCTIONS UTILITAIRES (Séparation des responsabilités)
function afficherMessage(texte, type) {
  messageDiv.textContent = texte;
  messageDiv.className = type; // 'erreur' ou 'succes' (remplace classes existantes)
}

function formaterFCFA(nombre) {
  // toLocaleString('fr-FR') : espace insécable comme séparateur de milliers (norme française)
  return nombre.toLocaleString('fr-FR') + ' FCFA';
}

function resetFormulaire() {
  form.reset(); // Remet select sur option value="" (désactivée), vide inputs
  nomInput.focus(); // Prêt pour saisie suivante (UX)
}

// 4. GESTIONNAIRE D'ÉVÉNEMENT PRINCIPAL (Soumission formulaire)
form.addEventListener('submit', function(e) {
  e.preventDefault(); // ⚠ ESSENTIEL : Empêche rechargement page (perte état JS)

  // Reset feedback visuel
  afficherMessage('', '');

  // Récupération & nettoyage valeurs (trim supprime espaces début/fin)
  const nom = nomInput.value.trim();
  const filiere = filiereSelect.value; // "" si option par défaut sélectionnée
  const quantiteStr = quantiteInput.value.trim();
  const telephone = telephoneInput.value.trim();

  // --- VALIDATION SÉQUENTIELLE (Court-circuit : return à la 1ère erreur) ---

  // 1. Nom : longueur >= 2
  if (nom.length < 2) {
    afficherMessage('⚠ Nom invalide : 2 caractères minimum.', 'erreur');
    nomInput.focus();
    return;
  }

  // 2. Filière : valeur non vide
  if (!filiere) {
    afficherMessage('⚠ Veuillez sélectionner une filière.', 'erreur');
    filiereSelect.focus();
    return;
  }

  // 3. Quantité : entier strict entre 1 et 500
  // parseInt(base 10) + vérifications robustes
  const quantite = parseInt(quantiteStr, 10);
```

```

    if (isNaN(quantite) || quantite < 1 || quantite > 500 || !Number.isInteger(quantite)) {
        afficherMessage('≡ Quantité invalide : entier entre 1 et 500 sacs.', 'erreur');
        quantiteInput.focus();
        quantiteInput.select(); // Sélectionne texte pour correction rapide
        return;
    }

    // 4. Téléphone : Regex ancrée stricte
    if (!REGEX_TEL_CM.test(telephone)) {
        afficherMessage('≡ Téléphone invalide : 9 chiffres, commençant par 6 ou 2 (ex : 690123456).', 'erreur');
        telephoneInput.focus();
        telephoneInput.select();
        return;
    }

    // --- SUCCÈS : TRAITEMENT MÉTIER & MISE À JOUR VUE ---

    const montant = quantite * PRIX_SAC;

    // Mise à jour ÉTAT (Modèle)
    totalSacs += quantite;
    totalMontant += montant;
    compteurLignes++;

    // Mise à jour DOM (Vue) – Création ligne tableau
    const tr = document.createElement('tr');
    tr.dataset.id = compteurLignes; // Identifiant technique optionnel

    // Données des 6 cellules dans l'ordre des colonnes
    const cellulesData = [
        compteurLignes, // 1. N°
        nom,             // 2. Nom
        filiere,         // 3. Filière
        quantite,        // 4. Quantité
        telephone,       // 5. Téléphone
        formaterFCFA(montant) // 6. Montant formaté
    ];

    cellulesData.forEach((contenu, index) => {
        const td = document.createElement('td');
        td.textContent = contenu; // textContent = PROTECTION XSS AUTOMATIQUE
        // Alignement droit pour colonnes numériques (index 3 = Quantité, 5 = Montant)
        if (index === 3 || index === 5) td.style.textAlign = 'right';
        tr.appendChild(td);
    });

    // Injection dans tbody (à la fin)
    corpsTableau.appendChild(tr);

    // Mise à jour pied de tableau (Totaux)
    totalSacsEl.textContent = totalSacs.toLocaleString('fr-FR');
    totalMontantEl.textContent = formaterFCFA(totalMontant);

    // Feedback utilisateur & préparation saisie suivante
    afficherMessage('≡ Apport de ${nom} (${quantite} sacs ${filiere}) enregistré.', 'succes');
    resetFormulaire();

}); // Fin addEventListener

})(); // Fin IIFE

```

</script>

</body>
</html>**PROPRIÉTÉ** Analyse détaillée du corrigé — Points clés pour l'oral/écrit du Baccalauréat

1. Architecture IIFE `((() => { ... } )())` : Encapsule tout le script. Variables `totalSacs`, `totalMontant`, `REGEX_TEL_CM` privées, pas de pollution `window`. Bonne pratique pro (propreté, évite conflits scripts tiers).
2. `novalidate` sur `<form>` : Désactive validation HTML5 native (bulles navigateur non stylables). On impose notre UI centralisée (`#message`) et notre logique JS. Savoir-faire attendu : « Contrôler la saisie... en utilisant JavaScript ».
3. Sélection DOM une seule fois (hors handler) : `getElementById` au chargement. Dans handler, accès direct `nomInput.value`. Performance, lisibilité, évite re-recherche à chaque submit.
4. `e.preventDefault()` : **Point crucial n°1**. Sans cela, `submit` recharge page → variables JS remises à 0, tableau vidé, totaux à 0. Erreur n°1 des élèves.
5. Validation `quantite` robuste (anti-pièges) :
 - `parseInt(..., 10)` : Base 10 explicite (évite interprétation octale ancien).
 - `isNaN(quantite)` : Détecte « abc », vide, « 10.5 » (renvoie 10 mais `Number.isInteger` catch).
 - `!Number.isInteger(quantite)` : Rejette « 10.5 » (même si `step="1"` aide UI, JS ne fait pas confiance au navigateur).
 - Bornes `>= 1 && <= 500`.
6. Regex Téléphone `/^[26][0-9]{8}$/` : Ancrée (`^` \$), stricte. Rejette +237, espaces, 8/10 chiffres, préfixes 3/5/7/8/9 non attribués au Cameroun.
7. `textContent` vs `innerHTML` : Usage systématique de `textContent` pour `nom`, `telephone`, etc. Protection XSS native. Si membre s'appelle `<script>alert(1)</script>`, affiché littéralement, non exécuté. Mentionner à l'oral = compétence « Sécurité » validée.
8. `toLocaleString('fr-FR')` : Formatage natif nombres avec espace insécable (1 250 000). Plus robuste/standard que regex manuelle.
9. Accessibilité (ARIA/HTML5) : `aria-live="polite"` sur `#message` (lecteurs d'écran annoncent changement). `scope="col"` sur `<th>`. `<label for="id">`. `<caption class="sr-only">`. Ce sont des « savoir-être » (qualité logicielle) attendus en Terminale TI.
10. `form.reset()` : Remet `<select>` sur option par défaut (`value=""`). Pratique pour enchaîner saisies rapides.

8.8 Bilan de la leçon et Perspectives

Cette leçon a concrétisé la chaîne complète du développement web **côté client** (Front-end) :

- **HTML5 sémantique & accessible** : `<form>`, `<fieldset>`/`<legend>`, `<label for>`, `<table>`/`<thead>`/`<tbody>` `<caption>`, `scope`, `aria-live`, `novalidate`.
- **CSS3 moderne** : Conteneur centré `max-width`, police système, `border-collapse`, alignement tabulaire nombres (`font-variant-numeric: tabular-nums`), états visuels (focus, hover, erreur/succès), design responsive simple.
- **JavaScript (ES6+) — Niveau Bac TI** :
 - **DOM API** : `getElementById`, `createElement`, `appendChild`, `textContent`, `className`, `dataset`.
 - **Événements** : `addEventListener('submit', ...)`, `preventDefault()`, objet `Event`.
 - **Algorithmique validation** : Chaîne `if/return` (court-circuit), `RegExp.prototype.test()`, `parseInt(base 10) + Number.isInteger()`, bornes.
 - **Gestion d'état** : Variables globales au module (`let totalSacs, totalMontant`) mutées à chaque succès (pattern Modèle-Vue simplifié).
 - **Internationalisation (i18n)** : `toLocaleString('fr-FR')` pour FCFA.
- **Sécurité intégrée** : Prise de conscience risque XSS via `textContent` exclusif (jamais `innerHTML` pour données utilisateur).
- **Qualité logicielle / Test** : Grille de relecture croisée (Pair Review) = culture du **test boîte noire** (cas limites, injection, bornes) et de la **revue de code**, compétences transversales essentielles pour BTS/DUT/Insertion pro.

EXERCICE 6 Extensions proposées — Projet personnel / Approfondissement

1. **Persistence localStorage** : Sauvegarder le tableau (`JSON.stringify`) à chaque ajout ; recharger au `DOMContentLoaded` (survie au F5).
2. **Export CSV** : Bouton « Exporter » générant `data:text/csv;charset=utf-8,\u000d...` avec `encodeURIComponent` et déclenchant téléchargement via `<a download="registre.csv">`.
3. **Suppression ligne** : Ajouter bouton par ligne (création `td` avec `button`, écouteur `click` délégué sur `tbody` ou direct), maj totaux à la baisse.
4. **Backend réel** : Remplacer stockage local par `fetch('/api/apports', {method: 'POST', body: JSON.stringify(data)})` vers API PHP/Node.js + Base de données MySQL/PostgreSQL (prochain module « Programmation côté serveur »).
5. **Authentification** : Page `login.html` + JWT stocké en `sessionStorage` / `cookie HttpOnly`, protection routes.

CORRIGÉ Exercice 1 — Persistance localStorage (ébauche)

```
// Au chargement (dans IIFE, avant addEventListener)
function chargerDepuisStockage() {
  const data = JSON.parse(localStorage.getItem('registreApports') || '[]');
  data.forEach(apport => {
    // Réinjecter dans DOM (réutiliser logique création ligne)
    // Mettre à jour totalSacs, totalMontant, compteurLignes
  });
}
chargerDepuisStockage();

// Dans handler succès, APRÈS maj DOM :
function sauverDansStockage() {
  const lignes = [...corpsTableau.querySelectorAll('tr')].map(tr => ({
    nom : tr.children[1].textContent,
    filiere : tr.children[2].textContent,
    quantite : parseInt(tr.children[3].textContent, 10),
    telephone : tr.children[4].textContent
  }));
  localStorage.setItem('registreApports', JSON.stringify(lignes));
}
sauverDansStockage();
```

9 MÉTHODES ET EXERCICES RÉSOLUS

10 RAPPELS DE PROGRAMMATION WEB : HTML ET JAVASCRIPT

10.1 Structure d'une page HTML

MÉTHODE Créer une page HTML5 structurée et sémantique

Objectif : Produire un squelette HTML5 valide (W3C) respectant la séparation structure/-contenu et les exigences d'accessibilité (WCAG) et de référencement (SEO).

1. Déclarer le type de document : Première ligne impérative : `<!DOCTYPE html>`. Elle active le mode standard (standard mode) du navigateur.
2. Ouvrir l'élément racine : `<html lang="fr">`. L'attribut **lang** indique la langue principale (ici français) pour les lecteurs d'écran et les moteurs de recherche.
3. Construire le `<head>` (métadonnées invisibles) :
 - Encodage universel : `<meta charset="UTF-8">` (obligatoire pour gérer accents, symboles monétaires FCFA, émojis).
 - Viewport mobile (responsive) : `<meta name="viewport" content="width=device-width, initial-scale=1.0">`.
 - Titre de l'onglet : `<title>Titre précis de la page</title>` (unique, ~60 car., crucial SEO).
 - Description SEO : `<meta name="description" content="Résumé de la page (150 car.)">`.
 - Liaison CSS : `<link rel="stylesheet" href="styles.css">`.
 - (Optionnel) Favicon : `<link rel="icon" href="favicon.ico">`.
4. Construire le `<body>` (contenu visible) avec balises sémantiques HTML5 :
 - `<header>` : En-tête du site (logo, titre principal, navigation principale `<nav>`).
 - `<nav>` : Zone de navigation majeure (menus). Utiliser `<ul>/<li>/<a>` à l'intérieur.
 - `<main>` : Contenu principal unique par page. Contient `<section>` (regroupement thématique avec titre), `<article>` (contenu autonome, ex : article de blog), `<aside>` (contenu annexe, sidebar).
 - `<footer>` : Pied de page (copyright, mentions légales, contacts, réseaux sociaux, plan du site).

5. Fermer les balises dans l'ordre inverse (LIFO) : `</main></body></html>`.
6. Valider la conformité : Utiliser validator.w3.org (copier-coller code, fichier ou URL). Corriger toutes les erreurs (rouge) et avertissements (jaune).

Réflexe Bac : Toujours indiquer `lang` sur `<html>` et `charset UTF-8`. Une page sans `<main>` ou avec plusieurs `<main>` est invalide. Les balises `<div>` et `<span>` (non sémantiques) ne servent qu'au stylage CSS quand aucune balise sémantique ne convient.

EXERCICE RÉSOLU 7

Structure complète d'une page d'accueil « École Numérique Cameroun »

Énoncé : Écrire le code HTML5 complet d'une page d'accueil pour la plateforme fictive « École Numérique Cameroun ». La page doit contenir : un en-tête avec le logo et un menu de navigation (Accueil, Cours, Inscription, Contact), une zone principale avec une section « Bienvenue » (titre h1, paragraphe) et une section « Nos formations » (titre h2, liste de 3 formations), et un pied de page avec copyright et lien vers les mentions légales.

SOLUTION

Solution détaillée :

1. On applique la méthode ci-dessus : DOCTYPE, `html [lang=fr]`, head (meta charset, viewport, title, description, lien CSS), body.
2. Dans `<header>` : `<img>` pour logo (alt obligatoire pour accessibilité), `<nav>` avec liste non ordonnée `<ul>` de liens `<a href="#"></a>`. Attribut `aria-current="page"` sur le lien de la page courante.
3. Dans `<main>` : une `<section id="bienvenue">` avec `<h1>` et `<p>`. Une `<section id="formations">` avec `<h2>` et `<ul>` de 3 `<li>`.
4. Dans `<footer>` : `<p>` copyright, `<a href="mentions.html">`Mentions légales`</a>`.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="École Numérique Cameroun – Formations d'excellence en
développement web, réseaux, cybersécurité et data science.">
  <title>École Numérique Cameroun – Accueil</title>
  <link rel="stylesheet" href="styles.css">
  <link rel="icon" href="favicon.ico">
</head>
<body>
  <header>
    
    <nav aria-label="Navigation principale">
      <ul>
```

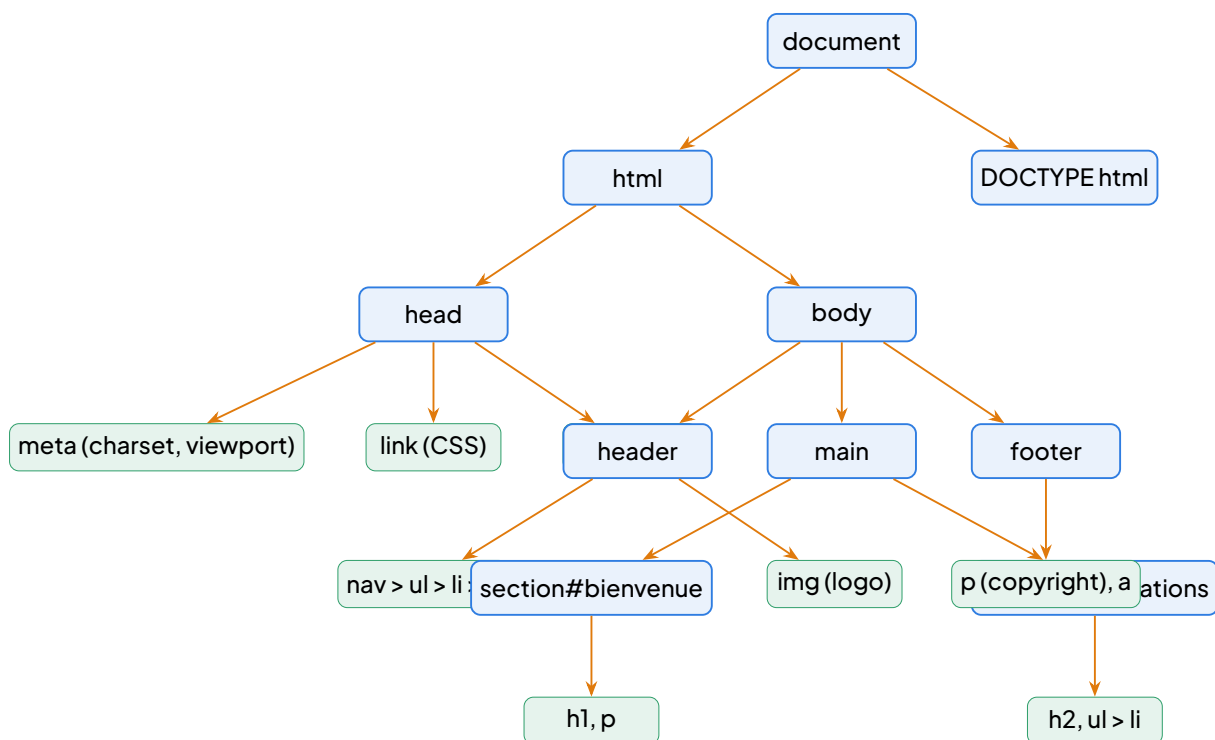
```
<li><a href="index.html" aria-current="page">Accueil</a></li>
<li><a href="cours.html">Cours</a></li>
<li><a href="inscription.html">Inscription</a></li>
<li><a href="contact.html">Contact</a></li>
</ul>
</nav>
</header>

<main>
  <section id="bienvenue" aria-labelledby="titre-bienvenue">
    <h1 id="titre-bienvenue">Bienvenue à l'École Numérique Cameroun</h1>
    <p>Votre passerelle vers l'excellence technologique au cœur de l'Afrique.</p>
  </section>

  <section id="formations" aria-labelledby="titre-formations">
    <h2 id="titre-formations">Nos formations phares</h2>
    <ul>
      <li>Développement Web Full Stack (PHP, JavaScript, MySQL)</li>
      <li>Administration Réseau & Cybersécurité (Cisco, Linux)</li>
      <li>Data Science & Intelligence Artificielle (Python, TensorFlow)</li>
    </ul>
  </section>
</main>

<footer>
  <p>&copy; 2025 École Numérique Cameroun. Tous droits réservés.</p>
  <p><a href="mentions.html">Mentions légales</a> | <a href="contact.html">Nous contacter</a>
</p>
</footer>
</body>
</html>
```

Conclusion : Le code est valide W3C, sémantique (header, nav, main, section, footer), accessible (alt, aria-current, aria-labelledby, lang) et responsive (viewport).



Arbre DOM simplifié de la page « École Numérique Cameroun »

10.2 Les formulaires HTML

MÉTHODE Concevoir un formulaire HTML complet, accessible et sécurisé

Objectif : Créer un formulaire robuste (`<form>`) avec contrôles variés, étiquettes liées (`<label>`), regroupements logiques (`<fieldset>`) et attributs de validation native HTML5 (première ligne de défense).

- Balise conteneur :** `<form action="traitement.php" method="POST" novalidate>`.
 - `action` : URL du script serveur (PHP, Node, Python...). Laisser vide pour soumission sur la même page.
 - `method` : POST (données sensibles, modification BD, upload) ou GET (recherche, filtrage, idempotent, données en URL).
 - `enctype="multipart/form-data"` : Obligatoire si `<input type="file">` présent.
 - `novalidate` : Indispensable pour désactiver la validation navigateur par défaut (bulles natives) et laisser la main à notre JavaScript personnalisé (sinon JS ne s'exécute pas si HTML5 bloque l'envoi).
- Chaque champ saisissable (`<input>`, `<select>`, `<textarea>`) DOIT avoir une `<label>` liée explicitement : `<label for="idChamp">Libellé </label> <input type="..."`

`id="idChamp" name="nomChamp" required>`. L'attribut `name` est obligatoire pour l'envoi des données (clé `$_POST['nomChamp']` en PHP, clé `FormData` en JS). L'id doit être unique dans la page.

3. Choisir le type adapté (validation native navigateur + clavier mobile adapté) :

<code>text</code>	Texte simple.
<code>email</code>	Format @ validé nativement.
<code>tel</code>	Clavier numérique mobile (pas de validation de format native).
<code>password</code>	Masqué (points). Utiliser <code>autocomplete="new-password"</code> pour création, <code>"current-password"</code> pour connexion.
<code>number</code>	Nombre (attributs <code>min</code> , <code>max</code> , <code>step</code>).
<code>date</code>	Sélecteur de date natif (format ISO YYYY-MM-DD).
<code>file</code>	Upload. <code>accept="image/*, .pdf"</code> (filtre boîte dialogue).
<code>checkbox / radio</code>	Case / Bouton radio. Même <code>name</code> pour groupe radio, valeur distinctes.
<code>hidden</code>	Champ invisible (token CSRF, ID technique).

4. Regrouper les champs liés : `<fieldset>` + `<legend>` (ex : « Civilité », « Adresse », « Abonnement »). Améliore l'accessibilité (lecteurs d'écran annoncent la légende).

5. Zone de texte multiligne : `<textarea id="msg" name="message" rows="5" cols="40" placeholder="Votre message..."></textarea>` (pas d'attribut `value`, contenu initial entre balises).

6. Liste déroulante : `<select id="pays" name="pays"> <option value="">-- Choisir --</option> <option value="CM">Cameroun</option> ... </select>`. Option vide `value=""` pour forcer un choix actif.

7. Boutons : `<button type="submit">Valider</button>` (défaut `submit`), `<button type="reset">Annuler</button>` (remet valeurs initiales), `<button type="button">Action JS</button>` (ne soumet pas, pour JS pur).

8. Attributs de contrainte utiles (API Constraint Validation) : `required`, `minlength`/`maxlength`, `min`/`max`/`step` (number/date), `pattern="regex"` (ex : `[0-9]10` pour téléphone CM, `ENC-[0-9]5` pour matricule), `placeholder` (exemple visuel, jamais remplacement de `<label>`), `autocomplete` (aide navigateur), `autofocus` (un seul par page).

EXERCICE RÉSOLU 8

Formulaire d'inscription « Concours ENC 2025 » avec validation HTML5

Énoncé : Réaliser le code HTML d'un formulaire d'inscription au concours de l'École Numérique Cameroun. Champs requis : Matricule (format `ENC-XXXXX`, 5 chiffres), Nom, Pré-

nom, Email institutionnel (@enc.cm), Date de naissance (mineur < 18 ans interdit), Région d'origine (liste 10 régions + Diaspora), Sexe (radio), Pièce d'identité (upload PDF/JPG max 2Mo), Mot de passe (min 8 car, 1 maj, 1 min, 1 chiffre), Confirmation mot de passe, Case à cocher CGU. Boutons : « S'inscrire » et « Réinitialiser ».

SOLUTION

Solution détaillée :

1. `<form action="inscription.php" method="POST" enctype="multipart/form-data" novalidate>` : **enctype indispensable pour type="file"**.
2. **Matricule** : `type="text" + pattern="ENC-[0-9]5" + placeholder="ENC-12345" + required`.
3. **Email** : `type="email" + pattern=".*@enc.cm$" (force domaine) + required`.
4. **Date naissance** : `type="date" + max="2007-12-31" (18 ans en 2025) + required`. Note : max dynamique côté serveur/JS préférable.
5. **Région** : `<select> avec <option value="CM-CE">Centre</option> etc. Première option vide value="" pour forcer choix`.
6. **Sexe** : `<fieldset> + <legend>Sexe</legend> + 2 <input type="radio" name="sexe" value="M" id="sexeM" required> <label for="sexeM">Masculin</label>...` (required sur un seul radio du groupe suffit).
7. **Fichier** : `<input type="file" name="piece_id" accept=".pdf,image/jpeg,image/png" required>`. Taille max 2Mo non gérable en HTML5 pur (côté serveur ou JS File API).
8. **MDP** : `type="password" + pattern="(?!.*[a-z])(?!.*[A-Z])(?!.*[0-9]).8," + required`. Attention : \d non standard dans HTML pattern, utiliser [0-9].
9. **Confirmation** : `type="password" + required` (égalité vérifiée en JS, pas en HTML5 natif).
10. **CGU** : `<input type="checkbox" name="cgu" id="cgu" required> <label for="cgu">J'accepte les CGU</label>`.

```
<form action="inscription.php" method="POST" enctype="multipart/form-data" novalidate>
  <div>
    <label for="matricule">Matricule :</label>
    <input type="text" id="matricule" name="matricule" pattern="ENC-[0-9]{5}" placeholder="ENC-12345" required title="Format : ENC- suivi de 5 chiffres">
  </div>
  <div>
    <label for="nom">Nom :</label>
    <input type="text" id="nom" name="nom" required maxlength="50">
  </div>
  <div>
    <label for="prenom">Prénom :</label>
    <input type="text" id="prenom" name="prenom" required maxlength="50">
  </div>
  <div>
    <label for="email">Email institutionnel (@enc.cm) :</label>
```

```

        <input type="email" id="email" name="email" pattern=".*@enc\.cm$" required placeholder="
        dupont@enc.cm">
    </div>
    <div>
        <label for="naissance">Date de naissance (18 ans min) :</label>
        <input type="date" id="naissance" name="naissance" max="2007-12-31" required>
    </div>
    <div>
        <label for="region">Région d'origine :</label>
        <select id="region" name="region" required>
            <option value="">-- Sélectionnez --</option>
            <option value="CM-AD">Adamaoua</option>
            <option value="CM-CE">Centre</option>
            <option value="CM-ES">Est</option>
            <option value="CM-LT">Littoral</option>
            <option value="CM-NO">Nord</option>
            <option value="CM-NW">Nord-Ouest</option>
            <option value="CM-OU">Ouest</option>
            <option value="CM-SU">Sud</option>
            <option value="CM-SW">Sud-Ouest</option>
            <option value="CM-EN">Extrême-Nord</option>
            <option value="CM-DI">Diaspora</option>
        </select>
    </div>
    <fieldset>
        <legend>Sexe</legend>
        <label><input type="radio" name="sexe" value="M" id="sexeM" required> Masculin</label>
        <label><input type="radio" name="sexe" value="F" id="sexeF"> Féminin</label>
    </fieldset>
    <div>
        <label for="piece">Pièce d'identité (PDF, JPG, PNG – Max 2Mo) :</label>
        <input type="file" id="piece" name="piece_id" accept=".pdf,image/jpeg,image/png" required>
    </div>
    <div>
        <label for="mdp">Mot de passe :</label>
        <input type="password" id="mdp" name="mdp" pattern="(?=.*[a-z])(?=.*[A-Z])(?=.*[0-9]).{8,}"
        required title="Min 8 caractères, 1 maj, 1 min, 1 chiffre">
    </div>
    <div>
        <label for="mdp2">Confirmer le mot de passe :</label>
        <input type="password" id="mdp2" name="mdp2" required>
    </div>
    <div>
        <input type="checkbox" id="cgu" name="cgu" required>
        <label for="cgu">J'accepte les <a href="cgu.pdf" target="_blank">Conditions Générales d'
        Utilisation</a></label>
    </div>
    <div class="boutons">
        <button type="submit">S'inscrire</button>
        <button type="reset">Réinitialiser</button>
    </div>
</form>

```

Conclusion : Le formulaire utilise les types HTML5 spécialisés (**email**, **tel**, **date**, **file**), **pattern** pour **regex** complexes, **required** obligatoire, **novalidate** sur le **form** pour déléguer au JS, **enctype** pour l'upload. La validation serveur (PHP) reste indispensable (le client est

manipulable via DevTools, curl, Postman, JS désactivé).

10.3 Les tableaux HTML

MÉTHODE

Construire un tableau HTML structuré, accessible et stylable

Objectif : Produire un tableau de données (`<table>`) sémantiquement correct (légende, en-têtes, portée, regroupements) pour affichage de données tabulaires (notes, produits, utilisateurs, planning). Interdiction formelle d'utiliser `<table>` pour la mise en page (layout) → CSS Grid/Flexbox.

1. **Conteneur :** `<table>`. Attributs de présentation dépréciés (`border`, `cellpadding`, `cellspacing`, `width`, `align`, `bgcolor`) interdits → CSS uniquement.
 2. **Légende (obligatoire accessibilité) :** `<caption>`Liste des étudiants admis session 2025`</caption>` (juste après `<table>`, avant `<thead>`). Décrit le contenu du tableau.
 3. **Regroupements de lignes (structure logique) :**
 - `<thead>` : Groupe les lignes d'en-tête (une ou plusieurs). Permet répétition en-têtes sur pages imprimées.
 - `<tbody>` : Corps principal (données). Peut y avoir plusieurs `<tbody>` pour grouper logiquement (ex : par classe, par catégorie).
 - `<tfoot>` : Pied de tableau (totaux, moyennes, résumé). Se place avant `<tbody>` dans le code source HTML5 (rendu visuel en bas via CSS `display: table-footer-group` ou comportement natif).
 4. **Lignes :** `<tr>` (Table Row).
 5. **Cellules d'en-tête :** `<th scope="col">Nom</th>` (en-tête de colonne) ou `<th scope="row">Total</th>` (en-tête de ligne, ex : 1ère cellule d'une ligne de données). L'attribut `scope` (**col**, **row**, **colgroup**, **rowgroup**) est obligatoire pour les lecteurs d'écran (associe en-tête aux données).
 6. **Cellules de données :** `<td>Contenu</td>`.
 7. **Fusion de cellules :** `colspan="n"` (horizontal), `rowspan="n"` (vertical). Attention au comptage des cellules dans les `<tr>` concernées (une fusion réduit le nombre de `<td>/<th>` à écrire dans la ligne).
 8. **Colonnes (stylage global rare mais possible) :** `<colgroup>` + `<col span="2" style="width: 20%;">/<col id="col-note">`.
- Formule Bac :** `<table>` → `<caption>` → `<thead>` → `<tr>` → `<th scope="col">` → `<tfoot>` → `<tr>` → `<th scope="row">` → `<tbody>` → `<tr>` → `<td>`.

EXERCICE RÉSOLU 9 Tableau de bulletins de notes « Lycée Bilingue de Bafoussam »

Énoncé : Construire le tableau HTML affichant le bulletin du 1er trimestre pour la classe de Terminale TI A. Colonnes : N°, Matricule, Nom, Prénom, Maths (coef 4), Physique (coef 3), Info (coef 3), Anglais (coef 2), Moyenne pondérée, Rang. Données : 3 élèves. Ajouter une ligne de totals (moyennes de classe par matière) dans `<tfoot>`. Assurer l'accessibilité.

SOLUTION

Solution détaillée :

1. **Structure** : `table > caption, thead, tfoot, tbody`.
2. **En-tête (thead)** : 1 ligne `tr` avec 10 `th scope="col"`. Pour « Moyenne pondérée » et « Rang », `scope="col"`.
3. **Pied (tfoot)** : 1 ligne `tr` : 1 `<th scope="row" colspan="4">Moyennes classe</th>`, puis 5 `td` pour moyennes matières, 1 `td` moyenne générale classe, 1 `td` vide.
4. **Corps (tbody)** : 3 lignes `tr`. 1ère cellule `th scope="row"` pour le N° (identifiant de ligne). Matricule, Nom, Prénom en `td`. Notes en `td`. Moyenne et Rang en `td`.
5. **Calculs (exemple)** : Élève 1 : $(14 \times 4 + 12 \times 3 + 16 \times 3 + 10 \times 2) / (4 + 3 + 3 + 2) = 160 / 12 = 13,33$.

```
<table>
  <caption>Bulletins 1er Trimestre – Terminale TI A – Lycée Bilingue de Bafoussam</caption>
  <thead>
    <tr>
      <th scope="col">N°</th>
      <th scope="col">Matricule</th>
      <th scope="col">Nom</th>
      <th scope="col">Prénom</th>
      <th scope="col">Maths (Coef 4)</th>
      <th scope="col">Physique (Coef 3)</th>
      <th scope="col">Info (Coef 3)</th>
      <th scope="col">Anglais (Coef 2)</th>
      <th scope="col">Moyenne Pondérée</th>
      <th scope="col">Rang</th>
    </tr>
  </thead>
  <tfoot>
    <tr>
      <th scope="row" colspan="4">Moyennes de la classe</th>
      <td>12,50</td>
      <td>11,80</td>
      <td>13,20</td>
      <td>10,50</td>
      <td>12,15</td>
      <td></td>
    </tr>
  </tfoot>
  <tbody>
    <tr>
      <th scope="row">1</th>
      <td>25TI001</td>
```

```

        <td>KAMGA</td>
        <td>Armand</td>
        <td>14</td>
        <td>12</td>
        <td>16</td>
        <td>10</td>
        <td>13,33</td>
        <td>2</td>
    </tr>
    <tr>
        <th scope="row">2</th>
        <td>25TI002</td>
        <td>NGONO</td>
        <td>Marie</td>
        <td>16</td>
        <td>14</td>
        <td>15</td>
        <td>12</td>
        <td>14,58</td>
        <td>1</td>
    </tr>
    <tr>
        <th scope="row">3</th>
        <td>25TI003</td>
        <td>ESSOMBA</td>
        <td>Paul</td>
        <td>10</td>
        <td>11</td>
        <td>12</td>
        <td>9</td>
        <td>10,58</td>
        <td>3</td>
    </tr>
</tbody>
</table>

```

Conclusion : Tableau valide, accessible (caption, scope, th pour N°), structure thead/tbody/tfoot **correcte** (tfoot avant tbody en source). Le style (bordures border-collapse: collapse, alignement nombres à droite text-align: right, zébrage tbody tr:nth-child(even)) se fait en CSS.

10.4 JavaScript et le DOM

MÉTHODE Sélectionner et manipuler des éléments du DOM en JavaScript (Vanilla JS)

Objectif : Accéder aux nœuds HTML depuis un script JS pour lire/modifier contenu, attributs, styles, classes, structure. **DOM** (Document Object Model) = arbre d'objets représentant le document.

1. Placement du script : `<script src="app.js" defer></script>` en fin de `<body>` (ou

dans `<head>` avec `defer`). `defer` garantit que le DOM est construit avant exécution (pas besoin de `DOMContentLoaded`). `async` charge en parallèle mais exécute dès que prêt (ordre non garanti, évité pour DOM).

2. Sélection unique (renvoie `Element` ou `null`) :

- `document.getElementById('idUnique')` : Le plus rapide, privilégié si ID existe.
- `document.querySelector('selecteurCSS')` : Polyvalent (ID #id, classe `.class`, tag, attribut `[name="x"]`, combinateurs `>`, `+`, `:first-child...`). Renvoie le premier correspondant.

3. Sélection multiple (renvoie `NodeList` statique, itérable `forEach`) : `document.querySelectorAll('selecteurCSS')`.

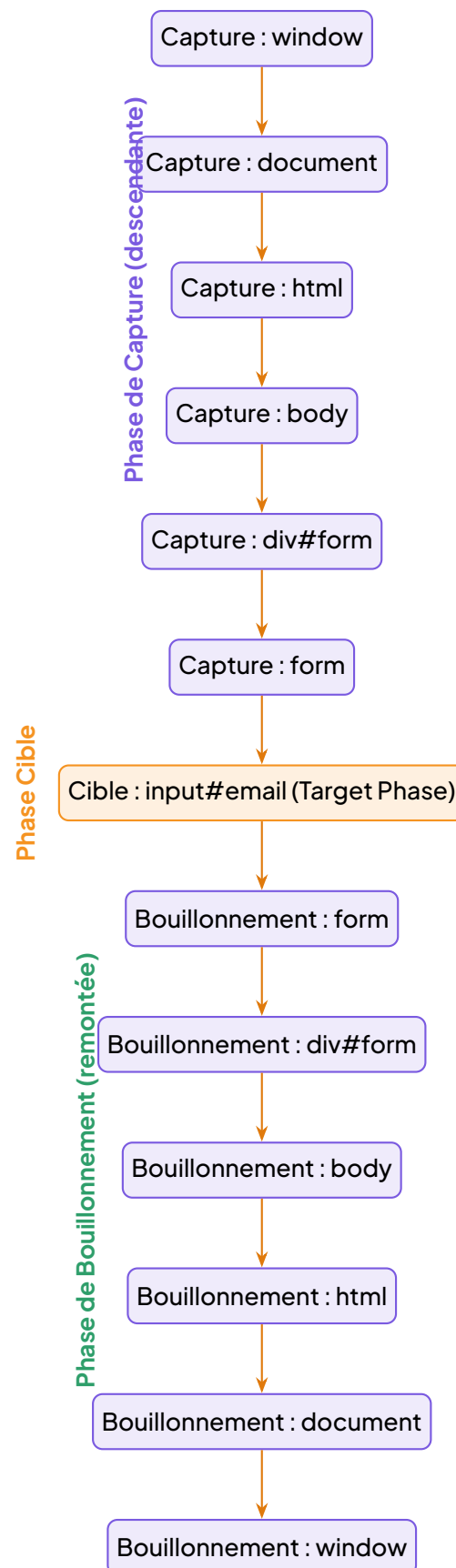
4. Navigation relative (depuis un élément `el`) : `el.parentElement`, `el.children` (HTML-Collection vive, éléments seulement), `el.firstElementChild`, `el.lastElementChild`, `el.previousElementSibling`, `el.nextElementSibling`, `el.closest('selecteur')` (remonte ancêtres jusqu'à correspondance, très puissant pour délégation).

5. Manipulation courante :

- **Contenu texte (sécurisé XSS)** : `el.textContent = "Nouveau texte" / const txt = el.textContent`.
- **Contenu HTML (dangereux si données non fiables → XSS)** : `el.innerHTML = "<strong>Gras</strong>"`.
- **Attributs** : `el.getAttribute('href')`, `el.setAttribute('disabled', '')`, `el.removeAttribute('hidden')`, `el.hasAttribute('required')`.
- **Propriétés DOM directes (préférées pour value, checked, disabled, className, id, style)** : `el.value`, `el.checked`, `el.disabled = true`, `el.id`, `el.classList.add('erreur')`, `el.classList.remove('valide')`, `el.classList.toggle('actif')`, `el.classList.contains('cache')`.
- **Styles inline** : `el.style.backgroundColor = 'red' (camelCase)`, `el.style.cssText = 'color: white; font-weight: bold;'`.

6. Création/Suppression dynamique : `const n = document.createElement('div');` `n.textContent = 'Hi'; el.appendChild(n); (fn), el.prepend(n); (début), el.before(n); (avant), el.after(n); (après), el.remove();` `el.replaceWith(nouveauNoeud);`

Attention : `innerHTML` + données utilisateur = faille **XSS** (Cross-Site Scripting). Toujours `textContent` pour texte pur. `querySelectorAll` renvoie `NodeList` (pas `Array`) : pas de `.map()`, `.filter()` natifs sans `Array.from()` ou `[...nodes]`.



Propagation d'un événement DOM : Capture → Cible → Bouillonnement (Bubbling).

`addEventListener(type, handler, true)` écoute en capture (défaut `false` = bouillonnement).

EXERCICE RÉSOLU 10 Mise à jour dynamique d'un panier d'achat « Jumia CM »

Énoncé : Une page affiche un tableau de panier (ID panier-body) avec lignes : Produit, Prix unitaire, Quantité (`<input type="number" min="1" class="qte">`), Sous-total, Bouton « Supprimer » (`<button class="btn-suppr">`). Écrire le JS qui : (1) Calcule et affiche le sous-total (Prix × Qté) à chaque changement de quantité. (2) Calcule et affiche le total général dans `<output id="total-general">`. (3) Supprime la ligne complète au clic sur « Supprimer » et recalcule le total.

SOLUTION

Solution détaillée :

1. On sélectionne le conteneur `const tbody = document.getElementById('panier-body');`
2. On utilise la délégation d'événement sur `tbody` (1 seul listener au lieu de N) : `tbody.addEventListener('input', ...)` pour les quantités (event input détecte frappe, flèches, copier-coller), `tbody.addEventListener('click', ...)` pour suppression.
3. Dans `gestionnaire input` : `if (e.target.matches('.qte')) { const tr = e.target.closest('tr'); const pu = +tr.querySelector('.prix').textContent; const qte = +e.target.value; tr.querySelector('.sous-total').textContent = (pu * qte).toFixed(2); calculerTotal(); }`
4. Dans `gestionnaire click` : `if (e.target.matches('.btn-suppr')) { e.target.closest('tr').remove(); calculerTotal(); }`
5. Fonction `calculerTotal()`: `const total = [...tbody.querySelectorAll('.sous-total')].reduce((sum, el) => sum + +el.textContent, 0); document.getElementById('total-general').value = total.toFixed(2);`

```
// app.js (chargé avec <script src="app.js" defer>)
document.addEventListener('DOMContentLoaded', () => { // Sécurité si pas de defer
  const tbody = document.getElementById('panier-body');
  const totalGeneralEl = document.getElementById('total-general');

  // 1. Délégation : changement quantité (event 'input' pour réactivité immédiate)
  tbody.addEventListener('input', (e) => {
    if (e.target.matches('.qte')) {
      const tr = e.target.closest('tr');
      const prixUnitaire = parseFloat(tr.querySelector('.prix').textContent);
      const quantite = parseFloat(e.target.value) || 0;
      const sousTotal = prixUnitaire * quantite;
      tr.querySelector('.sous-total').textContent = sousTotal.toFixed(2);
      calculerTotalGeneral();
    }
  });

  // 2. Délégation : clic suppression
  tbody.addEventListener('click', (e) => {
    if (e.target.matches('.btn-suppr')) {
      e.target.closest('tr').remove();
    }
  });
});
```

```
        calculerTotalGeneral();
    }
});

// 3. Fonction utilitaire : recalcul total général
function calculerTotalGeneral() {
    const sousTotaux = tbody.querySelectorAll('.sous-total');
    // NodeList -> Array via spread operator pour reduce
    const total = [...sousTotaux].reduce((acc, el) => acc + parseFloat(el.textContent), 0);
    totalGeneralEl.value = total.toFixed(2); // <output> utilise .value
}

// Calcul initial au chargement
calculerTotalGeneral();
});
```

Conclusion : Code modulaire, performant (délégation), robuste (parseFloat + || 0, closest pour trouver la ligne), séparation HTML/JS. <output> utilisé pour le total (sémantique formulaire, participe à form.elements).

10.5 Gestion des événements

MÉTHODE Gérer les événements utilisateur (écoute, propagation, objet Event)

Objectif : Réagir aux actions utilisateur (clic, saisie, soumission, clavier, souris) avec `addEventListener` (standard W3C, permet multiples listeners sur même cible).

1. **Syntaxe :** `cible.addEventListener(type, fonctionGestionnaire, options?)`.

- **type :** chaîne (ex: 'click', 'submit', 'input', 'change', 'keydown', 'focus', 'blur', 'DOMContentLoaded').
- **fonctionGestionnaire :** fonction fléchée (e) => {...} ou nommée. Reçoit l'objet Event (ou sous-classe : `InputEvent`, `KeyboardEvent`, `SubmitEvent`, `MouseEvent`, `FocusEvent`).
- **options (objet) :** { once: true } (auto-suppression après 1er déclenchement), { passive: true } (scroll/touch performance, navigateur ne attend pas `preventDefault`), { capture: true } (écoute en phase de capture, défaut false = bouillonnement).

2. **Objets Event clés (disponibles dans e) :**

- **e.target :** élément origine de l'événement (celui qu'on a cliqué/saisi, ex : le <button> ou un dedans).
- **e.currentTarget :** élément sur lequel le listener est attaché (ex : le <tbody> en délégation). `this === e.currentTarget` (sauf fonction fléchée où `this` est lexical).
- **e.type :** nom de l'événement ('click', 'submit'...).

- `e.preventDefault()` : Annule le comportement par défaut du navigateur (ex : envoi formulaire `submit`, suivi lien `click` sur `<a>`, menu contextuel `contextmenu`, sélection texte).
- `e.stopPropagation()` : Arrête la remontée (bouillonnement) vers les parents.
`e.stopImmediatePropagation()` : arrête aussi les autres listeners sur la même cible.

3. Événements formulaires critiques :

- `submit` sur `<form>` : Seul point de contrôle fiable avant envoi. `e.preventDefault()` stoppe l'envoi HTTP classique. `form.checkValidity()` lance validation HTML5 native (met pseudo-classe `:invalid`). `form.reportValidity()` force affichage 1ère bulle navigateur.
- `input` sur `<input>/<textarea>/<select>` : Déclenché immédiatement à chaque modification (touche, coller, flèches spin, reconnaissance vocale). Idéal pour validation temps réel, calculs, aperçu.
- `change` : Déclenché à la perte de focus (`blur`) ET si valeur changée. Pour `<select>`, `<input type="checkbox/radio/date/file/color/range">`.
- `blur / focus` : Sortie/entrée dans le champ. Validation à la sortie (`blur`) souvent meilleure UX qu'à la frappe (`input`) pour éviter messages agressifs.

4. Délégation d'événement (Pattern professionnel) : Attacher un seul listener sur un parent commun (`<form>`, `<tbody>`, `<ul>`, `document`) et filtrer via `e.target.matches('selecteurCSS')`. Avantages : performance (N listeners → 1), gère automatiquement éléments ajoutés dynamiquement (AJAX/JS), code centralisé.

EXERCICE RÉSOLU 11 Validation complète formulaire « Contact Pro » au submit (délégation + `preventDefault`)

Énoncé : Formulaire ID `form-contact` : Nom (text, requis, min 2), Email (email, requis), Sujet (select, requis), Message (textarea, requis, min 10, max 500), Case « Newsletter » (checkbox). Écrire le JS qui : (1) Empêche l'envoi si invalide. (2) Affiche message d'erreur spécifique sous chaque champ (création dynamique `<span class="erreur">`). (3) Supprime les erreurs dès que l'utilisateur corrige (event `input`). (4) Si tout valide : affiche « Message envoyé! » dans `<div id="reponse">` et réinitialise le formulaire.

SOLUTION

Solution détaillée :

1. Sélection : `const form = document.getElementById('form-contact');` `const reponse = document.getElementById('reponse');`;
2. Listener `submit` sur `form` : `e.preventDefault()` systématique au début. Validation boucle sur `form.elements` (`HTMLFormControlsCollection`) ou

```
querySelectorAll('[required]');
```

3. **Fonction afficherErreur(champ, msg) : crée/maj span.erreur après champ (ou dans parent).** `champ.classList.add('invalide');` `champ.setAttribute('aria-invalid', 'true');` `champ.setAttribute('aria-describedby', 'id-erreur');`;
4. **Fonction effacerErreur(champ) : supprime span, retire classes/attributs ARIA.**
5. **Listener input sur form (délégation) :** `if (e.target.matches('[required]))` `effacerErreur(e.target);`;
6. **Logique validation : validity.valid (API Constraint Validation) + vérifs custom (min-length textarea).** `champ.validity.valueMissing`, `.typeMismatch`, `.tooShort`, `.tooLong`, `.patternMismatch`, `.badInput` (lettres dans number).
7. **Si form.checkValidity() true + custom OK :** `reponse.textContent = 'Message envoyé !';` `reponse.className = 'succes';` `form.reset();` (reset efface valeurs + erreurs visuelles si CSS lié à `invalid` mais nos spans dynamiques restent → prévoir nettoyage).

```
const form = document.getElementById('form-contact');
const reponse = document.getElementById('reponse');

// 1. Gestion soumission
form.addEventListener('submit', (e) => {
  e.preventDefault(); // Bloque envoi HTTP classique
  let valide = true;

  // Nettoyage erreurs précédentes
  form.querySelectorAll('.erreur').forEach(el => el.remove());
  form.querySelectorAll('.invalide').forEach(el => {
    el.classList.remove('invalide');
    el.removeAttribute('aria-invalid');
    el.removeAttribute('aria-describedby');
  });

  // Validation champs requis + contraintes HTML5 + custom
  const champsRequis = form.querySelectorAll('[required]');
  champsRequis.forEach(champ => {
    if (!champ.validity.valid) {
      valide = false;
      let msg = '';
      if (champ.validity.valueMissing) msg = 'Ce champ est obligatoire.';
      else if (champ.validity.typeMismatch) msg = 'Format invalide (${champ.type}).';
      else if (champ.validity.tooShort) msg = 'Minimum ${champ.minLength} caractères.';
      else if (champ.validity.tooLong) msg = 'Maximum ${champ.maxLength} caractères.';
      else if (champ.validity.patternMismatch) msg = 'Format non respecté.';
      else if (champ.validity.badInput) msg = 'Valeur incorrecte pour ce type.';
      afficherErreur(champ, msg);
    }
  });

  // Validation custom : Message min 10 caractères (HTML5 minlength gère mais message custom)
  const message = form.elements.message;
  if (message.value.length < 10) {
    valide = false;
  }
});
```

```
    afficherErreur(message, 'Le message doit contenir au moins 10 caractères.');
```

```
  }
```

```
  if (valide) {
```

```
    // Simulation envoi AJAX (fetch) ici...
```

```
    reponse.textContent = 'Votre message a bien été envoyé !';
```

```
    reponse.className = 'succes';
```

```
    reponse.setAttribute('role', 'alert'); // ARIA live region
```

```
    form.reset(); // Réinitialise valeurs
```

```
    // Nettoyage visuel erreurs résiduelles si reset ne suffit pas
```

```
    form.querySelectorAll('.erreur').forEach(el => el.remove());
```

```
  }
```

```
});
```

```
// 2. Effacement erreur à la saisie (délégation sur form)
```

```
form.addEventListener('input', (e) => {
```

```
  if (e.target.matches('[required]')) {
```

```
    effacerErreur(e.target);
```

```
  }
```

```
});
```

```
// 3. Fonctions utilitaires
```

```
function afficherErreur(champ, message) {
```

```
  champ.classList.add('invalide');
```

```
  champ.setAttribute('aria-invalid', 'true');
```

```
  const idErreur = `erreur-${champ.id}`;
```

```
  champ.setAttribute('aria-describedby', idErreur);
```

```
  const span = document.createElement('span');
```

```
  span.className = 'erreur';
```

```
  span.id = idErreur;
```

```
  span.textContent = message;
```

```
  // Insertion après le champ (ou dans parent si structure complexe)
```

```
  champ.parentNode.insertBefore(span, champ.nextSibling);
```

```
}
```

```
function effacerErreur(champ) {
```

```
  champ.classList.remove('invalide');
```

```
  champ.removeAttribute('aria-invalid');
```

```
  champ.removeAttribute('aria-describedby');
```

```
  const span = champ.parentNode.querySelector(`.erreur#erreur-${champ.id}`);
```

```
  if (span) span.remove();
```

```
}
```

Conclusion : Validation centralisée au submit (sécurité), feedback immédiat à la saisie (UX), accessibilité ARIA (`aria-invalid`, `aria-describedby`, `role="alert"`), utilisation de l'API `ValidityState` native (robuste, pas de regex manuelles pour email/required). `preventDefault()` indispensable.

10.6 Validation de la saisie côté client

MÉTHODE Valider la saisie côté client : Stratégie complète (HTML5 + JS + UX + Accessibilité)

Objectif : Mettre en place une validation robuste, utilisateur-friendly (UX) et accessible (WCAG), en combinant attributs HTML5 (1ère ligne), API Constraint Validation (moteur) et JS (logique métier, UX, messages custom, AJAX).

1. Côté HTML (Déclaratif, 1ère défense, fonctionne sans JS) : `required`, `type="email/number/url/tel/date"`, `min/max/step`, `minlength/maxlength`, `pattern="regex"` (avec `title` pour bulle navigateur), `accept (file)`, `multiple`. Avantage : Accessible nativement (bulles traduites), performance max, zéro code JS.
2. Côté JS : Configuration formulaire : `form.setAttribute('novalidate', '')` (désactive bulles auto + soumission auto si invalide) pour prendre la main totalement sur l'UX.
3. Côté JS : Validation au submit (Point de contrôle unique) :
 - (a) `e.preventDefault()`.
 - (b) `const valide = form.checkValidity()`; (lance validation HTML5 sur tous champs, met : `invalid` pseudo-classe CSS).
 - (c) Si `!valide` : `form.reportValidity()` (affiche 1ère bulle navigateur) OU boucle custom pour afficher toutes erreurs (meilleur UX).
 - (d) Vérifications métier impossibles en HTML5 :
 - Comparaison 2 champs (MDP / Confirmation MDP).
 - Unicité email / login (appel `fetch` AJAX asynchrone).
 - Calculs complexes (montant vs type abonnement, date cohérente).
 - Taille fichier réel / type MIME réel (File API `files[0].size`, `.type`).
 - Checksum, captcha, etc.
 - (e) Si tout OK : `fetch()` (AJAX) ou `form.submit()` (envoi classique).
4. Côté JS : Feedback temps réel (UX) : Listener input (ou change) en délégation sur `form`.
 - Si `champ.validity.valid` : `effacerErreur(champ)`, `champ.classList.add('valide')`.
 - Sinon : `afficherErreur(champ, messageCustom(champ))`.
5. Messages d'erreur custom (fonction) : Basé sur `champ.validity` (objet **ValidityState**) : `valueMissing`, `typeMismatch`, `patternMismatch`, `tooShort`, `tooLong`, `rangeUnderflow`, `rangeOverflow`, `stepMismatch`, `badInput` (ex : lettres dans number), `valid` (true si tout ok).
6. Accessibilité (WCAG 2.1) :
 - `aria-invalid="true/false"` sur champ.
 - `aria-describedby="id-message-erreur"` liant champ à son message d'erreur.

- `aria-live="polite"` ou `role="alert"` sur conteneur erreurs globales / succès.
 - Focus automatique sur 1er champ invalide (`champ.focus()`) après validation `submit` échouée.
 - Contraste couleurs erreurs (rouge sur blanc ≥ 4.5 :1), pas que couleur (icône + texte).
7. Sécurité (Règle d'or) : Jamais faire confiance au client. Validation côté serveur (PHP/-Node/Python/Java) obligatoire (injection SQL, XSS, contournement JS désactivé, outils DevTools, `curl/Postman` direct, modification `novalidate`, `required`, `pattern` dans le DOM). Au Bac, mentionner cette distinction vaut des points de savoir-être / rigueur professionnelle.

EXERCICE RÉSOLU 12 Validation formulaire « Inscription Newsletter MTN MoMo » avec contraintes métier

Énoncé : Formulaire : Email (requis, format email), Téléphone CM (requis, format 6xx xxx xxx ou 237 6xx xxx xxx), Type abonnement (radio : Journalier, Hebdo, Mensuel - requis), Montant FCFA (number, min 100, max 10000, step 50, requis), Case CGU (checkbox required). Contraintes métier : (1) Si « Journalier », montant ≥ 500 . (2) Téléphone doit commencer par 65, 67, 68, 69, 60, 61, 62, 63, 64, 66 (opérateurs CM). Écrire le JS complet (HTML5 + JS custom) avec messages d'erreur sous champs, focus 1er erreur, envoi `fetch` simulé si OK.

SOLUTION

Solution détaillée :

1. HTML : `<input type="tel" name="tel" pattern="(?:237\s)?6[0-9]{8}" placeholder="6xx xxx xxx">`
`<input type="number" name="montant" min="100" max="10000" step="50" required>`. Radios même `name="type"`. `<form novalidate>`.
2. JS : Sélection `form`. Listener `submit`.
3. `e.preventDefault().form.checkValidity()` (HTML5 de base).
4. Custom checks :
 - Tél : regex stricte `/^(?:237\s)?6[0-9]{8}$/` + vérif préfixe opérateur `/^6[0-9]/` (après nettoyage).
 - Montant vs Type : si `type.value === 'journalier'` et `montant < 500` → erreur sur montant.
5. Affichage erreurs : fonction `setError(champ, msg)` (créé `span`, ARIA, focus si 1er).
6. Succès : `fetch('/api/inscription', {method: 'POST', body: new FormData(form)})` puis UI succès.

```

const form = document.getElementById('form-momo');
const prefixOperateurs = ['65', '67', '68', '69', '60', '61', '62', '63', '64', '66'];

form.addEventListener('submit', async (e) => {
  e.preventDefault();
  clearAllErrors(); // Nettoyage visuel + ARIA

  // 1. Validation HTML5 native (required, type, min, max, step, pattern)
  if (!form.checkValidity()) {
    form.reportValidity(); // Affiche 1ère bulle navigateur (fallback)
    // On force notre affichage custom pour tous les invalides
    form.querySelectorAll('.invalid').forEach(champ => {
      if (champ.validity.valueMissing) setError(champ, 'Champ obligatoire. ');
      else if (champ.validity.typeMismatch) setError(champ, 'Format ${champ.type} invalide. ');
      else if (champ.validity.patternMismatch) setError(champ, 'Format non reconnu (ex : 6xx xxx
xxx). ');
      else if (champ.validity.rangeUnderflow) setError(champ, 'Minimum ${champ.min} FCFA. ');
      else if (champ.validity.rangeOverflow) setError(champ, 'Maximum ${champ.max} FCFA. ');
      else if (champ.validity.stepMismatch) setError(champ, 'Montant multiple de ${champ.step}
FCFA. ');
      else if (champ.validity.badInput) setError(champ, 'Valeur numérique invalide. ');
    });
    return;
  }

  // 2. Validations MÉTIER (impossibles en HTML5 pur)
  let valide = true;
  const tel = form.elements.tel;
  const montant = form.elements.montant;
  const typeAbo = form.querySelector('input[name="type"]:checked'); // Radio coché

  // Tel : nettoyage + vérif préfixe opérateur
  const telPropre = tel.value.replace(/s|237/g, '');
  if (!prefixOperateurs.some(p => telPropre.startsWith(p))) {
    setError(tel, 'Numéro non reconnu opérateur Cameroun (65, 67, 68, 69, 60-66). ');
    valide = false;
  }

  // Montant vs Type abonnement
  if (typeAbo.value === 'journalier' && montant.valueAsNumber < 500) {
    setError(montant, 'Pour un abonnement Journalier, le minimum est 500 FCFA. ');
    valide = false;
  }

  if (!valide) return; // Arrêt si erreur métier

  // 3. Succès : Envoi AJAX (Fetch API)
  try {
    const formData = new FormData(form);
    // Ajout tel nettoyé si besoin serveur
    formData.set('tel', telPropre);

    const response = await fetch('https://api.mtn.cm/v1/newsletter/subscribe', {
      method: 'POST',
      body: formData,
      headers: { 'Accept': 'application/json' }
    });
  }

```

```

    const data = await response.json();
    if (response.ok) {
        showSuccess('Inscription confirmée ! Vous recevrez les offres MoMo.');
```

form.reset();

```
    } else {
        setError(form.querySelector('button[type="submit"]'), data.message || 'Erreur serveur.');
```

}

```
    } catch (err) {
        setError(form.querySelector('button[type="submit"]'), 'Erreur réseau. Réessayez.');
```

}

```
});

// --- Utilitaires ---
function setError(champ, msg) {
    clearError(champ);
    champ.classList.add('invalide');
    champ.setAttribute('aria-invalid', 'true');
    const idErr = `err-${champ.name}`;
    champ.setAttribute('aria-describedby', idErr);
    const span = document.createElement('span');
    span.className = 'erreur';
    span.id = idErr;
    span.textContent = msg;
    champ.parentNode.appendChild(span); // ou insertBefore selon structure
    // Focus 1er erreur
    if (!form.querySelector('.invalide :focus')) champ.focus();
}

function clearError(champ) {
    champ.classList.remove('invalide');
    champ.removeAttribute('aria-invalid');
    champ.removeAttribute('aria-describedby');
    const err = document.getElementById(`err-${champ.name}`);
    if (err) err.remove();
}

function clearAllErrors() {
    form.querySelectorAll('.erreur').forEach(e => e.remove());
    form.querySelectorAll('.invalide').forEach(e => {
        e.classList.remove('invalide');
        e.removeAttribute('aria-invalid');
        e.removeAttribute('aria-describedby');
    });
}

function showSuccess(msg) {
    const div = document.getElementById('reponse') || document.createElement('div');
    div.id = 'reponse';
    div.className = 'succes';
    div.textContent = msg;
    div.setAttribute('role', 'alert'); // ARIA live region
    form.after(div);
}
```

Conclusion : Architecture pro : HTML5 pour 80% (required, type, pattern, min/max/step), JS pour 20% métier (inter-dépendance champs, regex complexe opérateur, AJAX).

`novalidate + checkValidity() + reportValidity() + custom`. Accessibilité ARIA soignée.
fetch avec `FormData` (standard, gère encodage multipart automatiquement).

ATTENTION Les 5 erreurs qui coûtent des points au Bac

1. Oublier `novalidate` sur `<form>` quand on gère la validation en JS. Conséquence : Le navigateur bloque l'envoi (bulles natives) avant que ton `addEventListener('submit')` ne s'exécute → ton code JS est ignoré, tu ne peux pas faire d'AJAX ni de messages custom. Règle : `<form novalidate>` systématique si JS validation.
2. Confondre `e.target` et `e.currentTarget` en délégation d'événement. Piège : Sur `tbody.addEventListener('click', ...)`, `e.target` est le `<button>` (ou `<span>` inside), `e.currentTarget` est le `<tbody>`. Pour trouver la ligne : `e.target.closest('tr')` (depuis cible) PAS `e.currentTarget.closest('tr')` (renvoie null).
3. Utiliser `innerHTML` avec des données utilisateur non échappées (Faille XSS). Exemple : `div.innerHTML = '<p>' + userInput + '</p>';`. Si `userInput = "<img src=x onerror=alert(1)>"` → script exécuté dans le navigateur de la victime. Règle : Toujours `textContent` pour texte, `setAttribute` pour attributs, `createElement + append` pour structure. `innerHTML` uniquement pour HTML de confiance (template statique, pas de données utilisateur).
4. Ne pas valider côté serveur (PHP) en se fiant seulement au JS/HTML5. Réalité : L'utilisateur désactive JS, modifie le DOM via DevTools, ou envoie une requête `curl/Postman` directe sur `action` du formulaire. La validation client est UX (confort), la validation serveur est SÉCURITÉ (intégrité données). Au Bac, mentionner « La validation côté client améliore l'UX mais ne dispense pas de la validation côté serveur (injection SQL, XSS, contournement) » vaut des points de savoir-être / rigueur.
5. Mauvaise gestion des types dans les calculs JS (concaténation au lieu d'addition). Classique : `const total = prix + quantite;` où `prix = "1500"` (string depuis `textContent` ou `value`) et `quantite = 2` → `"15002"`. Réflexe : `const total = Number(prix) + Number(quantite);` ou `+prix + +quantite` ou `parseFloat(prix)`. Toujours caster explicitement les valeurs lues du DOM (`.value`, `.textContent`) qui sont toujours des chaînes de caractères.

LESAVAIS-TU ? Le Cameroun numérique et le Web

Écosystème local : Le Cameroun compte plus de 10 millions d'internautes (2024). Les opérateurs MTN Cameroon et Orange Cameroun dominent l'accès mobile (3G/4G) et les services financiers mobiles (MTN MoMo, Orange Money) via APIs (REST/JSON, USSD). CAMTEL déploie la fibre optique nationale (backbone). L'administration dématématise : `impots.cm`

(DGI), cnps.cm, antic.cm (cybersécurité), cenadi.cm (identité numérique). Opportunités TI : Startups fintech (Ejara, Diool), edtech (Kwadra, Scholar), e-commerce (Jumia CM, Glo-telho), healthtech (GiftedMom). Compétences recherchées : Fullstack (PHP/Laravel, Node/Express, React/Vue), DevOps (Docker, CI/CD), Cybersécurité (pentest, audit), Data (Python, PowerBI). Histoire : HTML inventé par Tim Berners-Lee (CERN, 1991). JavaScript créé par Brendan Eich (Netscape, 1995, 10 jours). Standardisé ECMAScript (ES6/ES2015 = tournant moderne : `let/const`, arrow functions, classes, modules, promises). `fetch` API (standard moderne) remplace `XMLHttpRequest` (AJAX historique).

11 EXERCICES

11.1 Je vérifie mes connaissances

EXERCICE 7 QCM : Structure HTML et balises de formulaire

Parmi les affirmations suivantes, une seule est exacte. Laquelle ?

1. L'attribut `name` d'un `<input>` sert uniquement à identifier l'élément dans le CSS.
2. La balise `<textarea>` est une balise auto-fermante (ex. `<textarea />`).
3. L'attribut `id` doit être unique dans toute la page HTML, tandis que `name` peut être partagé (ex. boutons radio).
4. L'élément `<select>` ne peut contenir que des éléments `<option>` directs, pas de groupes.

EXERCICE 8 Vrai ou Faux : Tableaux HTML et accessibilité

Dire si les affirmations sont vraies ou fausses. Justifiez brièvement chaque réponse.

1. Les balises `<thead>`, `<tbody>` et `<tfoot>` sont obligatoires dans un tableau HTML valide.
2. L'attribut `scope="col"` sur une balise `<th>` améliore l'accessibilité pour les lecteurs d'écran.
3. La fusion de cellules horizontalement se fait avec l'attribut `rowspan`.
4. Un tableau de mise en page (layout) est une pratique recommandée en HTML5 moderne.

EXERCICE 9 Définitions et distinctions clés

Définissez précisément et distinguez les paires de concepts suivants :

1. Attribut `id` vs attribut `name` sur un champ de formulaire.
2. Balise `<th>` vs balise `<td>`.
3. Validation côté client (JavaScript) vs validation côté serveur (PHP, Node.js, etc.).
4. Propriété `value` vs propriété `innerText` (ou `textContent`) sur un élément DOM.

EXERCICE 10 Calcul direct : Sélecteurs DOM et événements

Soit le fragment HTML suivant :

```
<form id="inscription">
  <input type="text" id="nom" name="nom" value="Ngonon">
  <input type="email" class="contact" name="email" value="ngono@email.cm">
  <input type="tel" class="contact" name="tel" value="6XX XX XX XX">
  <button type="submit" id="btnValider">Valider</button>
</form>
```

Écrivez l'instruction JavaScript exacte (une ligne par question) pour :

1. Sélectionner l'élément `<form>` et le stocker dans une variable `form`.
2. Récupérer la valeur actuelle du champ `nom`.
3. Sélectionner tous les éléments ayant la classe `contact` sous forme de liste (NodeList).
4. Ajouter un écouteur d'événement `submit` sur le formulaire qui appelle une fonction `verifier`.

11.2 Je m'entraîne

EXERCICE 11 Construction d'un formulaire complet : Inscription à un concours

Écrivez le code HTML complet (balises `<form>` incluses) pour un formulaire d'inscription au concours d'entrée à l'ENSET de Douala. Le formulaire doit contenir :

- Un champ texte pour le Nom complet (obligatoire, `id="nom"`).
- Un champ email pour l'Adresse électronique (obligatoire, type adapté).
- Une liste déroulante `<select>` pour le Choix de la filière (options : "Génie Civil", "Génie Électrique", "Génie Mécanique", "Informatique"), avec une première option "— Choisir —" désactivée et sélectionnée par défaut.
- Un groupe de deux boutons radio pour le Type de baccalauréat (F / D), même `name`, `id` distincts.
- Une zone de texte `<textarea>` pour la Lettre de motivation (5 lignes visibles).
- Un bouton `submit` "Soumettre ma candidature" et un bouton `reset` "Annuler".

Chaque champ doit être associé à un `<label>` via l'attribut `for`.

EXERCICE 12 Tableau de notes avec fusion et structure sémantique

Réalisez le code HTML d'un tableau affichant les notes de 3 élèves de Terminale TI au Lycée

de Nkolbisson pour le 1er trimestre.

- En-tête (<thead>) : "Élève", "Mathématiques", "Physique", "Informatique", "Moyenne".
- Corps (<tbody>) : 3 lignes d'élèves avec des notes numériques (sur 20) et une moyenne calculée (vous mettez une valeur fictive).
- Pied (<tfoot>) : Une seule ligne fusionnée sur 5 colonnes (colspan="5") indiquant : "Moyenne de la classe : 14,5 / 20".
- La première cellule de chaque ligne de données (nom de l'élève) doit être une <th scope="row">.

EXERCICE 13 Validation JavaScript : Contrôle d'un champ "Âge"

On dispose du formulaire HTML suivant (incomplet) :

```
<form id="formAge">
  <label for="age">Âge du candidat (15-25 ans) :</label>
  <input type="text" id="age" name="age">
  <span id="msgErreur" style="color : red; display : none;"></span>
  <button type="submit">Vérifier</button>
</form>
```

Écrivez le code JavaScript (à placer dans une balise <script> en fin de <body>) qui :

1. Sélectionne le formulaire, le champ âge et la zone de message.
2. Ajoute un écouteur sur l'événement submit.
3. Dans le gestionnaire :
 - Empêche l'envoi du formulaire (preventDefault).
 - Récupère la valeur, la convertit en nombre.
 - Vérifie : champ non vide, est un nombre (isNaN), compris entre 15 et 25 inclus.
 - Si invalide : affiche un message d'erreur précis dans #msgErreur (rend visible).
 - Si valide : cache le message d'erreur, affiche "Âge valide : X ans" dans la console, et soumet réellement le formulaire (indice : form.submit()).

EXERCICE 14 Débogage : 5 erreurs dans un code HTML/JS mélange

Le code suivant comporte exactement 5 erreurs (syntaxe HTML, logique JS, DOM, événements). Identifiez-les, expliquez pourquoi c'est une erreur, et donnez le code corrigé.

```
<!DOCTYPE html>
```

```
<html lang="fr">
<head><meta charset="UTF-8"><title>Test</title></head>
<body>
  <form id="login">
    <input type="text" id="user" name="login" placeholder="Identifiant">
    <input type="password" id="pass" name="pwd">
    <button id="btnConnect">Connexion</button>
  </form>
  <div id="resultat"></div>

  <script>
    const btn = document.getElementById('btnConnexion'); // Erreur 1 ?
    const inputUser = document.getElementById('user');

    btn.addEventListener('click', verifier()); // Erreur 2 ?

    function verifier() {
      let val = inputUser.value;
      if (val.length < 3) {
        document.getElementById('resultat').innerHTML = "Trop court";
        return false; // Erreur 3 ?
      }
      // Simulation AJAX
      alert("Bienvenue " + val);
    }

    // Tentative de soumission formulaire
    document.getElementById('login').addEventListener('submit', function(e) {
      verifier(); // Erreur 4 ?
    });
  </script>
</body>
</html>
```

11.3 Je me prépare au Bac

EXERCICE 15 Sujet Bac Type : Formulaire de commande "Jumia Cameroon"

Dans le cadre du développement d'une version allégée du site Jumia Cameroon pour les zones à faible débit (régions du Nord, Extrême-Nord), on souhaite créer une page de commande simple, sans framework, valide HTML5 et accessible.

Le formulaire doit permettre de commander un smartphone. Il contient les champs suivants :

1. Modèle : liste déroulante (<select>) avec 3 options : "Tecno Spark 20", "Infinix Hot 30", "Itel A70". La première option est un placeholder "— Choisir un modèle —" (désactivée, sélectionnée, valeur vide).
2. Quantité : champ numérique (type="number"), min=1, max=10, valeur par défaut 1.
3. Moyen de paiement : deux boutons radio (même name="paiement") : "MTN MoMo" (va-

leur_{momo}) et "Orange Money" (valeur_{om}). "MTN MoMo" coché par défaut.

4. Numéro de téléphone : champ `tel` (obligatoire), placeholder "+237 6XX XX XX XX".
5. Adresse de livraison : `<textarea>` (4 lignes, 50 colonnes), obligatoire.
6. Bouton "Passer la commande" (`submit`) et "Réinitialiser" (`reset`).

Questions :

1. Écrivez le code HTML complet et strictement valide de ce formulaire (balise `<form>` avec `action="traitement.php"` et `method="POST"`, `<fieldset>` et `<legend>` pour regrouper, `<label>` correctement liés via `for/id`, attributs HTML5 de validation `required` là où c'est nécessaire).
2. Expliquez pourquoi l'attribut `name` est indispensable sur chaque contrôle de saisie (`input`, `select`, `textarea`) pour que le script `traitement.php` reçoive les données, alors que l'attribut `id` ne suffit pas.
3. On souhaite ajouter une validation côté client en JavaScript : le numéro de téléphone doit commencer par +237 suivi de 9 chiffres (ex. +2376XXXXXXX). Donnez l'expression rationnelle (RegExp) JavaScript correspondante.

EXERCICE 16 Sujet Bac Type : Tableau de résultats Baccalauréat et Script d'affichage

L'Office du Baccalauréat du Cameroun (OBC) publie un extrait des résultats de la session de juin pour le centre "Lycee Bilingue de Bafoussam". On vous donne les données brutes pour 4 candidats :

N° Table	Nom	Maths	Physique	Moyenne
24T1001	FOPA Pierre	16	14	15,00
24T1002	TCHUISSEU Grace	18	17	17,50
24T1003	NGUEFACK Bruno	09	11	10,00
24T1004	KAMGAING Marie	14	13	13,50

Questions :

1. Écrivez le code HTML complet d'un tableau sémantique (`<table>`, `<caption>`, `<thead>`, `<tbody>`, `<tfoot>`) affichant ces données.
2. Dans `<tfoot>`, ajoutez une ligne unique fusionnée sur 5 colonnes (`colspan="5"`) : "Session Juin 2024 – Centre : Lycée Bilingue Bafoussam – 4 candidats présentés".
3. Les moyennes sont affichées avec 2 décimales. La moyenne de la classe (moyenne des 4 moyennes) est 14,00. Ajoutez une deuxième ligne dans `<tfoot>` (non fusionnée) avec : 1^{re} cellule "Moyenne classe" (fusionnée sur 4 colonnes `colspan="4"`), 2^{de} cellule "14,00".

4. Écrivez une fonction JavaScript `colorierMoyennes()` qui, au chargement de la page (`DOMContentLoaded`), parcourt les lignes du `<tbody>`, récupère la moyenne (dernière cellule de chaque ligne), et met la ligne en fond vert clair (`style.backgroundColor = "#d4edda"`) si la moyenne ≥ 10 , sinon en fond rouge clair (`"#f8d7da"`). Indice : utilisez `querySelectorAll('tbody tr')`.

EXERCICE 17 Sujet Bac Type : Gestionnaire d'inscription complet avec cumul d'erreurs

On développe la page d'inscription en ligne pour le Concours National d'Entrée aux Écoles Normales Supérieures (ENES). Le formulaire HTML (fourni ci-dessous) est déjà en place. Vous devez écrire uniquement le code JavaScript (dans une balise `<script>` ou fichier externe) qui gère la validation complète à la soumission.

```
<form id="formENES" action="inscrire.php" method="POST" novalidate>
  <div class="champ">
    <label for="nom">Nom et Prénom :</label>
    <input type="text" id="nom" name="nom" required>
    <span class="erreur" id="errNom"></span>
  </div>
  <div class="champ">
    <label for="etab">Établissement d'origine :</label>
    <select id="etab" name="etab" required>
      <option value="">-- Sélectionner --</option>
      <option value="lycee_leco">Lycee Leclerc (Yaoundé)</option>
      <option value="lycee_bilingue_dla">Lycee Bilingue (Douala)</option>
      <option value="autre">Autre</option>
    </select>
    <span class="erreur" id="errEtab"></span>
  </div>
  <div class="champ">
    <fieldset>
      <legend>Épreuve choisie :</legend>
      <label><input type="radio" name="epreuve" value="maths" required> Mathématiques</label>
      <label><input type="radio" name="epreuve" value="pc"> Physique-Chimie</label>
      <label><input type="radio" name="epreuve" value="svt"> SVT</label>
    </fieldset>
    <span class="erreur" id="errEpreuve"></span>
  </div>
  <div class="champ">
    <label for="tel">WhatsApp (pour convocation) :</label>
    <input type="tel" id="tel" name="tel" required placeholder="+237 6XX XX XX XX">
    <span class="erreur" id="errTel"></span>
  </div>
  <button type="submit" id="btnInscrire">Valider mon inscription</button>
  <div id="bilan" style="margin-top:1rem; font-weight:bold;"></div>
</form>
```

Règles de validation (côté client) :

1. Nom : non vide après `trim()`, longueur ≥ 3 caractères.

2. Établissement : valeur non vide (différente de "").
3. Épreuve : un bouton radio doit être coché (l'attribut `required` sur l'un d'eux suffit en HTML5, mais en JS il faut vérifier `document.querySelector('input[name="epreuve"]:checked')`).
4. Téléphone : doit correspondre au format camerounais strict : +237 suivi de 9 chiffres (ex. +237677889900). Utilisez une RegExp.

Comportement attendu du script :

1. Au `submit` : empêcher l'envoi par défaut (`preventDefault`).
2. Réinitialiser l'affichage des erreurs (vider les `<span class="erreur">`, enlever une classe `invalid` sur les `inputs` si vous en mettez une).
3. Vérifier tous les champs, cumuler les messages d'erreur dans les `<span>` correspondants (ex. `errNom.textContent = "Nom trop court"`).
4. Si au moins une erreur : afficher dans `#bilan` : "Formulaire invalide : X erreur(s) détectée(s)." (en rouge).
5. Si aucune erreur : afficher dans `#bilan` : "Inscription validée pour [Nom] - Épreuve : [Épreuve]." (en vert), puis soumettre le formulaire programmatically (`form.submit()`).

12 CORRIGÉS DES EXERCICES

13 EXERCICES CORRIGÉS — RAPPELS DE PROGRAMMATION WEB : HTML ET JAVASCRIPT

CORRIGÉ Exercice 1 — QCM : Structure HTML et balises de formulaire

Réponse exacte : c.

- a. Faux. L'attribut `name` sert à nommer la donnée envoyée au serveur lors de la soumission du formulaire (c'est la clé de la paire `name=value` reçue par le script serveur). Pour le CSS, c'est `id` ou `class` qui sert de sélecteur.
- b. Faux. `<textarea>` possède un contenu textuel (la valeur par défaut) : elle exige une balise fermante `</textarea>`. Écrire `<textarea />` est une erreur de syntaxe HTML5.
- c. Vrai. L'`id` identifie un seul élément dans toute la page (unicité obligatoire). Le `name` peut être partagé : c'est précisément ce qui permet de regrouper des boutons radio (un seul choix possible par groupe de même `name`).
- d. Faux. Un `<select>` peut contenir des éléments `<optgroup>` qui regroupent des `<option>` par catégories.

CORRIGÉ Exercice 2 — Vrai ou Faux : Tableaux HTML et accessibilité

1. Faux. Seules `<table>` et `<tr>/<td>` sont indispensables. `<thead>`, `<tbody>` et `<tfoot>` sont optionnelles mais fortement recommandées pour la sémantique, l'accessibilité et le stylage.
2. Vrai. `scope="col"` indique explicitement aux technologies d'assistance (lecteurs d'écran) que la cellule d'en-tête s'applique à toute la colonne, ce qui facilite la navigation vocale dans le tableau.
3. Faux. La fusion horizontale (sur plusieurs colonnes) se fait avec `colspan`. L'attribut `rowspan` fusionne verticalement (sur plusieurs lignes).
4. Faux. Utiliser un tableau pour la mise en page est une pratique obsolète et déconseillée : les tableaux sont réservés aux données tabulaires. La mise en page se fait avec CSS (Flexbox, Grid).

CORRIGÉ Exercice 3 — Définitions et distinctions clés

1. **id vs name.** L'id est un identifiant unique dans la page : il sert à cibler l'élément en CSS (#id), en JavaScript (getElementById) et à lier un <label for="...">. Le name nomme la donnée transmise au serveur à la soumission ; il peut être partagé (groupe de radios). Un champ sans name n'est jamais envoyé au serveur.
2. **<th> vs <td>.** <th> (table header) désigne une cellule d'en-tête : gras et centrée par défaut, elle porte une valeur sémantique (avec scope). <td> (table data) désigne une cellule de données ordinaire.
3. **Validation client vs serveur.** La validation côté client (JavaScript, attributs HTML5) s'exécute dans le navigateur : elle est rapide et améliore l'ergonomie, mais elle est contournable (désactivation de JS, requêtes forgées). La validation côté serveur (PHP, etc.) s'exécute après réception des données : elle est incontournable et seule garante de la sécurité et de l'intégrité des données. Les deux sont complémentaires et la validation serveur est obligatoire.
4. **value vs innerText/textContent.** value s'applique aux contrôles de formulaire (input, select, textarea) et donne la valeur saisie ou choisie. textContent (ou innerText) s'applique aux éléments de contenu (div, span, p...) et donne le texte qu'ils contiennent. Utiliser value sur un <div> renvoie undefined.

CORRIGÉ Exercice 4 — Calcul direct : Sélecteurs DOM et événements

```
// 1. Selection du formulaire par son id
const form = document.getElementById('inscription');

// 2. Valeur actuelle du champ nom (renvoie "Ngon")
const nom = document.getElementById('nom').value;

// 3. Tous les elements de classe "contact" : NodeList de 2 elements
const contacts = document.querySelectorAll('.contact');

// 4. Ecouteur d'evenement submit appelant la fonction verifier
form.addEventListener('submit', verifier);
```

Points de vigilance :

- On passe la référence de la fonction (verifier), et non son appel (verifier()), sinon elle s'exécuterait immédiatement au chargement.
- querySelectorAll retourne toujours une NodeList, même vide ou à un seul élément.
- Alternative acceptée pour la question 1 : document.querySelector('#inscription').

CORRIGÉ

Exercice 5 — Construction d'un formulaire complet : Inscription à un concours

```

<form action="inscription.php" method="POST">
  <p>
    <label for="nom">Nom complet :</label>
    <input type="text" id="nom" name="nom" required>
  </p>
  <p>
    <label for="email">Adresse electronique :</label>
    <input type="email" id="email" name="email" required>
  </p>
  <p>
    <label for="filiere">Choix de la filiere :</label>
    <select id="filiere" name="filiere" required>
      <option value="" disabled selected>--- Choisir ---</option>
      <option value="gc">Genie Civil</option>
      <option value="ge">Genie Electrique</option>
      <option value="gm">Genie Mecanique</option>
      <option value="info">Informatique</option>
    </select>
  </p>
  <fieldset>
    <legend>Type de baccalaureat :</legend>
    <label for="bacF">Serie F</label>
    <input type="radio" id="bacF" name="bac" value="F" required>
    <label for="bacD">Serie D</label>
    <input type="radio" id="bacD" name="bac" value="D">
  </fieldset>
  <p>
    <label for="motivation">Lettre de motivation :</label><br>
    <textarea id="motivation" name="motivation" rows="5" cols="40"></textarea>
  </p>
  <p>
    <button type="submit">Soumettre ma candidature</button>
    <button type="reset">Annuler</button>
  </p>
</form>

```

Points de vigilance :

- Les deux radios partagent `name="bac"` (choix exclusif) mais ont des `id` distincts (`bacF`, `bacD`) pour que chaque `<label for>` pointe vers le bon bouton.
- L'option placeholder porte à la fois `disabled` et `selected` : elle s'affiche par défaut mais ne peut pas être choisie ; combinée à `required` sur le `<select>`, elle force un vrai choix.
- Chaque champ possède un `name` : sans lui, la donnée ne partirait jamais vers `inscription.php`.

CORRIGÉ

Exercice 6 — Tableau de notes avec fusion et structure sémantique

```

<table border="1">
  <caption>Notes du 1er trimestre – Terminale TI – Lycee de Nkolbisson</caption>
  <thead>
    <tr>
      <th scope="col">Eleve</th>
      <th scope="col">Mathematiques</th>
      <th scope="col">Physique</th>
      <th scope="col">Informatique</th>
      <th scope="col">Moyenne</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">ABENA Claire</th>
      <td>15</td>
      <td>13</td>
      <td>17</td>
      <td>15,00</td>
    </tr>
    <tr>
      <th scope="row">MBALLA Junior</th>
      <td>12</td>
      <td>14</td>
      <td>16</td>
      <td>14,00</td>
    </tr>
    <tr>
      <th scope="row">ETOUNDI Sandrine</th>
      <td>16</td>
      <td>15</td>
      <td>18</td>
      <td>16,33</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <td colspan="5">Moyenne de la classe : 15,11 / 20</td>
    </tr>
  </tfoot>
</table>

```

Vérification des moyennes (sur 20) :

$$\text{ABENA : } \frac{15 + 13 + 17}{3} = \frac{45}{3} = 15,00$$

$$\text{MBALLA : } \frac{12 + 14 + 16}{3} = \frac{42}{3} = 14,00$$

$$\text{ETOUNDI : } \frac{16 + 15 + 18}{3} = \frac{49}{3} \approx 16,33$$

$$\text{Moyenne de classe : } \frac{15,00 + 14,00 + 16,33}{3} \approx 15,11$$

Points de vigilance : la ligne du <tfoot> ne contient qu'une seule cellule (colspan="5") : ajouter d'autres <td> dans cette ligne produirait un tableau invalide (6 colonnes effectives). Le nom de l'élève est bien un <th scope="row"> car il joue le rôle d'en-tête de sa ligne.

CORRIGÉ Exercice 7 — Validation JavaScript : Contrôle d'un champ « Âge »

```
<script>
// 1. Selection des elements
const form = document.getElementById('formAge');
const champAge = document.getElementById('age');
const msgErreur = document.getElementById('msgErreur');

// 2. Ecouteur sur la soumission
form.addEventListener('submit', function(e) {
  // Empeche l'envoi automatique
  e.preventDefault();

  // Recuperation et conversion
  const saisie = champAge.value.trim();
  const age = Number(saisie);

  // 3. Verifications en cascade
  if (saisie === '') {
    msgErreur.textContent = "Le champ age est obligatoire.";
    msgErreur.style.display = 'inline';
  } else if (isNaN(age)) {
    msgErreur.textContent = "L'age doit etre un nombre.";
    msgErreur.style.display = 'inline';
  } else if (age < 15 || age > 25) {
    msgErreur.textContent = "L'age doit etre compris entre 15 et 25 ans.";
    msgErreur.style.display = 'inline';
  } else {
    // Cas valide : on cache l'erreur et on soumet reellement
    msgErreur.style.display = 'none';
    console.log("Age valide : " + age + " ans");
    form.submit();
  }
});
</script>
```

Points de vigilance :

- `e.preventDefault()` bloque l'envoi avant toute vérification ; sans lui, la page se rechargerait même en cas d'erreur.
- `form.submit()` soumet le formulaire sans déclencher à nouveau l'événement `submit` : pas de boucle infinie.
- On teste d'abord le champ vide avant `isNaN`, car `Number('')` vaut 0, qui passerait le test numérique à tort.
- `trim()` élimine les espaces parasites saisis par l'utilisateur.

CORRIGÉ Exercice 8 — Débogage : 5 erreurs dans un code HTML/JS mélange

Erreur 1 — Identifiant inexistant. `getElementById('btnConnexion')` retourne `null` car le

bouton a pour id la valeur `btnConnect`. L'instruction suivante (`addEventListener` sur `null`) lève alors une `TypeError`. **Correction** : `document.getElementById('btnConnect')`.

Erreur 2 — Appel au lieu de référence. `addEventListener('click', verifier())` exécute `verifier` immédiatement au chargement et enregistre sa valeur de retour (`undefined`) comme gestionnaire. **Correction** : passer la référence `verifier` sans parenthèses.

Erreur 3 — `return false` inefficace. Dans un gestionnaire attaché par `addEventListener`, `return false` n'annule pas le comportement par défaut (contrairement aux anciens gestionnaires `onclick`). Il faut utiliser `e.preventDefault()`.

Erreur 4 — Soumission non bloquée. Dans l'écouteur `submit`, on appelle `verifier()` mais on ne fait jamais `e.preventDefault()` : le formulaire est donc envoyé (et la page rechargée) quoiqu'il arrive, rendant la validation inutile. **Correction** : appeler `e.preventDefault()` puis `verifier()`.

Erreur 5 — Bouton de type `submit` implicite. Le `<button id="btnConnect">` n'a pas d'attribut `type` : dans un formulaire, un bouton sans `type` est `submit` par défaut. Le clic déclenche donc à la fois l'événement `click` et la soumission du formulaire (double comportement). **Correction** : `type="button"`.

Code corrigé :

```
<form id="login">
  <input type="text" id="user" name="login" placeholder="Identifiant">
  <input type="password" id="pass" name="pwd">
  <button type="button" id="btnConnect">Connexion</button>
</form>
<div id="resultat"></div>

<script>
  const form = document.getElementById('login');
  const btn = document.getElementById('btnConnect');
  const inputUser = document.getElementById('user');

  btn.addEventListener('click', verifier);

  form.addEventListener('submit', function (e) {
    e.preventDefault();
    verifier();
  });

  function verifier() {
    let val = inputUser.value;
    if (val.length < 3) {
      document.getElementById('resultat').textContent = "Trop court";
      return;
    }
    alert("Bienvenue " + val);
    form.submit(); // Soumission programmatique réelle en cas de succès
  }
</script>
```

Remarque : on préfère `textContent` à `innerHTML` pour afficher un simple message : cela évite toute injection de balises (bonne pratique de sécurité). La fonction `verifier` appelle désormais `form.submit()` pour envoyer les données au serveur après validation réussie.

CORRIGÉ

Exercice 9 — Sujet Bac Type : Formulaire de commande « Jumia Cameroon »

Question 1 — Code HTML complet et valide :

```
<form action="traitement.php" method="POST">
  <fieldset>
    <legend>Commande de smartphone – Jumia Cameroon</legend>

    <p>
      <label for="modele">Modele :</label>
      <select id="modele" name="modele" required>
        <option value="" disabled selected>--- Choisir un modele ---</option>
        <option value="tecno_spark20">Tecno Spark 20</option>
        <option value="infinix_hot30">Infinix Hot 30</option>
        <option value="itel_a70">Itel A70</option>
      </select>
    </p>

    <p>
      <label for="quantite">Quantite :</label>
      <input type="number" id="quantite" name="quantite"
        min="1" max="10" value="1" required>
    </p>

    <fieldset>
      <legend>Moyen de paiement :</legend>
      <input type="radio" id="momo" name="paiement" value="momo" checked>
      <label for="momo">MTN MoMo</label>
      <input type="radio" id="om" name="paiement" value="om">
      <label for="om">Orange Money</label>
    </fieldset>

    <p>
      <label for="tel">Numero de telephone :</label>
      <input type="tel" id="tel" name="tel"
        placeholder="+237 6XX XXX XX" required>
    </p>

    <p>
      <label for="adresse">Adresse de livraison :</label><br>
      <textarea id="adresse" name="adresse" rows="4" cols="50" required></textarea>
    </p>

    <p>
      <button type="submit">Passer la commande</button>
      <button type="reset">Reinitialiser</button>
    </p>
  </fieldset>
</form>
```

Question 2 — Pourquoi `name` est indispensable. Lors de la soumission, le navigateur construit les données envoyées sous forme de paires `nom=valeur` en utilisant exclusivement l'attribut `name` de chaque contrôle. Le script `traitement.php` les lit alors via `$_POST['modele']`, `$_POST['tel']`, etc. L'attribut `id` ne sert qu'en local (CSS, JavaScript, liaison `<label for>`) et n'est jamais transmis au serveur. Un champ dépourvu de `name` est donc purement ignoré lors de l'envoi, même s'il possède un `id` et une valeur saisie.

Question 3 — Expression rationnelle du téléphone :

```
const regexTel = /^\\+237[0-9]{9}$/;
// Exemple de test : regexTel.test('+237677889900') === true
```

L'ancrage `^` impose le début de chaîne, `\\+` désigne le caractère littéral `+` (échappé car `+` est un méta-caractère), `[0-9]{9}` exige exactement 9 chiffres, et `$` impose la fin de chaîne (aucun caractère supplémentaire toléré).

Barème indicatif (sur 10) : structure `form/fieldset/legend` correcte : 2 pts ; `select` avec placeholder désactivé-sélectionné : 1,5 pt ; radios groupés avec `checked` : 1,5 pt ; `label/for` corrects : 1 pt ; attributs `required`, `min/max` : 1 pt ; explication de `name` : 2 pts ; RegExp exacte : 1 pt.

Points de vigilance : exiger la mention explicite que `id` n'est pas transmis au serveur ; vérifier que le placeholder du `select` a `value=""` (sinon `required` ne bloque rien) ; la RegExp doit contenir les ancres `^` et `$` pour être « stricte ».

CORRIGÉ Exercice 10 — Sujet Bac Type : Tableau de résultats Baccalauréat et script d'affichage

Questions 1, 2 et 3 — Code HTML complet :

```
<table border="1">
  <caption>Resultats Baccalaureat – Session Juin 2024 –
    Lycee Bilingue de Bafoussam</caption>
  <thead>
    <tr>
      <th scope="col">N° Table</th>
      <th scope="col">Nom</th>
      <th scope="col">Maths</th>
      <th scope="col">Physique</th>
      <th scope="col">Moyenne</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>24TI001</td><td>FOPA Pierre</td>
      <td>16</td><td>14</td><td>15,00</td>
    </tr>
    <tr>
      <td>24TI002</td><td>TCHUISSEU Grace</td>
      <td>18</td><td>17</td><td>17,50</td>
    </tr>
    <tr>
      <td>24TI003</td><td>NGUEFACK Bruno</td>
      <td>09</td><td>11</td><td>10,00</td>
    </tr>
    <tr>
      <td>24TI004</td><td>KAMGAING Marie</td>
      <td>14</td><td>13</td><td>13,50</td>
    </tr>
  </tbody>
```

```

<tfoot>
  <tr>
    <td colspan="5">Session Juin 2024 – Centre : Lycee Bilingue
      Bafoussam – 4 candidats presentes</td>
  </tr>
  <tr>
    <td colspan="4">Moyenne classe</td>
    <td>14,00</td>
  </tr>
</tfoot>
</table>

```

Vérification de la moyenne de classe :

$$\frac{15,00 + 17,50 + 10,00 + 13,50}{4} = \frac{56,00}{4} = 14,00$$

Le compte des colonnes est respecté : ligne 1 du tfoot : 5 colonnes (1× colspan="5"); ligne 2 : 4 + 1 = 5 colonnes.

Question 4 — Fonction JavaScript :

```

function colorierMoyennes() {
  const lignes = document.querySelectorAll('tbody tr');
  lignes.forEach(function (ligne) {
    // Dernière cellule de la ligne = moyenne
    const cellules = ligne.querySelectorAll('td');
    const texteMoy = cellules[cellules.length - 1].textContent;
    // Conversion : remplacer la virgule par un point
    const moyenne = parseFloat(texteMoy.replace(',', '.'));
    if (moyenne >= 10) {
      ligne.style.backgroundColor = '#d4edda'; // vert clair
    } else {
      ligne.style.backgroundColor = '#f8d7da'; // rouge clair
    }
  });
}

document.addEventListener('DOMContentLoaded', colorierMoyennes);

```

Avec les données fournies, les 4 lignes seront colorées en vert clair (toutes les moyennes sont ≥ 10).

Barème indicatif (sur 10) : structure sémantique complète avec caption : 3 pts; tfoot à deux lignes avec fusions correctes : 2,5 pts; fonction de parcours du tbody : 2 pts; conversion virgule/point et comparaison : 1,5 pt; attachement à DOMContentLoaded : 1 pt. Points de vigilance : le piège majeur est la virgule décimale française : parseFloat('15,00') renvoie 15 (la partie après la virgule est ignorée) et parseFloat sur certaines chaînes peut renvoyer NaN; le remplacement replace(',', '.') est donc attendu dans la justification. Autre point : le script doit s'exécuter après la construction du DOM, d'où DOMContentLoaded (ou un <script> en fin de <body>).

CORRIGÉ Exercice 11 — Sujet Bac Type : Gestionnaire d'inscription complet avec cumul d'erreurs

Code JavaScript complet :

```
// Selection des elements
const form = document.getElementById('formENES');
const champNom = document.getElementById('nom');
const champEtab = document.getElementById('etab');
const champTel = document.getElementById('tel');
const bilan = document.getElementById('bilan');

// Expression rationnelle : +237 suivi de 9 chiffres exactement
const regexTel = /^+237[0-9]{9}$/;

form.addEventListener('submit', function (e) {
  // 1. Empêcher l'envoi par défaut
  e.preventDefault();

  // 2. Reinitialiser l'affichage des erreurs
  const spansErreur = document.querySelectorAll('.erreur');
  spansErreur.forEach(function (span) {
    span.textContent = '';
  });
  bilan.textContent = '';

  let nbErreurs = 0;

  // 3a. Validation du nom : non vide et au moins 3 caracteres
  const nom = champNom.value.trim();
  if (nom === '') {
    document.getElementById('errNom').textContent =
      "Le nom est obligatoire.";
    nbErreurs++;
  } else if (nom.length < 3) {
    document.getElementById('errNom').textContent =
      "Nom trop court (3 caracteres minimum).";
    nbErreurs++;
  }

  // 3b. Validation de l'établissement : valeur non vide
  if (champEtab.value === '') {
    document.getElementById('errEtab').textContent =
      "Veuillez selectionner un etablissement.";
    nbErreurs++;
  }

  // 3c. Validation de l'épreuve : un radio coché
  const epreuveCochee =
    document.querySelector('input[name="epreuve"]:checked');
  if (epreuveCochee === null) {
    document.getElementById('errEpreuve').textContent =
      "Veuillez choisir une epreuve.";
    nbErreurs++;
  }

  // 3d. Validation du telephone : format +237XXXXXXXXX
```

```
const tel = champTel.value.trim();
if (!regexTel.test(tel)) {
    document.getElementById('errTel').textContent =
        "Format attendu : +237 suivi de 9 chiffres (ex. +237677889900).";
    nbErreurs++;
}

// 4. et 5. Bilan et decision finale
if (nbErreurs > 0) {
    bilan.style.color = 'red';
    bilan.textContent = "Formulaire invalide : " + nbErreurs +
        " erreur(s) detectee(s).";
} else {
    bilan.style.color = 'green';
    bilan.textContent = "Inscription validee pour " + nom +
        " - Epreuve : " + epreuveCochee.value + ".";
    form.submit(); // Soumission programmatique reelle
}
});
```

Explications des points techniques :

- **Cumul des erreurs** : contrairement à une validation en cascade avec `return` au premier échec, on teste chaque champ indépendamment et on incrémente le compteur `nbErreurs` : l'utilisateur voit toutes ses erreurs en une seule fois.
- **Réinitialisation préalable** : vider tous les `.erreur` au début du questionnaire évite qu'un message d'erreur corrigé ne persiste à l'écran.
- **Radios** : le sélecteur `input[name="epreuve"]:checked` cible le bouton coché du groupe ; s'il n'y en a aucun, `querySelector` renvoie `null`.
- `form.submit()` : soumet le formulaire vers `inscrire.php` sans redéclencher l'événement `submit` (aucune boucle infinie).
- **Attribut `novalidate`** : présent sur le `<form>`, il désactive la validation native HTML5 afin que ce soit bien notre script JavaScript qui contrôle tout (exigence de l'énoncé).

Barème indicatif (sur 10) : `preventDefault` et réinitialisation des erreurs : 2 pts ; validation du nom avec `trim()` et longueur : 1,5 pt ; validation du `select` : 1 pt ; vérification du radio coché : 1,5 pt ; RegExp téléphone exacte : 1,5 pt ; comptage et affichage du bilan (rouge/vert) : 1,5 pt ; soumission programmatique : 1 pt.

Points de vigilance : exiger les ancres `^` et `$` dans la RegExp (sinon `+237677889900123` serait accepté) ; le bilan d'erreur doit afficher le nombre exact d'erreurs ; la validation JavaScript ne dispense jamais d'une revalidation côté serveur dans `inscrire.php` (le JS est désactivable ou contournable) ; attention à ne pas écrire `verifier()` dans `addEventListener`.

14 FICHE BILAN

HTML / JS Rappels

Structure Page HTML

Formulaires HTML

- input, select, textarea
- label, fieldset, legend

Tableaux HTML

- table, thead, tbody
- tr, th, td

JavaScript & DOM

- document.querySelector
- textContent, value

Événements

- addEventListener
- submit, input, change

Validation Côté Client

- Regex, required
- preventDefault()

Carte mentale : Rappels HTML / JavaScript (Terminale TI)

FICHE BILAN L'essentiel à connaître pour l'examen

Définitions clés

- **DOM (Document Object Model)** : Représentation arborescente de la page HTML en mémoire ; chaque balise est un nœud manipulable par JavaScript.
- **Événement** : Action utilisateur (clic, saisie, envoi) ou système déclenchant l'exécution d'un gestionnaire (handler).
- **Validation côté client** : Contrôle immédiat dans le navigateur (UX, réactivité) ; ne dispense pas de la validation côté serveur (sécurité).
- **Attribut required** : Contrainte HTML5 native interdisant l'envoi si le champ est vide.

Syntaxes indispensables (à connaître par cœur)

Action	Code / Syntaxe
Sélection unique	<code>const el = document.querySelector('selecteurCSS');</code>
Sélection multiple	<code>const els = document.querySelectorAll('selecteurCSS');</code>
Modifier contenu texte	<code>el.textContent = 'Nouveau texte';</code>
Modifier valeur champ	<code>el.value = 'valeur';</code>
Écouter un événement	<code>el.addEventListener('type', fonctionHandler);</code>
Annuler envoi formulaire	<code>e.preventDefault();</code> dans le handler submit
Regex email simple	<code>/^[\\w .-]+@[\\w .-]+\\.\\w {2,}\$ /</code>
Tester regex	<code>regex.test(chaine)</code> renvoie true/false

Méthodes express

MÉTHODE Créer un formulaire HTML5 complet

1. **Balise** `<form action="traitement.php" method="POST" id="monForm">`.
2. Un `<label for="idChamp">` par champ (accessibilité).
3. **Champs** : `<input type="text|email|password|number|date" id="idChamp" name="nomChamp" required>`, `<select>`, `<textarea>`.
4. **Bouton** `<button type="submit">Valider</button>` (ou `<input type="submit">`).
5. **Fermer** `</form>`.

MÉTHODE Valider un formulaire en JS (pattern standard)

1. **Sélectionner le formulaire** : `const form = document.querySelector('#monForm');`
2. **Écouter submit** : `form.addEventListener('submit', (e) => { ... });`
3. Dans le handler : `e.preventDefault();` pour bloquer l'envoi natif.
4. **Récupérer les valeurs** : `const val = form.elements.nomChamp.value.trim();`
5. Tester chaque règle (vide, regex, longueur, correspondance).
6. Afficher les erreurs (ex : `span.erreur` ou `alert()`).
7. Si tout valide : `form.submit();` ou `fetch()` (AJAX).

À RETENIR Checklist avant le Bac

- ☐ Écrire la structure minimale d'une page HTML5 (`<!DOCTYPE html>`, `html`, `head`, `body`, `meta charset="utf-8"`).
- ☐ Construire un formulaire sémantique : label lié par `for/id`, `name` sur chaque champ, types HTML5 adaptés (`email`, `tel`, `number`).
- ☐ Structurer un tableau accessible : `<table>`, `<thead>`, `<tbody>`, `<th scope="col">`, `<td>`.
- ☐ Sélectionner un élément du DOM avec `document.querySelector()` (ID, classe, sélecteur CSS complexe).
- ☐ Lire/écrire la valeur d'un champ : `element.value` (`input/select/textarea`) vs

`element.textContent (div/span/p).`

- ☐ Attacher un gestionnaire d'événement avec `addEventListener('type', callback)` (pas `onclick=` dans le HTML).
- ☐ Utiliser l'objet `Event (e)` dans le `callback` : `e.target`, `e.preventDefault()`, `e.stopPropagation()`.
- ☐ Valider côté client : `required`, `pattern (regex)`, `min/max/step`, et JS personnalisé sur `submit`.
- ☐ Afficher un message d'erreur contextuel (créer/injecter un `<span class="erreur">` ou utiliser `setCustomValidity()`).
- ☐ Différencier validation client (UX) et validation serveur (sécurité : injection SQL, XSS, contournement JS).
- ☐ Déboguer avec la console (F12) : `console.log()`, `console.dir()`, onglet Network pour voir la requête POST.
- ☐ Respecter les conventions de nommage : `kebab-case` pour `id/name HTML`, `camelCase` pour variables JS.



Signé • OnBuch+

Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+